的数据分组交付给网络层。图11.19显示了该协议中的发送窗口。窗口中的着色时隙指的是那些乱序到达的帧，这些帧被交付给网络层前正在等待其相邻帧的到达。

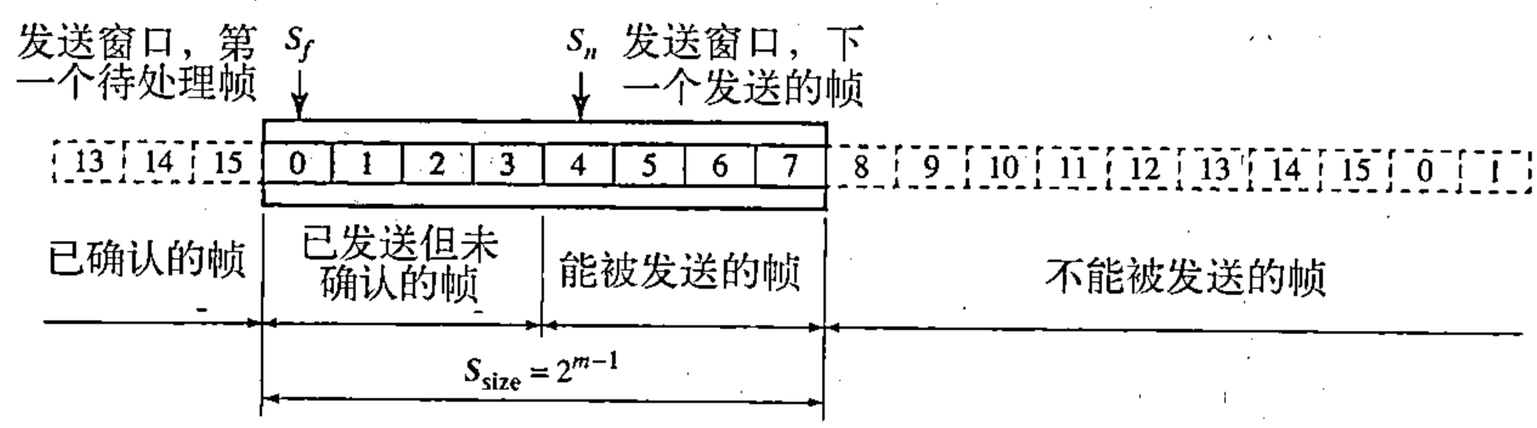


图11.18 选择性重复ARQ的发送窗口

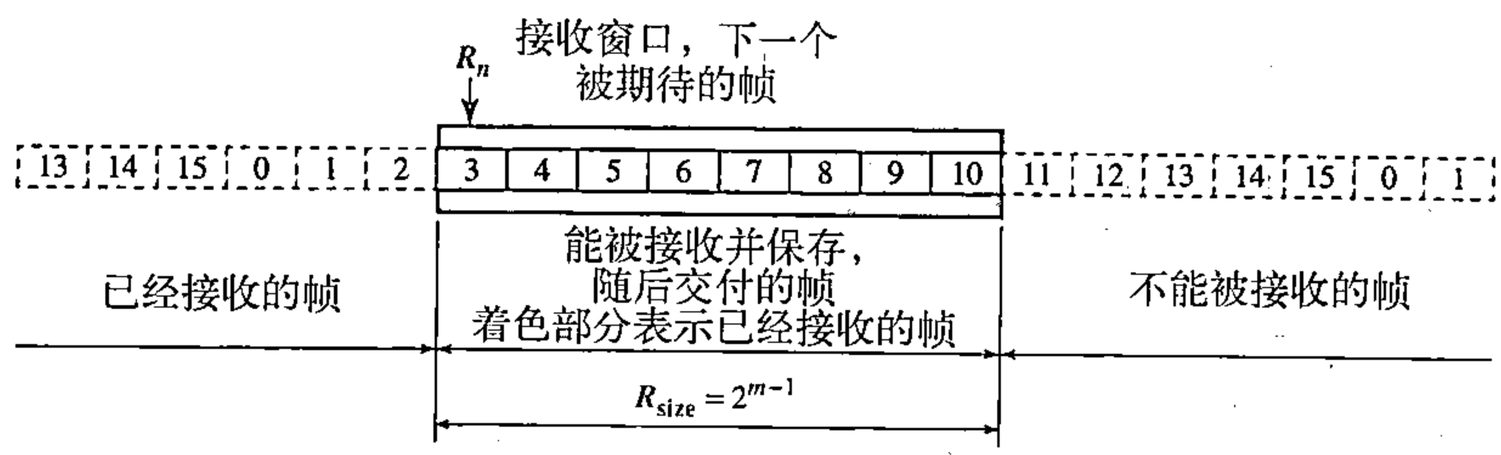


图11.19 选择性重复ARQ的接收窗口

设计

该协议的设计与我们描述的回退N帧协议的设计有一定程度的相似，但是正如图11.20所示，它要复杂的多。

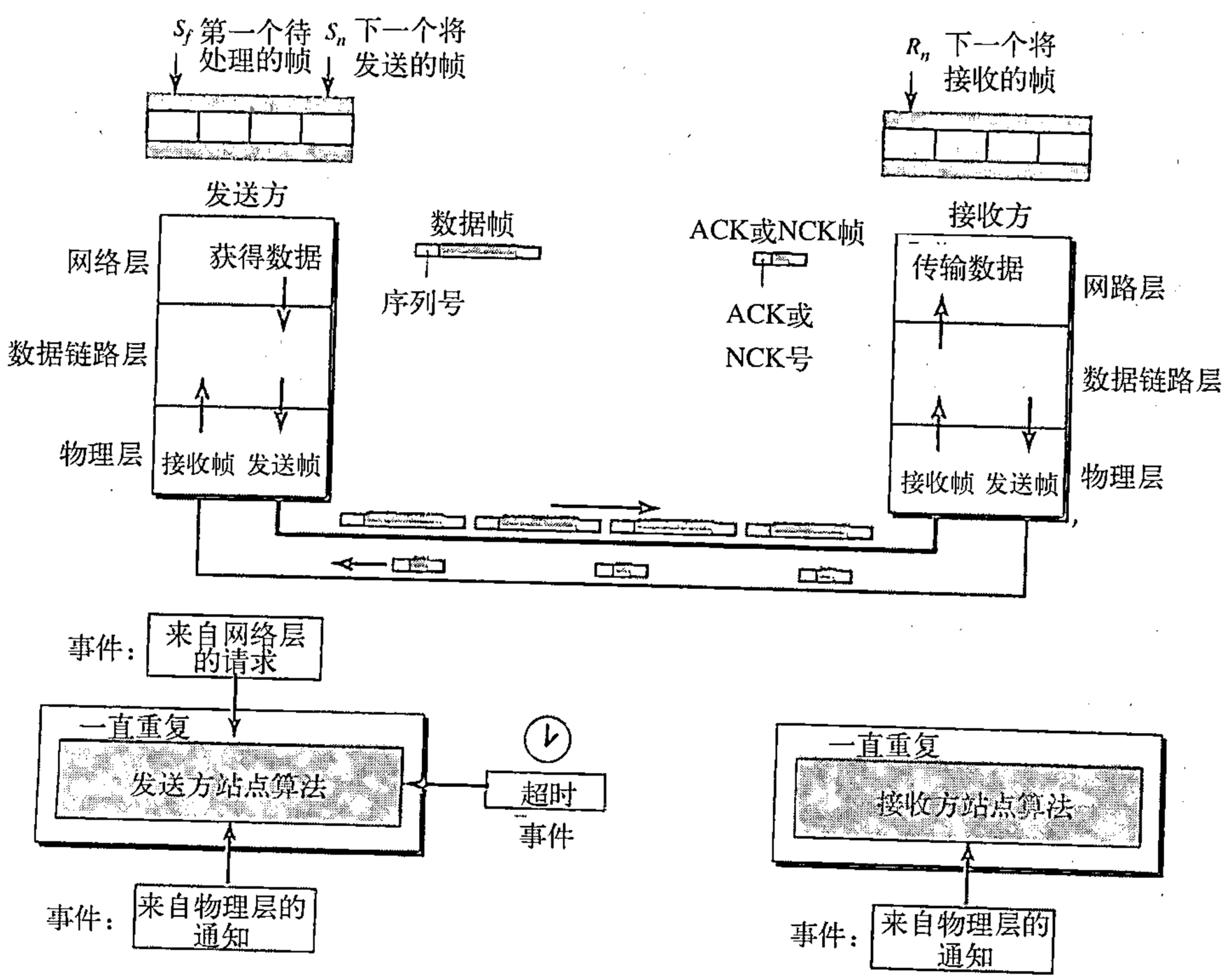
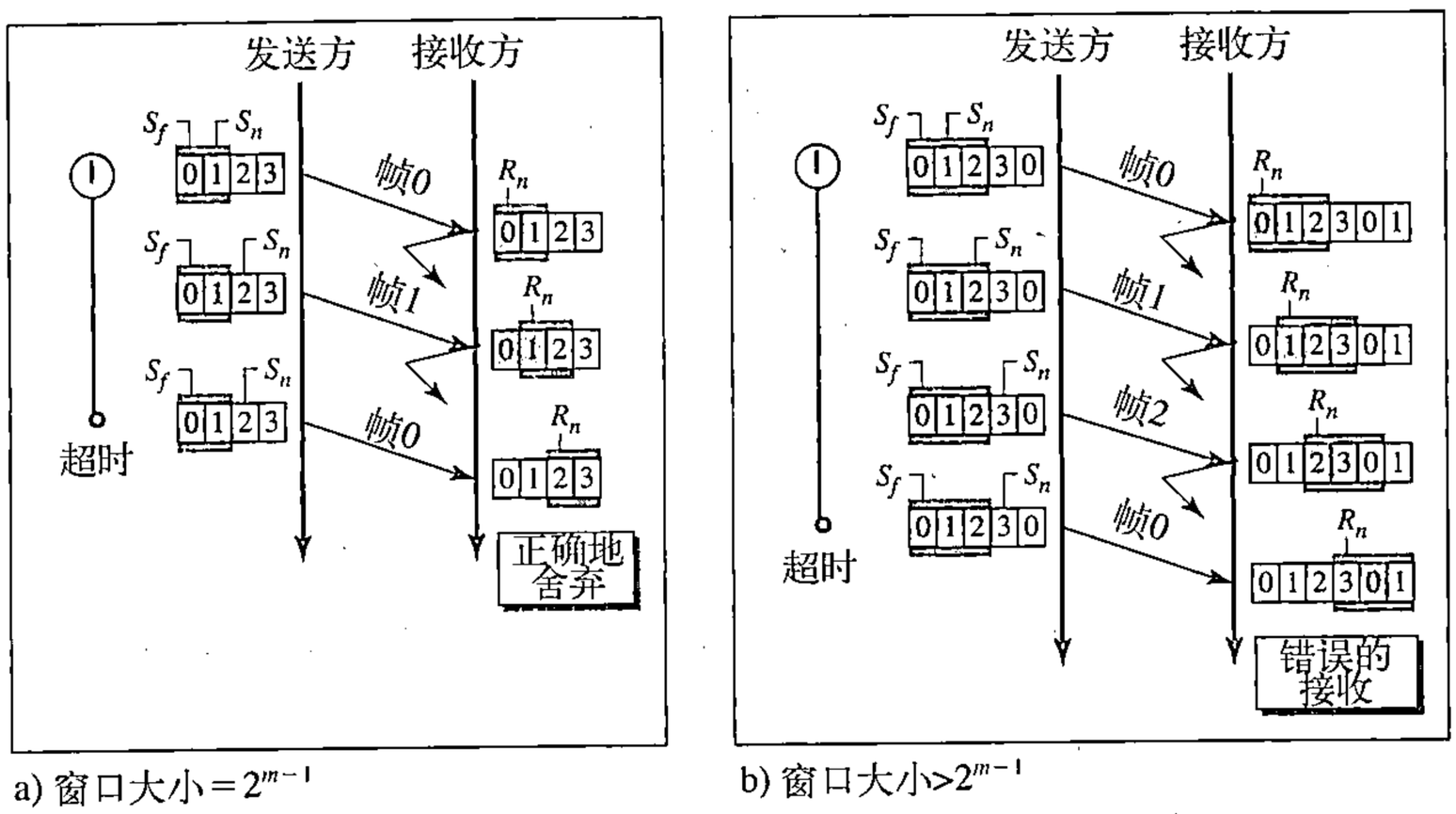


图11.20 选择性重复ARQ协议的设计

窗口大小

我们现在能解释为什么发送窗口和接收窗口的大小最多是2^m的一半了。举例来说，我们使$m=2$，意味着窗口的大小为$2^m/2$，即2。图11.21将大小为2的窗口与大小为3的窗口做了一个比较。

如果窗口的大小为2且所有的确认都丢失了，那么当帧0的定时器到时时，帧0就被重发。然而，接收方的窗口现在期待的是帧2而不是帧0，那么帧0的副本就会被正确地丢弃。当窗口大小为3且所有的确认丢失了时，发送方发送了一个帧0的副本。而这次，接收方窗口期待接收帧0（0是窗口的一部分），所以它接收帧0，但它是下一个周期的第一个帧，而不是一个副本。这明显是一个错误。



a) 窗口大小$=2^{m-1}$　　b) 窗口大小$>2^{m-1}$

图11.21　选择性重复ARQ中的窗口大小

> 在选择性重复ARQ中，发送方窗口和接收方窗口的大小最多只能是$2m$的一半。

算法

算法11.9描述了发送方的程序。

算法11.9　发送方选择性重复算法

```
Sw = 2^(m-1) ;
Sf = 0;
Sn = 0;

while (true)                    //不断重复
{
   WaitForEvent();
   if(Event(RequestToSend))     //有包要发送
   {
      if(Sn-Sf >= Sw)           //如窗口满
            Sleep();
      GetData();
      MakeFrame(Sn);
      StoreFrame(Sn);
      SendFrame(Sn);
      Sn = Sn + 1;
      StartTimer(Sn);
   }

   if(Event(ArrivalNotification)) //帧到达
   {
      Receive(frame);           //接收ACK或NAK
      if(corrupted(frame))
```

```
24            Sleep();
25       if (FrameType == NAK)
26          if (nakNo between Sf and Sn)
27          {
28           resend(nakNo);
29           StartTimer(nakNo);
30          }
31       if (FrameType == ACK)
32          if (ackNo between Sf and Sn)
33          {
34            while(sf < ackNo)
35            {
36              Purge(sf);
37              StopTimer(sf);
38              Sf = Sf + 1;
39            }
40          }
41    }
42
43    if(Event(TimeOut(t)))          //计时器超时
44    {
45     StartTimer(t);
46     SendFrame(t);
47    }
48 }
```

分析　请求事件的处理与前一协议相类似，除了每一个帧的发送都启动一个定时器外。这里到达的事件要复杂得多。到达的可能是一个ACK，也可能是一个NAK。如果一个有效的NAK帧到达了，我们只需重发相应的帧。如果一个有效的ACK到达了，我们使用一个循环来清除缓冲区，停止相应的定时器，并移动窗口的左墙。此处的超时事件比较简单，只有超时的帧才重发。

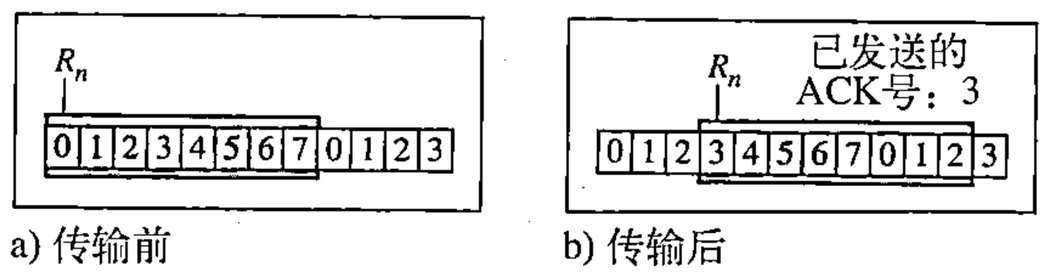
算法11.10描述了接收方的程序。

算法11.10　接收方选择性重复算法

```
 1 Rn = 0;
 2 NakSent = false;
 3 AckNeeded = false;
 4 Repeat(for all slots)
 5     Marked(slot) = false;
 6
 7 while (true)                          //不断重复
 8 {
 9   WaitForEvent();
10
11   if(Event(ArrivalNotification))      //数据帧到达
12   {
13      Receive(Frame);
14      if(corrupted(Frame))&& (NOT NakSent)
15      {
16       SendNAK(Rn);
17       NakSent = true;
18       Sleep();
19      }
20      if(seqNo <> Rn)&& (NOT NakSent)
21      {
22       SendNAK(Rn);
23       NakSent = true;
24       if ((seqNo in window)&&(!Marked(seqNo))
25       {
26        StoreFrame(seqNo)
27        Marked(seqNo)= true;
28        while(Marked(Rn))
29        {
30         DeliverData(Rn);
```

```
        Purge(Rn);
        Rn = Rn + 1;
        AckNeeded = true;
      }
        if(AckNeeded);
        {
        SendAck(Rn);
        AckNeeded = false;
        NakSent = false;
        }
    }
  }
}
```

分析　此处我们要有更多的初始化。为了不使另一端因为NAK而超负荷，我们使用了一个称为NakSent的变量。为了明确何时我们需要发送一个ACK，我们使用一个名为AckNeeded的变量。这两个变量都初始化为错误状态（false）。一旦相应的帧到达并被保存了，我们会用一系列变量来标记接收窗口中的时隙。如果我们收到一个损坏的帧而NAK帧还未被发送，我们就发送一个NAK去告知另一方我们还未收到我们期待的帧。如果帧没被损坏且序列号在窗口中，我们保存这个帧并在时隙上做标记。如果从R_n开始相邻的帧都被标记了，我们将它们的数据交付给网络层并滑动窗口。图11.22描述了这一情形。



a) 传输前　　b) 传输后

图11.22　在选择性ARQ中传输数据

例11.8

这个例子与例11.3相似，都是一个帧丢失了。它描述了选择性重复在这个情况下如何工作。图11.23 描述了这一情形。

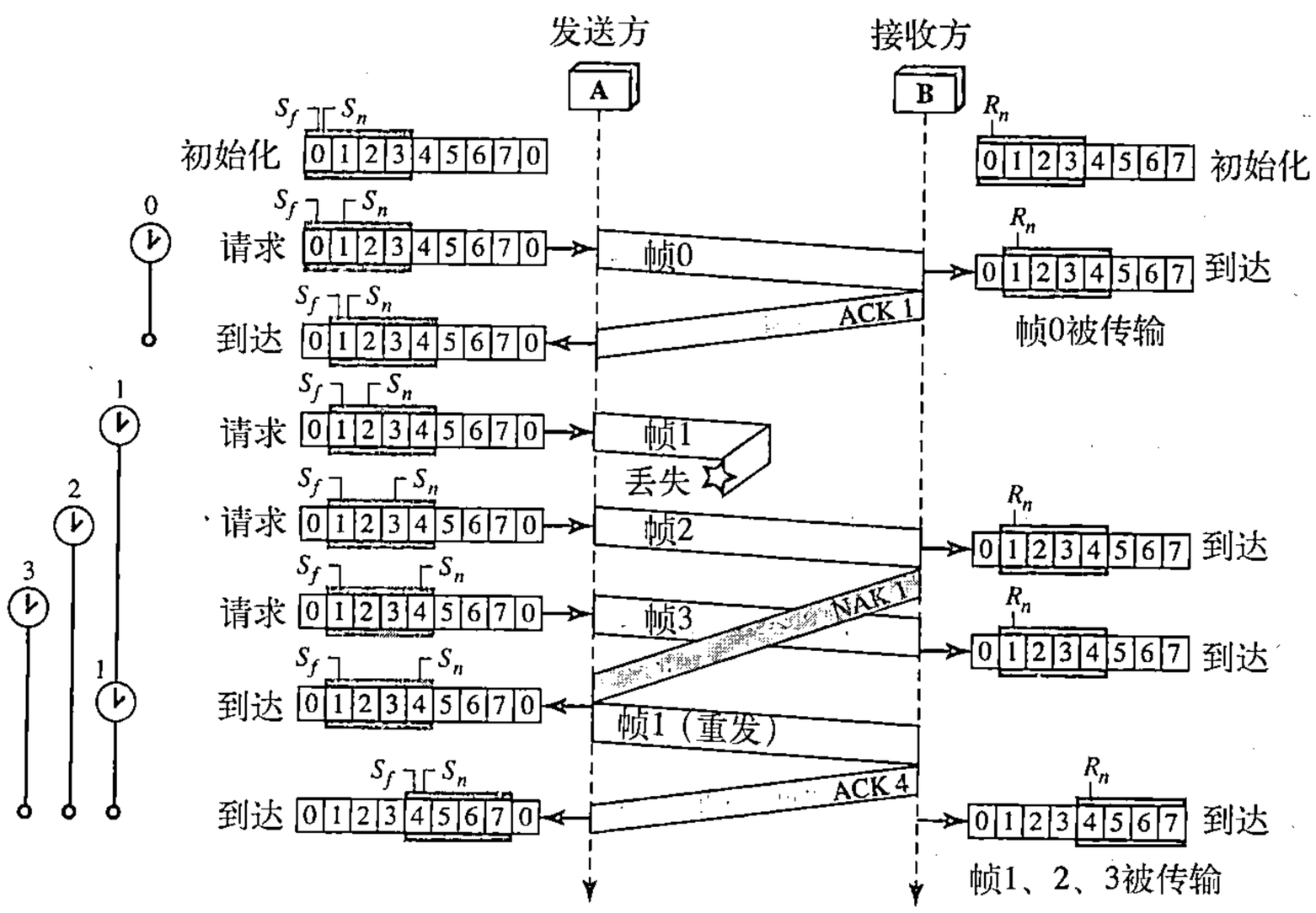


图11.23　例11.8的流量图

一个主要的区别是定时器的数量。这里，每发送或重发一个帧都需要一个定时器，这就意味着定时器也要编号（0、1、2和3）。在第一个请求时，帧0的定时器启动，并在这个帧的ACK到达时停止。第二个请求时，帧1的定时器启动，当NAK到达时再重启，直到最后的ACK到达时才最终停止。其他两个定时器在相应的帧发送时启动而在最后的到达事件发生时

停止。在接收方站点我们需要分辨帧的接收和向网络层交付帧。在第二次到达发生时，帧2到达了并被保存和标记了（着色时隙），但是它不能被交付因为帧1丢失了。下一个到达事件，帧3到达了并被保存和标记了，但是仍然没有帧能被交付。只有当最后一个到达事件发生，即帧1的副本到达了，帧1、2和3才能被交付给网络层。将帧交付给网络层有两个条件：第一，一组连续帧到达；第二，这组帧从窗口的起点开始。在第一个到达事件后，只有一个帧到达，它是从窗口的起点开始的。到最后一个到达事件后，有三个帧且第一个是从窗口的起点开始的。

另一个重要之处是在第二个到达事件后，一个NAK帧被发送了，但在第三个到达事件后没有发送NAK，虽然情况看上去一样。原因是这个协议不想让网络因不必要的NAK帧和不必要的重发帧而拥堵。第二个NAK帧仍然会是NAK1去提醒发送者再重发帧1，这已经由第一个NAK做过了。第一个NAK被发送是被记忆的（使用nakSent变量），不应该再发送直到帧滑动。每一个窗口位置只发送一个NAK并指明窗口中的第一个时隙。

接下来这点是关于ACK的。注意这里只发送了两个ACK。第一个ACK只确认了第一个帧，而第二个确认了三个帧。在选择性重复协议中，只有当数据被交付到网络层时才发送ACK。就算属于一个时隙的n个帧的数据被传输，也只发送一个ACK。

11.5.4 捎带

在这部分我们讨论的三个协议都是单向的：数据帧只朝一个方向流动而诸如ACK和NAK等的控制信息则能反方向传输。在现实生活中，数据帧通常可以双向流动：从节点A到节点B，或者从节点B到节点A。这就是说控制信息也要能双向传输。一项称之为**捎带技术**的运用提高了双向协议的效率。当一个帧从A到B运载数据时，它也能从B运载关于帧到达或丢失的控制信息，当一个帧从B到A运载数据时，它也能从A运载关于帧到达或丢失的控制信息。

我们在图11.24中描述了使用捎带的回退N帧ARQ的设计。注意每一个节点都有两个窗口：一个发送窗口和一个接收窗口。两者都需要使用定时器。两者都涉及三个类型的事件：请求，

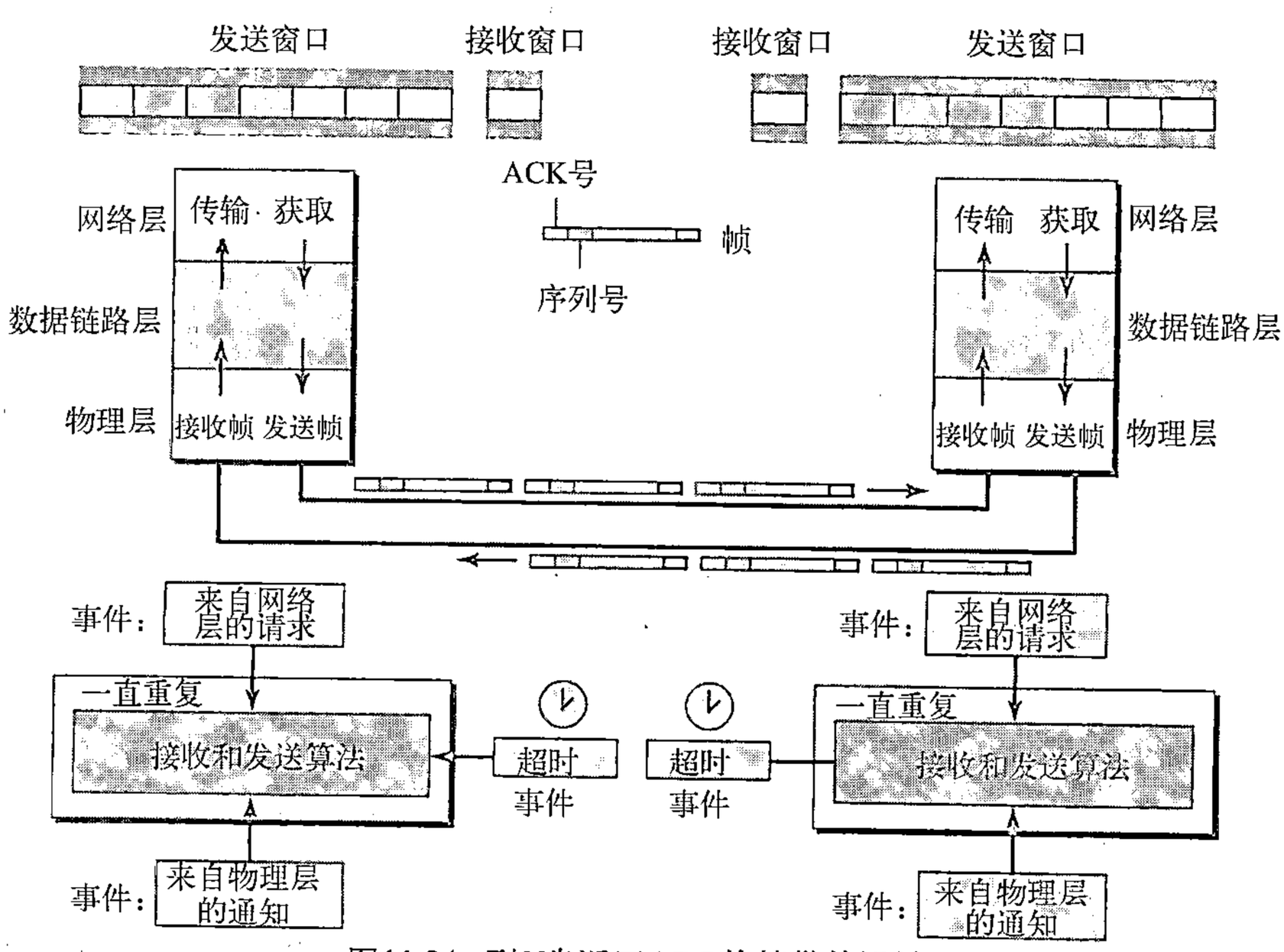


图11.24 到N步返回ARQ的捎带的设计

到达和超时。然而，此处的到达事件比较复杂，当一个帧到达时，每一端都要同时处理帧和控制信息。所以必须小心处理到达事件。请求事件只使用每一方的发送窗口，而到达事件既要用到发送窗口也要用到接收窗口。

关于捎带，另一重要之处是发送方和接收方要使用同一种算法。该算法是复杂的，因为它要将两个到达事件合而为一。我们将它留为一个练习。

11.6 高级数据链路控制

高级数据链路控制（HDLC）是一种面向位的点到点和多点链路进行通信的协议。它实现了在本章中讨论的ARQ机制。

11.6.1 配置和传输方式

HDLC提供了在不同的配置中都能使用的两种通用的传输模式：**正常响应方式**（normal response mode，NRM）和**异步平衡方式**（asynchronous balanced mode， ABM）。

正常响应方式

在正常响应方式中，站点配置是不平衡的。有一个**主站**（primary station）和多个**从站**（secondary station）。主站能发送指令，而从站只能响应。如图11.25所示，NRM能在点到点和多点链路中使用。

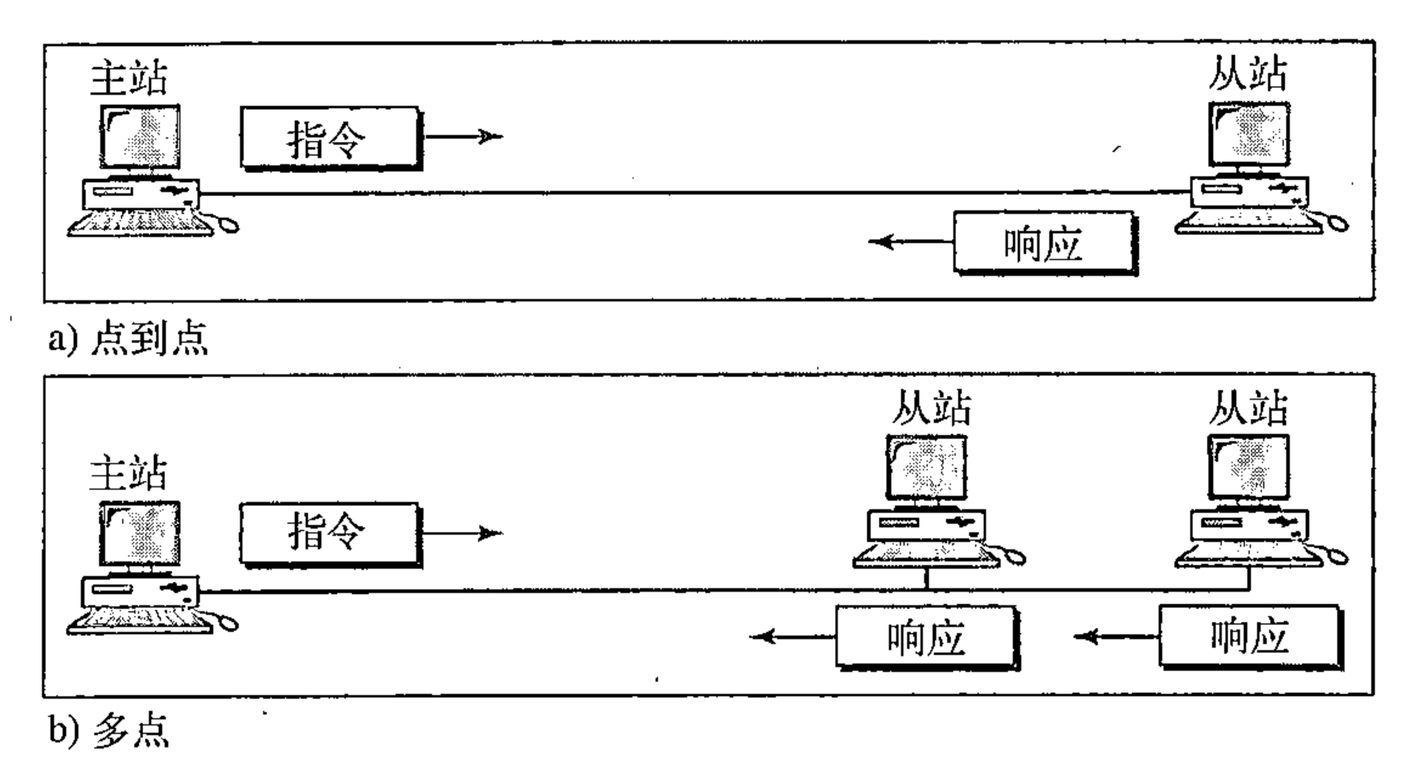


图11.25　正常响应方式

异步平衡方式

在异步平衡方式（AMB）中，配置是平衡的。如图11.26所示，该链路是点到点的，且每个站点既有主站的功能又有从站的功能（作为对等站点）。这是现在普遍应用的方式。

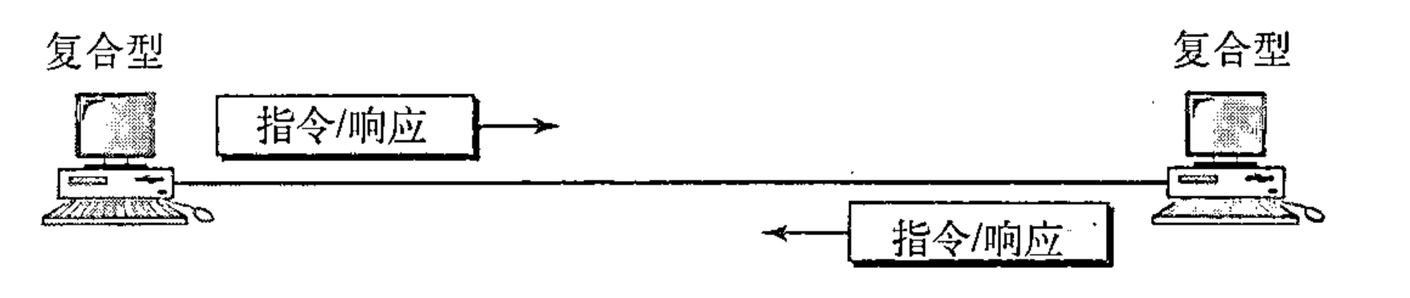


图11.26　异步平衡方式

11.6.2 帧

为了对刚阐述的方式和配置中所有可能的选择提供灵活性的支持，HDLC定义了三种类型的帧：**信息帧**（information frames，I-frames）、**管理帧**（supervisory frames，S-frames）和**无编号帧**（unnumbered frames，U-frames）。每一种类型的帧都在不同类型的信息传输中扮演信

封的角色。信息帧用来传输用户数据及与用户数据相关的控制信息（捎带）。管理帧只用来传输控制信息。保留无编号帧作为系统管理之用。无编号帧携带的信息用来管理链路自身。

帧的格式

在HDLC中，每个帧都最多可包含六个字段：如图11.27所示：开始标记字段，地址字段，控制字段，信息字段，帧校检序列字段，结束标记字段。在多帧传输中，一个帧的结束标记可以作为下一个帧的开始标记。

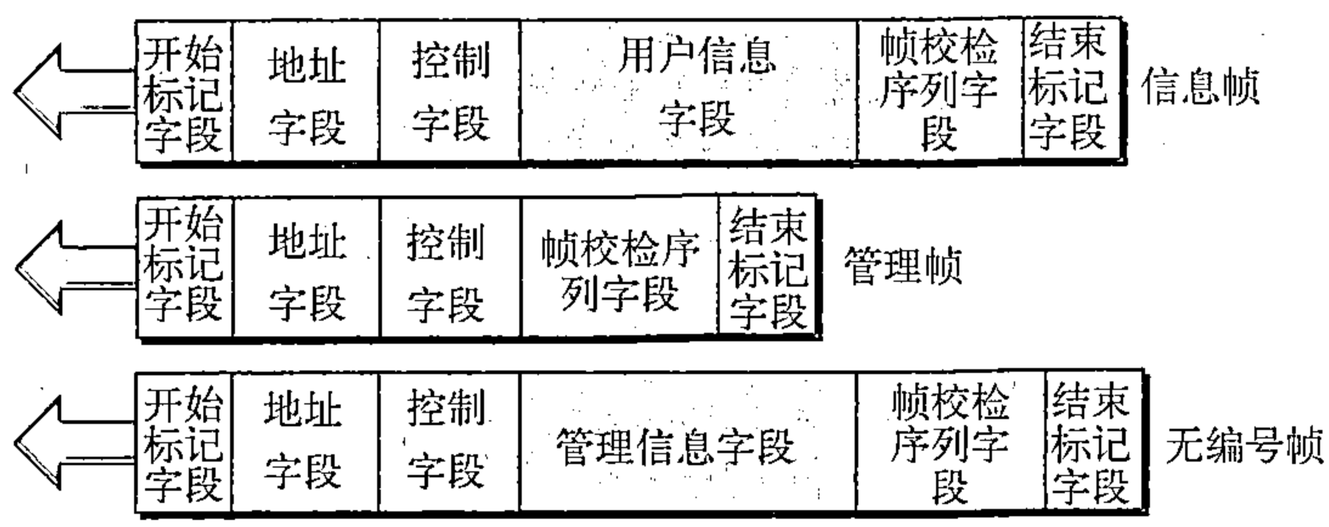


图11.27 HDLC的帧

字段

现在我们来讨论各个字段及它们在不同类型的帧中的作用。

- **标记字段** 一个HDLC帧的标记字段是8位序列，其模式是01111110，既可作为一个帧的开始也可以作为一个帧的结束，为接收方提供同步模式。
- **地址字段** 一个HDLC帧的第二个字段包含了从站的地址。如果主站产生帧，它包含着一个“去往”地址。如果从站产生帧，它则包含一个“来自”地址。一个**地址字段**可能是一个字节也可能是多个字节，这取决于网络的需求。1字节可以最多定义128个站点（1个位用于其他目的了）。更大的网络需要多字节地址字段。如果地址字段只有一个字节，最后的1位通常是1。如果地址字段多于一个字节，所有的字节除了最后一个字节外都以0结束，只有最后一个字节以1结束。中间的字节都以0结束是为了提示接收方会有更多的地址字节到来。
- **控制字段** 控制字段是帧的1个字节或2个字节段，用于流量控制和差错控制。这个字段中位的解释取决于帧的类型。我们将以后讨论这种字段及它在每种类型的帧中的格式。
- **信息字段** 信息字段包含了来自网络层的用户数据或者管理信息。不同的网络该字段的长度不同。
- **帧校验序列字段** 帧校验序列字段（FCS）是HDLC的差错校验字段。它可以包含2到4字节的ITU-T CRC。

11.6.3 控制字段

控制字段决定帧的类型及其功能。因此，让我们更为详细地阐述这种字段的格式。其格式对于一种类型的帧是特定的，如图11.28所示。

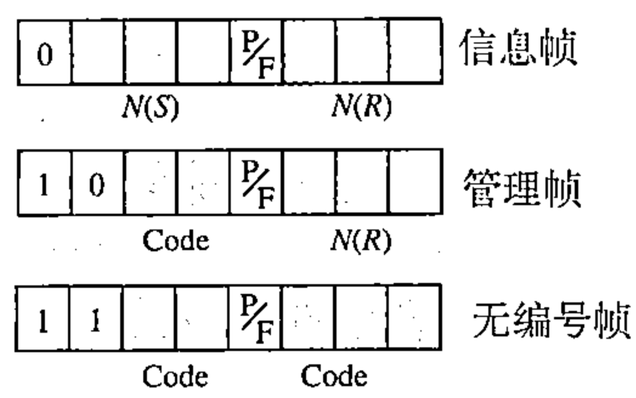


图11.28 不同类型帧的控制字段

信息帧的控制字段

信息帧被设计用来运载来自网络层的用户数据。另外，它们也能包含流量控制和差错控制信息（捎带）。控制字段的子字段用来定义这些功能。第一个位决定类型。如果控制字段的第一个位是0，这就意味着这是一个信息帧。接下来的三个位，

称为$N(S)$，定义帧的序号。注意通过这三个字节，我们能定义的序号是从0到7；但是在控制字段为2字节的扩展格式中，这部分也会大得多。最后的三个位，称为$N(R)$，在使用捎带时与确认号相对应。在$N(S)$和$N(R)$之间的单个位称之为P/F位。P/F虽是一个单个位，但是有双重用途。它只有被设定时（位＝1）才有意义，意味着轮询或终止。当帧由主站发送到从站时（即地址字段包含接收方的地址），它意味着轮询。当帧由从站向主站发送时，它意味着（即地址字段包含发送方的地址）终止。

管理帧的控制字段

无论当捎带是不可能的还是不恰当时（例如，当站点没有自身的数据要发送时，或者站点需要发送一个指令或响应而不是确认时），管理帧都用来进行流量控制和差错控制。管理帧没有信息字段。如果控制字段的前两个位是10，就说明这是一个管理帧。最后三个位，称之为$N(R)$，与确认（ACK）号或否定确认（NAK）号相对应，它取决于管理帧的类型。称为编码的2个位用来决定管理帧自身的类型。使用2个位，我们能产生4种类型的管理帧，如下所述：

- **准备接收（RR）**。如果编码的子字段是00，这就是一个RR管理帧。这种帧对一个或一群安全接收的帧加以确认。在这种情况下，$N(R)$的值定义了确认号。
- **不准备接收（RNR）**。如果编码的子字段是10，这就是一个RNR管理帧。这是一类有附加功能的RR帧。它对一个或一群帧的接收加以确认，并宣布接收方正忙不能接收更多的帧。它通过要求发送方减速来实现拥堵控制机能。$N(R)$的值是确认号。
- **拒收（REJ）**。如果编码的子字段是01，这就是一个REJ管理帧。这是一个NAK帧，但与适用于选择性重复ARQ协议中的不同。它是一个能在回退N帧ARQ协议中使用的NAK，它能在发送方计时到时前，告知发送方最后一个帧丢失或者损坏了，以此来提高程序的效率。$N(R)$的值是否定确认号。
- **选择性拒收（SREJ）**。如果编码的子字段是11，这就是一个SREJ的管理帧。这是一个适用于选择性重复ARQ协议的NAK帧。注意，在HDLC协议中使用术语*选择性拒收*而不是*选择性重复*。$N(R)$的值是否定确认序号。

无编号帧的控制字段

无编号帧负责在连接的设备之间交换会话管理和控制信息。与管理帧不同，无编号帧包含一个信息字段，但是这个信息字段是用作系统管理的，而不是用户数据。虽然与管理帧有相似之处，但无编号帧承载的多数信息包含在含有控制字段中的编码中。无编号帧编码被分成两部分：在P/F位前有一个2位的前缀，在P/F位后有一个3位的后缀。总的而言，这两部分（5位）共能产生32种不同类型的无编号帧。其中常用的一些无编号帧如表11.1所示。

表11.1　无编号帧的指令和响应

编　码	指　令	响　应	含　义
00 001	SNRM		设为正常响应方式
11 011	SNRME		设为正常响应方式，扩展的
11 100	SABM	DM	设为异步平衡方式或断开连接方式
11 110	SABME		设为异步平衡方式，扩展的
00 000	UI	UI	无编号信息
00 110		UA	无编号确认
00 010	DISC	RD	断开连接或请求断开连接
10 000	SIM	RIM	设为初始化方式或请求信息方式
00 100	UP		无编号轮询

（续）

编 码	指 令	响 应	含 义
11 001	RSET		重置
11 101	XID	XID	交换ID
10 001	FRMR	FRMR	拒收帧

例11.9 连接/断开

图11.29描述了如何使用无编号帧来建立连接和断开连接。节点A通过一组异步平衡方式（SABM）帧来请求连接；节点B通过一个无编号确认（UA）帧给出一个肯定的响应。通过这两个交换，数据能在两个节点之间传输（图中并未显示）。数据传输完毕，节点A发送一个DISC（断开）帧请求断开连接；节点B通过一个UA（无编号确认）帧响应加以确定。

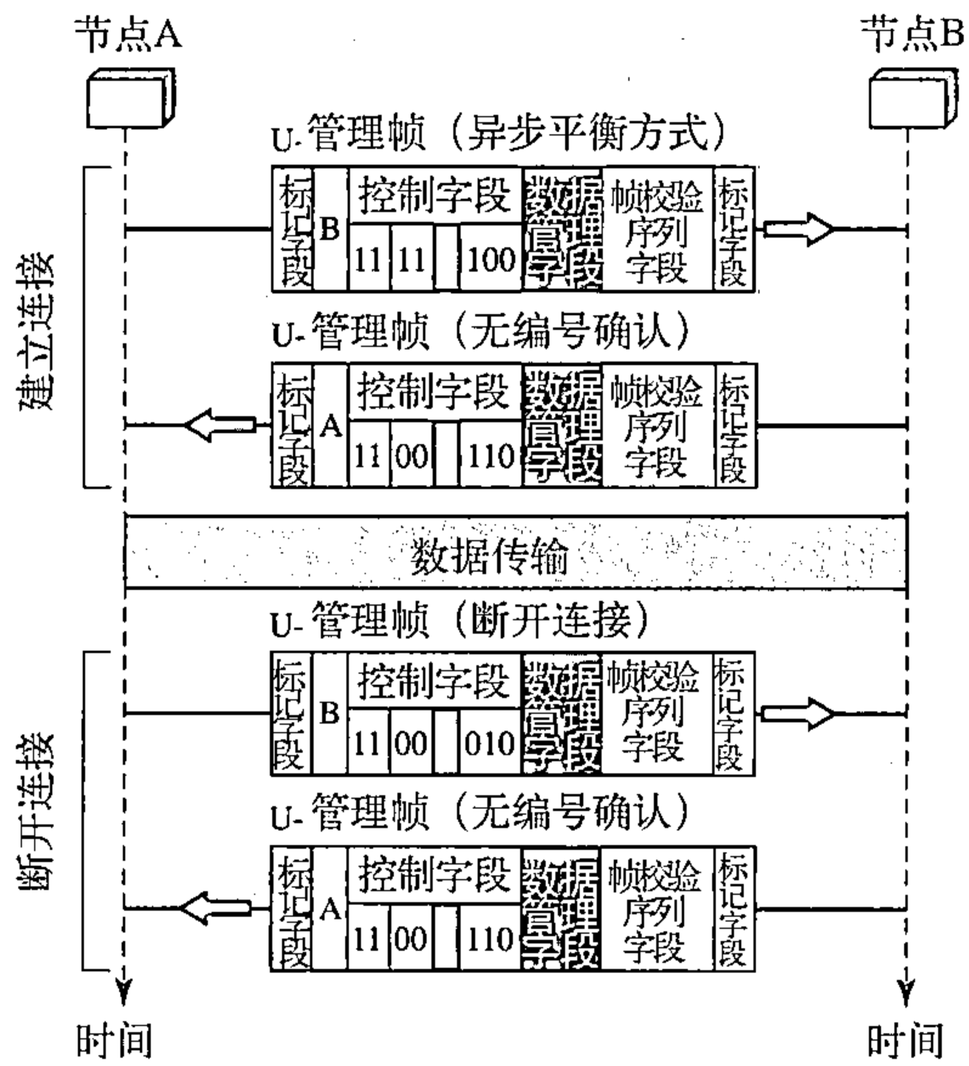


图11.29 建立连接和断开的范例

例11.10 无差错捎带

图11.30描述了一个使用捎带的交换。节点A通过一个编号为0及紧随其后的编号为1的信息帧来开始交换。节点B发送自身的一个信息帧并捎带以上两个帧的确认。节点B的第一个信息帧同样编号为0[$N(S)$字段]并在其$N(R)$字段有一个2，以此来确认对来自节点A的帧0和1的接收，并提示接下来它等待帧2的到达。在从节点A接收更多的帧之前，节点B传输其第二个和第三个信息帧（编号1和2）。它的$N(R)$信息并没有因此变化：B的帧1和2提示节点B仍然等待节点A的帧2的到达。节点A发送了它所有的数据。因此，它不能在一个信息帧上捎带上确认，故而发送一个管理帧以代之。编码RR指明节点A仍然处于准备接收状态。$N(R)$字段的数字3告知B帧0、1和2已经被接收且A现在正等待的是帧3。

例11.11 有差错的捎带

图11.31描述了丢失了一个帧的交换。节点B发送了数据帧（0、1、2），但是帧1被丢失了。当节点A收到帧2，它丢弃了帧2并为帧1发送了一个拒收（REJ）帧。注意这是一个使用回退N帧的协议，因此一个REJ帧有着如同NAK帧的特殊用途。在这里NAK要做两件事：它对帧0的接收加以确定并宣布帧1及其后的任何帧都必须重发。收到REJ帧后，节点B重发了帧1和2。

节点A通过发送一个确认号为3的RR帧（ACK帧）对接收加以确认。

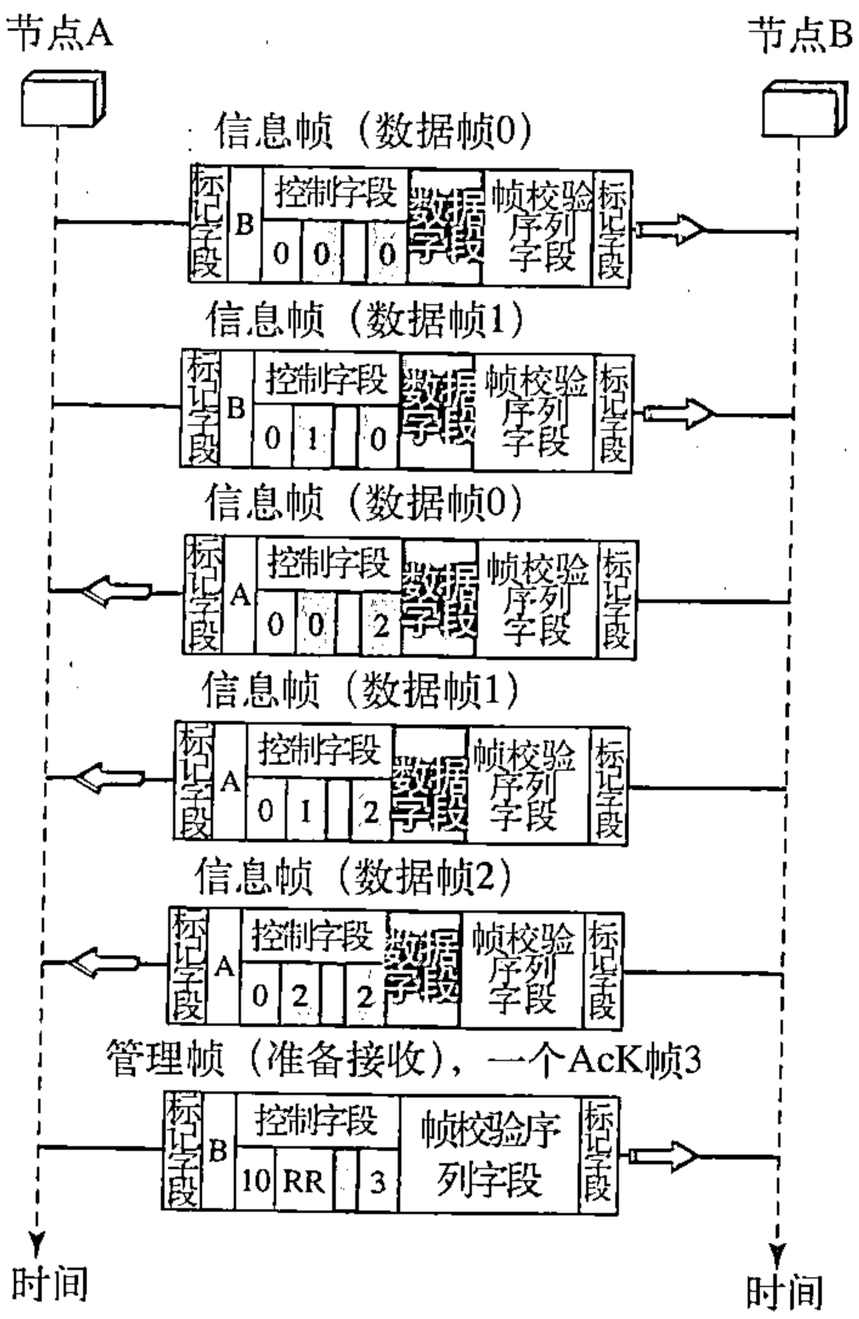


图11.30　无差错捎带的示例

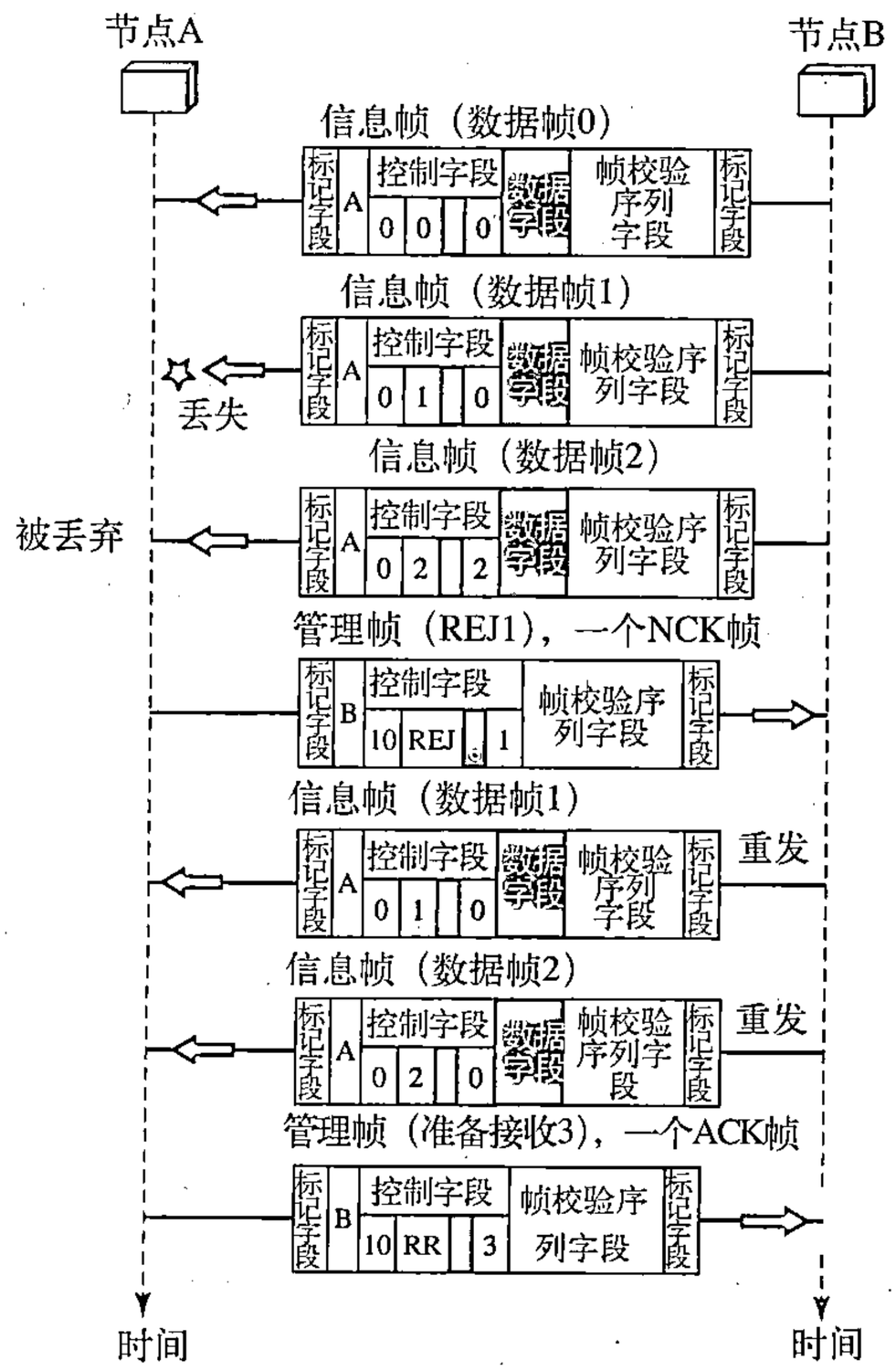


图11.31　有差错捎带的示例

11.7 点到点协议

虽然高级数据链路控制协议是点到点和多点配置都能使用的一个通用协议，但点到点访问最通用的协议还是**点到点协议**（Point-to-Point Protocol ， PPP）。如今，成千上万的因特网用户需要通过PPP来将他们的家庭电脑与因特网服务提供商的服务器连接起来。这些用户中的大多数有一个额外的调制解调器；通过提供物理层服务，它们通过电话线与因特网连接。但是，为了数据传输的控制和管理，在数据链路层也需要一个点到点协议。到目前为止，PPP是最通用的协议。

PPP提供以下服务：

1. PPP决定了设备间交换的帧的格式。
2. PPP决定了两台设备如何为链路的建立和数据的交换达成一致。
3. PPP决定了网络层数据如何封装到数据链路帧中。
4. PPP决定了两台设备如何互相鉴别。
5. PPP通过支持一系列不同的协议来提供多个网络层服务。
6. PPP提供多链路连接。
7. PPP提供网络地址配置。当一个家庭用户在与因特网连接需要一个临时地址时尤其有用。

另一方面，为了使PPP尽量简化，有些功能被删除了：

1. PPP不提供流量控制。发送方能一个接着一个地发送帧，而不关心接收方是否超负荷。
2. PPP对差错控制有一个很简单的机制。一个CRC字段用来检测差错。如果帧被损坏了，它便被无响应地丢弃；上层协议必须解决这一问题。缺乏差错控制和顺序编号，这可能会导致数据包被乱序接收。
3. 在多点配置中，PPP无法提供高级的地址分配机制来处理帧。

11.7.1 成帧

PPP是一个面向字节的协议。根据本章开始部分讨论的面向字节协议实现成帧。

帧的格式

图11.32显示了一个PPP帧的格式。每个字段的描述如下：

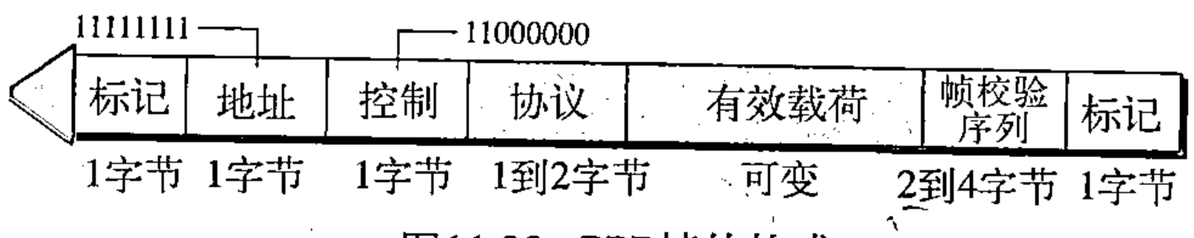


图11.32 PPP帧的格式

- 标记。一个PPP帧分别以位模式为01111110的一个字节标记开始和结束。虽然这个位模式与在HDLC中使用的相同，但是仍然存在一个很大的区别，即：PPP是一个面向字节的协议，而HDLC是一个面向位的协议。这个标记作为字节处理，我们随后会加以解释。
- 地址。在这个协议中地址字段是常量11111111（广播地址），协商时双方可同意省略这个字节。
- 控制。这个字段被设定为常量11000000（效仿HDLC中的无编号帧）。正如我们随后会阐述的那样，PPP不提供任何流量控制。关于差错检测方面的差错控制也是有限的。这就意味着这个字段是根本不需要的，而且在协商时，双方能达成一致忽略这个字节。
- 协议。协议字段指明了数据字段承载着什么：用户数据或者其他信息。我们将简短但详细地阐述这个字段。这个字段被默认为2个字节，但是双方可以同意只用一个字节。
- 有效载荷字段。这个字段承载着用户数据或其他信息，我们将简短的加以讨论。数据字

段中有一系列的字节，最大数量的默认值是1 500字节；但是通过协商这是可以变化的。如果标记字节模式出现在数据字段的话，数据字段就是字节填充（转义）的。因为没有字段来界定数据字段的大小，如果数据字段的大小小于默认的最大值或者协商的最大值的话，就需要填充了。

- 帧校验序列。帧校验序列是简单的2到4字节标准CRC。

字节填充

正如我们之前讨论的那样，PPP和HDLC的帧格式的结尾有相似之处，但是前者是面向字节的协议，这点完全区别于后者。作为一个面向字节的协议，PPP的标记是一个字节，而且只要它出现在帧的数据部分，就需要被转义。转义字节是01111101，表明每次类似标记模式的字节出现在数据部分，都要填充这个额外的字节来告知接收方接下来的字节并不是标记。

PPP是一个面向字节的协议，通过转义字节01111101来进行字节填充。

11.7.2 传输阶段

一个PPP连接可以通过**转换阶段**（transition phase）图来说明（见图11.33）。

- **闲置**。在闲置阶段，链路没有被使用。没有活动的载体（在物理层）且线路是静默的。
- **建立**。当一个节点开始通信，连接就进入这个阶段。在这个阶段，双方就选项进行协商。如果协商成功，系统就进入鉴别阶段（如果鉴别是需要的话）或者直接进入联网阶段。链路控制协议信息分组的应用就是以此为目的的。此处交换了一些信息包。
- **鉴别**。鉴别阶段是可选的；两个节点在建立阶段可以决定是否跳过该阶段。然而，如果它们决定使用鉴别，它们会发送一些鉴别信息分组，我们以后会加以讨论。如果结果是成功的，连接进入联网阶段，否则，它将进入终止阶段。
- **联网**。在联网阶段，将进行网络层协议的协商。因为PPP在网络层是支持多协议的，故PPP规定在网络层的数据进行交换之前，节点双方必须达成一个网络协定。如果一个节点在网络层同时运行多个协议的话，接收方需要知道哪个协议用来接收数据。
- **打开**。在打开阶段，将进行数据传输。当一个连接到了这个阶段，数据分组的交换就可以开始了。在这个阶段，连接将持续直到其中一个终端希望终止这次连接为止。
- **终止**。在终止阶段，终止连接。两端交换一些分组以消除和关闭链路。

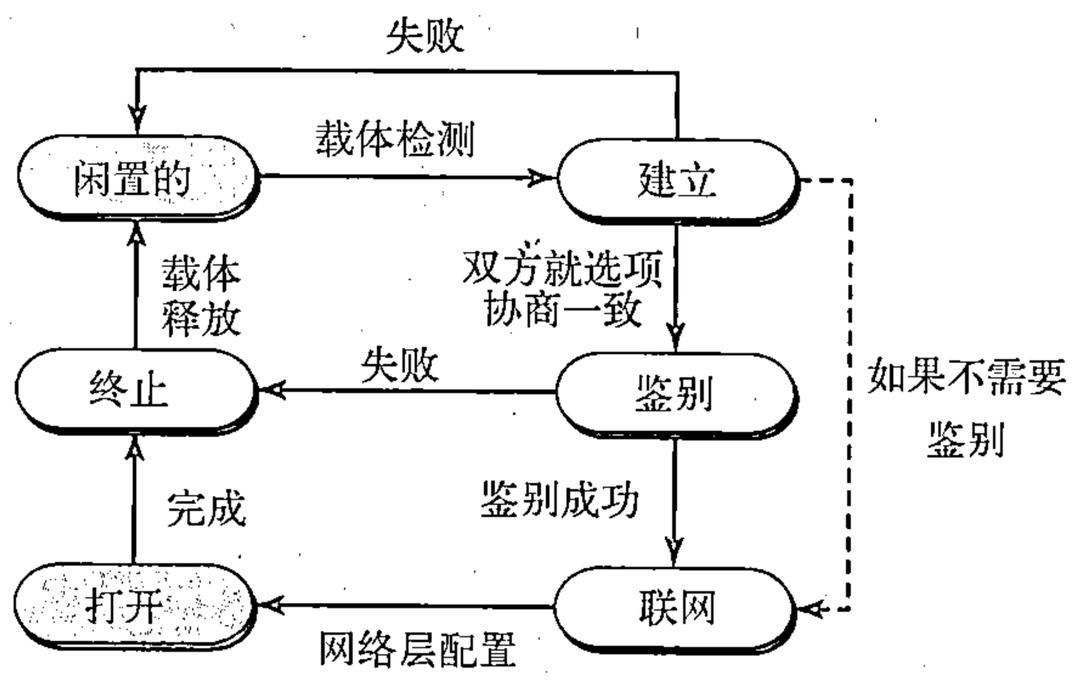


图11.33　转换阶段

11.7.3 多路复用

虽然PPP是一项数据链路层协议，但PPP也使用其他一组协议来建立链路，对涉及的参与者加以鉴别，并承载网络层数据。定义了三组协议使得PPP更为强大：链路控制协议（LCP）、两个鉴别协议（AP）和多个网络控制协议（NCP）。任何时刻，一个PPP信息分组都能在它的数据字段中携带来自这些协议的数据，如图11.34所示。注意，图中有一个链路控制协议、两个鉴别协议和多个网络控制协议。数据也可能来自不同的网络层。

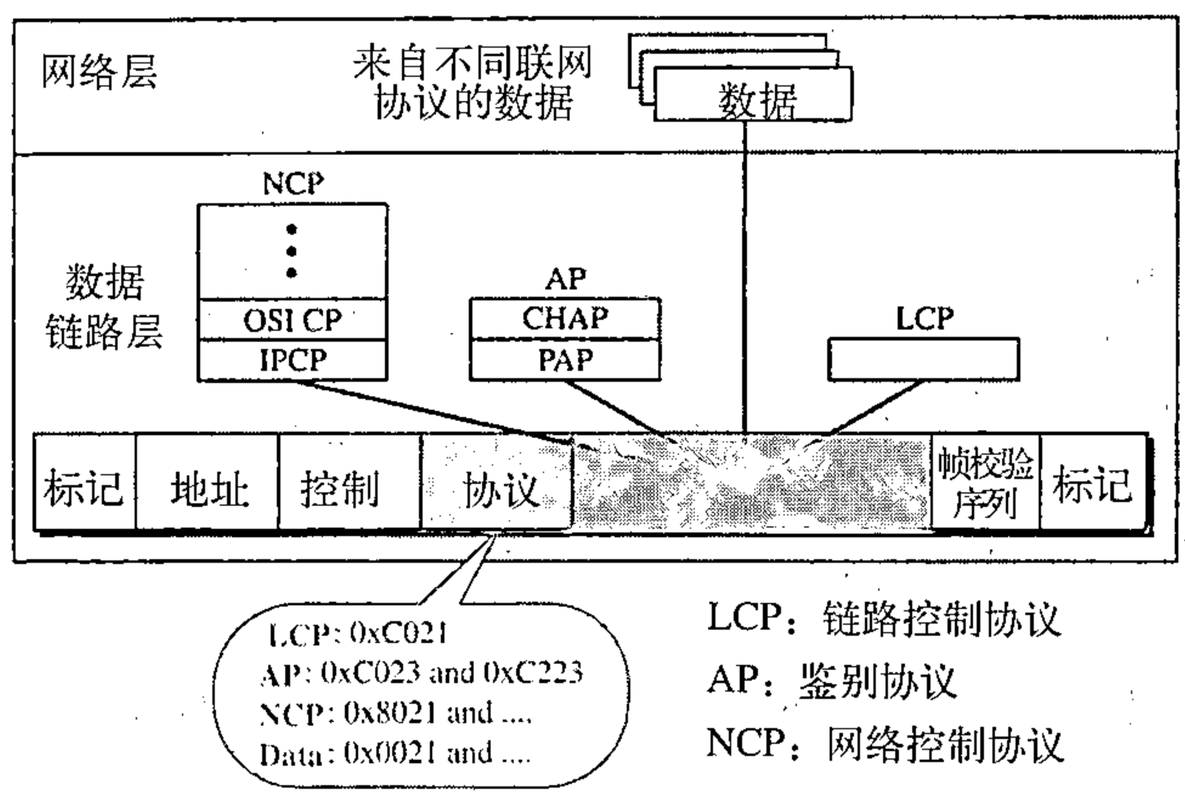


图11.34 PPP中的多路复用

链路控制协议

链路控制协议（Link Control Protocol，LCP）的职责是建立、维系、确认并终止链路。它还提供协商机制来设置两个端点间的选项。在链路建立之前，链路的两个端点必须就选项达成一个协定。见图11.35。

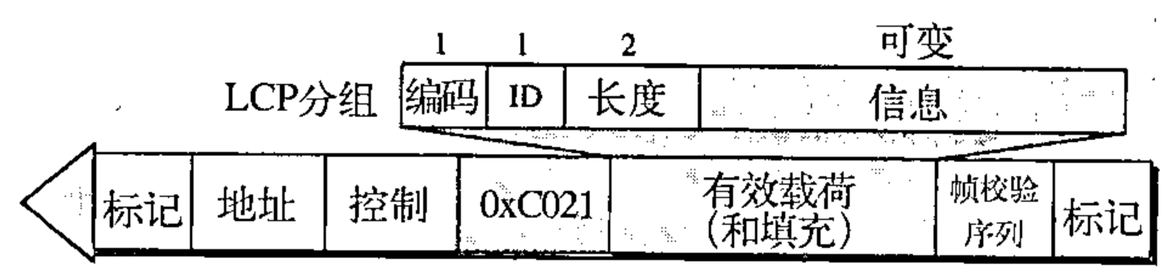


图11.35 封装在一个帧中的LCP分组

所有的LCP分组都位于PPP帧的有效载荷字段中，该帧的协议字段被设定为十六进制的C021。

编码字段决定了LCP分组的类型。如表11.2所示，共有11种类型的信息分组。

表11.2 LCP分组

编 码	分组类型	说 明
0x01	配置请求	包含建议选项及其值的列表
0x02	配置确认	接受所有建议的选项
0x03	配置不确认	告知某些选项不能被接受
0x04	配置拒绝	告知某些选项不可识别
0x05	终止请求	请求关闭线路
0x06	终止确认	接受关闭请求
0x07	编码拒绝	告知一个未知编码
0x08	协议拒绝	告知一个未知协议
0x09	回声请求	检测另一端是否活动的一种呼叫报文
0x0A	回声应答	对回声请求报文的响应
0x0B	丢弃请求	丢弃分组的请求

分组可以分为三类。第一类，包括前四种分组类型，用于建立阶段的链路配置。第二类，包括类型5和6，在终止阶段用于链路终止。后五种分组用于链路监控和调试。

ID字段持有一个将请求和应答相匹配的值。一个端点将一个值插入这个字段，这个值便被复制到应答分组中。长度字段决定了整个LCP分组的长度。为了满足某些LCP分组的需求，信息字段包含诸如选项等信息。

在两个端点之间有许多能被协商的选项。选项插入配置分组的信息字段。这种情况下，信

息字段被分为三段：选项类型，选项长度和选项数据。表11.3列示了其中最常用的一些选项。

鉴别协议

在PPP中鉴别具有很重要的作用，因为PPP被设计用于拨号链路中，这就使得用户身份的鉴别十分必要。**鉴别**（Authentication）意味着使一个需要访问一系列资源的用户的身份合法化。PPP有两种鉴别协议：口令鉴别协议和查询握手鉴别协议。注意这两种协议均用于鉴别阶段。

表11.3　常用选项

选　项	默认值
最大接收单位（有效载荷字段大小）	1 500
鉴别协议	无
协议字段压缩	关闭
地址和控制字段压缩	关闭

PAP　**口令鉴别协议**（Password Authentication Protocol，PAP）是一个两步骤的简单鉴别过程。

1. 需要访问系统的用户发送一个鉴别身份（通常是用户名）和一个口令。

2. 系统检测身份和口令的合法性，并选择接收连接或拒绝连接。

图11.36显示了PAP使用的三种类型的分组以及它们如何交换。当一个PPP帧承载PAP分组时，协议字段的值是0xC023。这三个PAP分组分别是鉴别-请求，鉴别-确认和鉴别-不确认。第一个分组被用户用来发送用户名和口令。系统用第二个分组来表示准许访问。系统用第三个分组来表示拒绝访问。

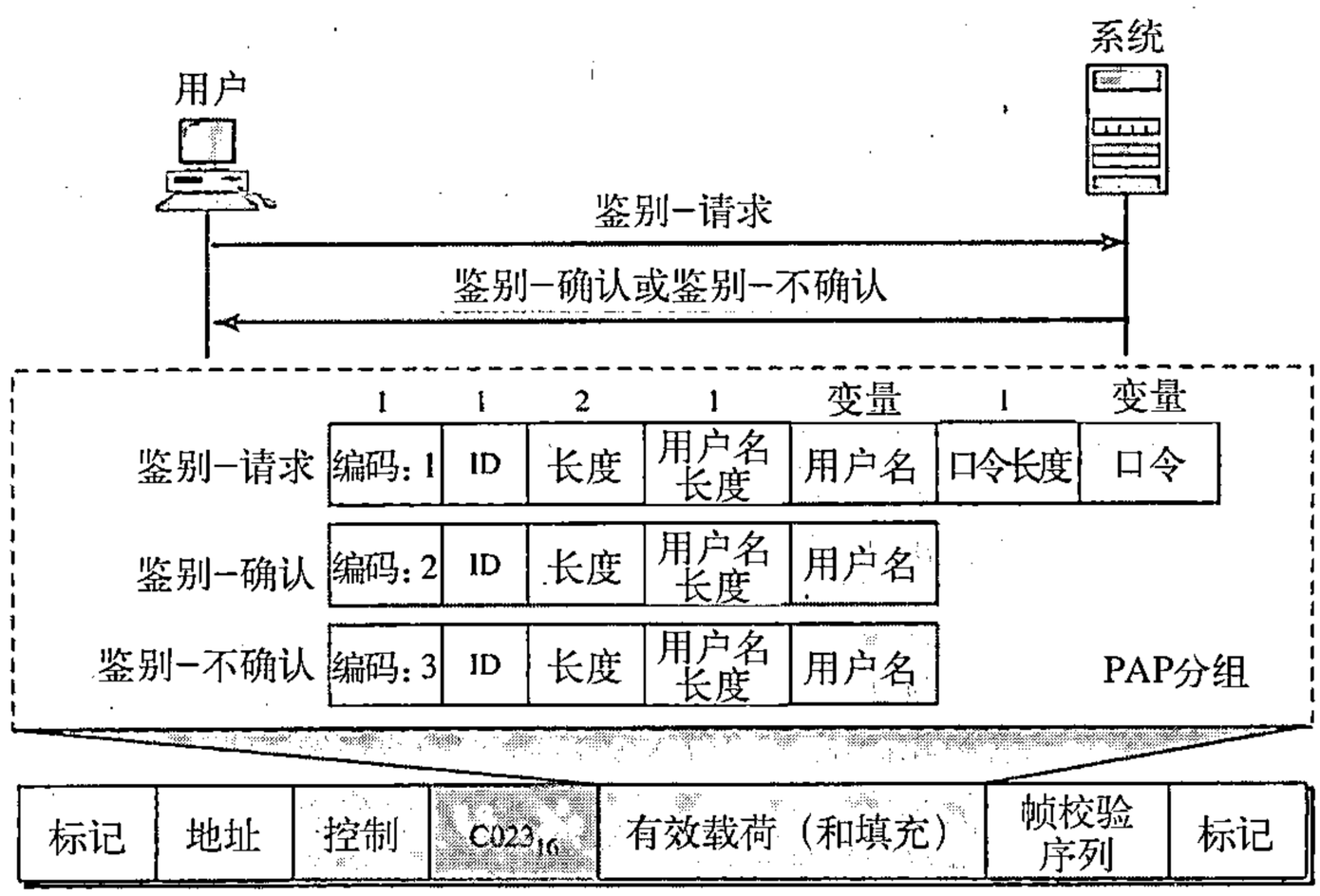


图11.36　封装入一个PPP帧中的PAP分组

CHAP　**查询握手鉴别协议**（Challenge Handshake Authentication Protocol，CHAP）是一个三步握手鉴别协议，比PAP更具安全性。在这种方式下，口令是保密的，无需上线发送。

1. 系统发送给用户一个包含查询值（通常是一些字节）的查询分组。

2. 用户应用一项预定义功能，根据查询值和用户自身的口令产生一个结果。用户将结果放入一个响应分组发送给系统。

3. 系统同样操作。它应用同样的功能，使用户的口令（系统已知的）和查询值产生一个结果。如果这个结果与响应分组中发送的结果相同，访问被允许；否则，访问被拒绝。CHAP比PAP安全得多，特别是当系统不断地更换查询值时。甚至当入侵者得知查询值和结果时，口令始终是保密的。图11.37显示了分组及其如何被使用。

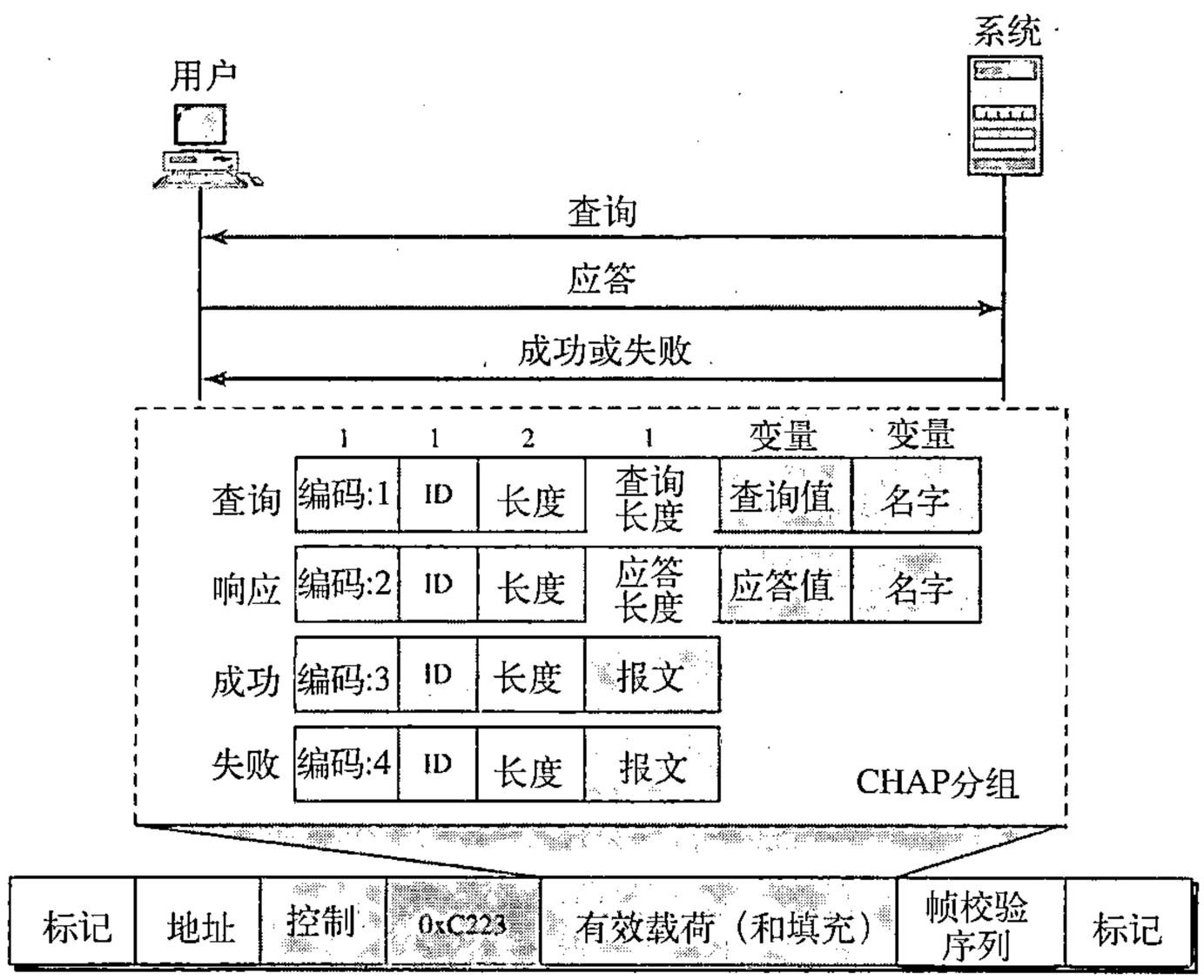


图11.37 封装入一个PPP帧中的CHAP分组

CHAP分组以十六进制的协议值C223封装在一个PPP帧中。有四种CHAP分组：查询、响应、成功或失败。第一种分组被系统用来发送查询值。第二种分组被用户用来返回计算结果。系统用第三种分组来表示准许访问，而用第四种分组来表示拒绝访问。

网络控制协议

PPP是一个多网络层的协议。它能使用因特网限定的协议，如OSI、Xerox、DECnet、AppleTalk、Novel等来运载网络层数据分组。为此，PPP为每一个网络协议都界定了一个特定的网络控制协议。例如，IPCP（互联网络协议控制协议）配置用来承载IP数据分组的链路。Xerox CP配置用来承载Xerox协议数据分组的链路等。注意，没有一个网络控制协议承载网络层数据，它们仅仅是在网络层为进入的数据配置链路。

IPCP 互联网络协议控制协议（Internet Protocol Control Protocol， IPCP）是一种网络控制协议。这个协议配置用于承载因特网中IP分组的链路。IPCP对我们而言有特殊的重要性。图11.38显示了一个IPCP分组的格式。注意协议字段的值是十六进制的8021。

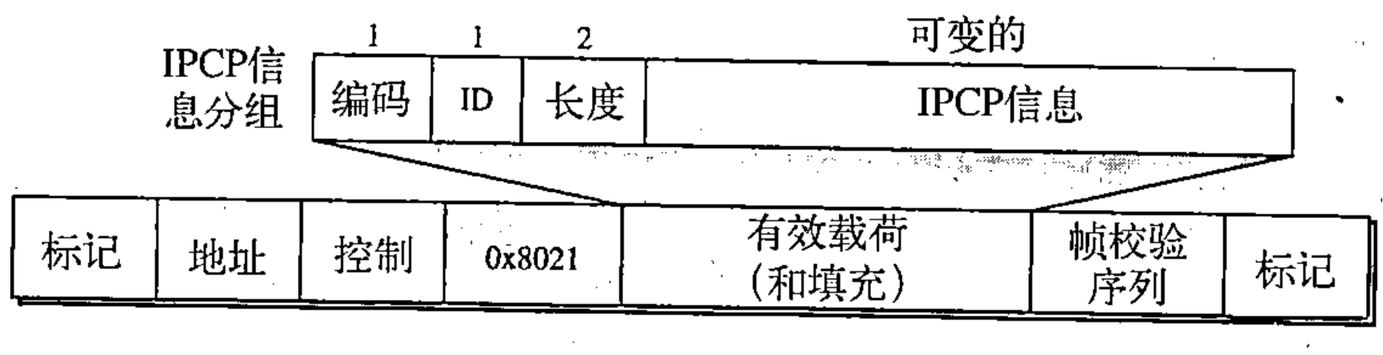


图11.38 封装入一个PPP帧的IPCP分组

IPCP定义了七个分组，通过不同编码值加以区分，如表11.4所示。

其他协议 还有针对其他网络层协议的NCP协议。OSI的网络层控制协议的协议字段值是8023；Xerox NS IDP 控制协议的协议字段值是8025，诸如此类等。其他协议的编码值和分组的格式与表11.4所示的相同。

来自网络层的数据

当由一个NCP协议完成网络层确认后，用户可以交换来自网络层的数据分组。再次重申，

不同的网络层有不同的协议字段。例如，如果PPP运载来自IP网路层的数据，字段值就是0021（注意这个值最右边的三个数字跟IPCP的相同）。如果PPP运载来自OSI网络层的数据，协议字段值就是0023，等等。图11.39显示了IP帧的格式。

表11.4 IPCP分组的编码值

编 码	IPCP分组	编 码	IPCP分组
0x01	配置－请求	0x05	终止－请求
0x02	配置－确认	0x06	终止－确认
0x03	配置－不确认	0x07	编码－拒绝
0x04	配置－拒绝		

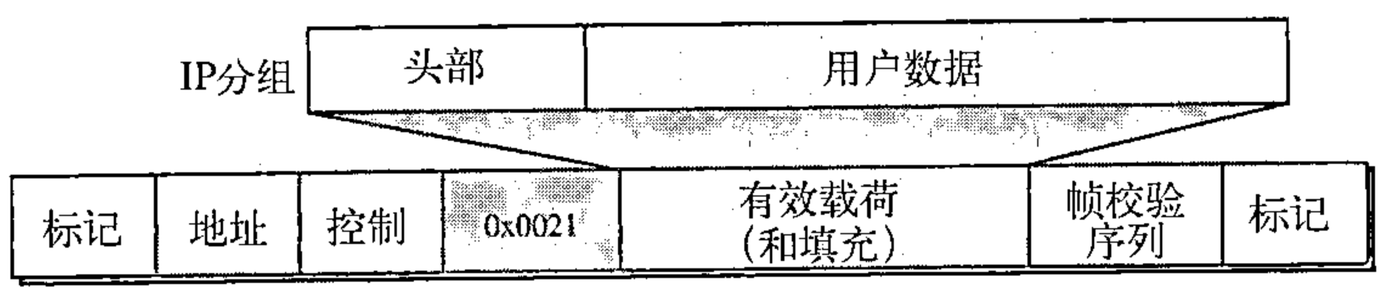


图11.39 封装在一个PPP帧的IP数据报

11.7.4 多链路PPP

PPP最初是为了单通道点到点的物理链路设计的。在单个点到点链路中多通道的实用性促使了多链路PPP的发展。在这种情况下，一个PPP逻辑帧被分割成多个实际的PPP帧。逻辑帧的一段由一个实际PPP帧的有效载荷字段所承载，如图11.40所示。为了显示一个实际PPP帧正承载一个PPP逻辑帧的一段，协议字段被设置为0x003d。这个新发展带来了复杂性。举例来说，必须在实际PPP帧上增加一系列数字以说明这部分在逻辑帧中的位置。

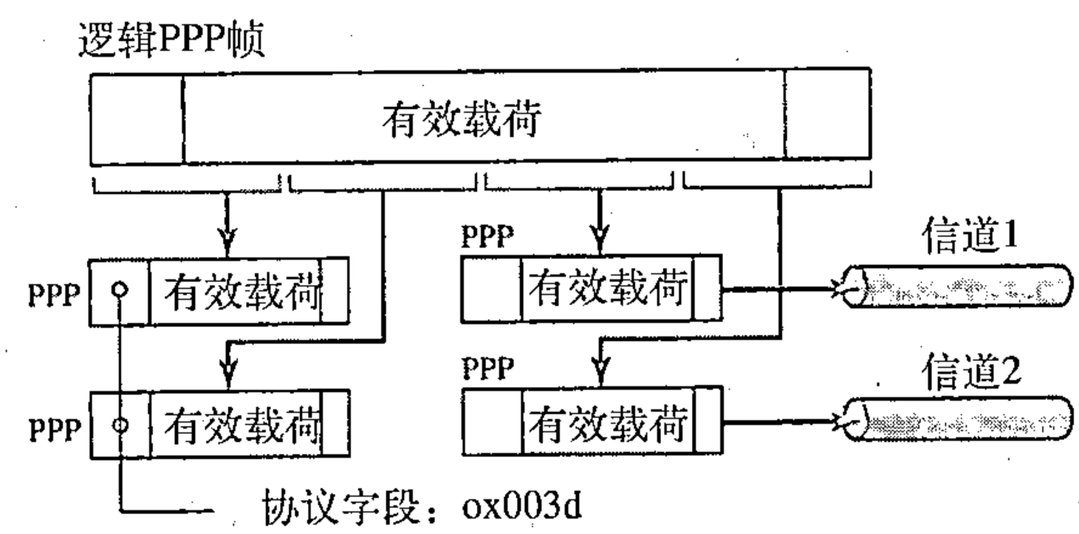


图11.40 多链路PPP

例11.12

让我们来看一下一个网络层分组通过PPP连接传输的各个阶段。图11.41描述了这些步骤。为了简化，我们假定数据从用户端向系统端单向运动（例如通过ISP发送一封邮件）。

前两个帧用作链路的建立。我们选择了两个选项（并没有显示在图中）：使用PAP来鉴别并隐匿地址控制字段。帧3和帧4用来鉴别。帧5和帧6使用IPCP建立了网络层连接。

接下来的几个帧显示了封装在PPP帧的一些IP分组。系统（接收方）已经运行了一些网络层协议，但是它知道进入的数据必须交付给IP协议，因为在数据传输前使用的NCP协议是IPCP。

数据传输之后，用户终止了数据链路连接，并且这被系统所确认。当然如果想要运行另一个NCP协议的话，用户和系统也可以选择终止网络层协议IPCP，而维持数据链路层的运行。

这个例子是微不足道的，但是它指出了LCP、AP和NCP分组的相似之处。它同样描述了协议字段值和特定协议的编码数字。

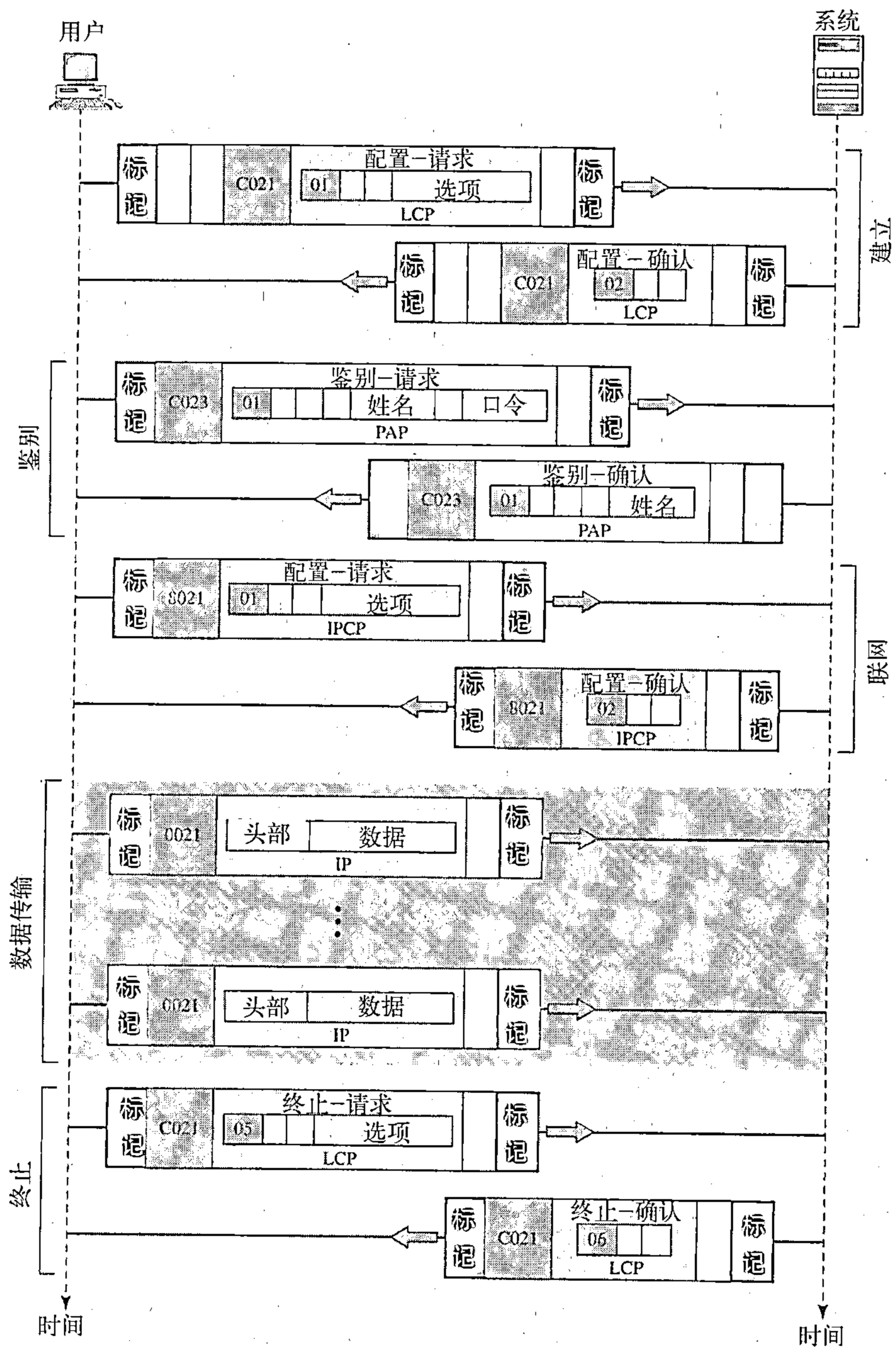


图11.41 一个例子

推荐读物

为了深入讨论与本章有关的主题，推荐下列读物。在本书末列出括号[……]中的参考资料。

读物

数据链路控制的讨论可以在[GW04]、[Tan03]的第3章、[Sta04]的第7章、[Kes97]的第12章与[PD03]的第2章中找到。更进一步的材料可以在[KMK04]找到。

本章小结

- 数据链路控制处理两个邻近节点之间的通信（即节点到节点的通信）的设计和程序。
- 数据链路层的成帧是将一个从源方到目的方的报文与从其他源方到其他目的方的报文分离开来。
- 帧既可以是固定长度的也可以是可变长度的。固定长度成帧中，不需要定义帧的分界；在可变长度成帧中，我们需要一个分隔符（标记）来定义帧与帧的边界。
- 可变长度成帧使用两类协议：面向字节（或者面向字符）的协议和面向位的协议。在面向字节的协议中，帧的数据部分是字节序列；而在面向位的协议中，帧的数据部分是位序列。
- 在面向字节的协议（或面向字符的协议）中，我们使用字节填充；当一个字符的模式与标记相同时，帧的数据部分就会增加一个特定的字节。
- 在面向位的协议中，我们使用位填充；当一串位的模式与标记相同时，帧的数据部分就增加一个额外的0 。
- 流量控制是指一组程序，它用来严格限定发送方在等到确认前能发送的数据量。差错控制指的是差错检测和纠正的方法。
- 对无噪声通道，我们讨论了两种协议：最简协议和停止等待协议。前者不具备差错控制和流量控制；后者不具备差错控制。在最简协议中，发送方可以一个接一个地发送帧而不需要顾及接收方。在停止等待协议中，发送方每发送一个帧，就要停止直到收到接收方的确认，随后再发送下一个帧。
- 对有噪声通道，我们介绍了三种协议：停止等待ARQ，回退N帧的ARQ和选择性重复ARQ。停止等待ARQ在停止等待协议的基础上增加了一个简单的差错控制机制。在回退N帧ARQ中，在收到确认之前，我们可以发送几个帧，这提高了传输的效率。在选择性重复ARQ中，我们避免了不必要的传输，除非帧被损坏了。
- 回退N帧ARQ和选择性重复ARQ两者都使用到滑动窗口。在回退N帧ARQ中，如果m是序列号的位的个数，发送窗口的大小必须小于2^m；接收窗口的大小永远是1。在选择性重复ARQ中，发送方和接收方窗口的大小最多只能是2^m的一半。
- 一项称之为捎带技术被用来提高双向协议的效率。当一个帧从A到B运载数据时，它同时运载了来自B的控制信息；当一个帧从B到A运载数据时，它同样能同时运载来自A的控制信息。
- 高级链路控制（HDLC）一个面向位的协议，用于点到点和多点链路的通信。然而，点到点访问最通用的协议还是点到点协议（PPP），它是一个面向字节的协议。

习题

复习题

1. 简要阐述数据链路层所提供的服务。
2. 定义成帧及其原因。
3. 比较和对比面向字节的协议和面向位的协议。过去哪一种更为流行（解释原因）？而现在哪一个更流行（解释原因）？
4. 比较和对比字节填充和位填充。在面向字节的协议中使用哪一种技术？而面向位的协议又使用哪一种技术？
5. 比较和对比流量控制和差错控制。

6. 在本章中，我们介绍了无噪声通道中的哪两种协议？
7. 在本章中，我们介绍了有噪声通道中的哪三种协议？
8. 解释从停止等待ARQ协议发展到回退*N*帧ARQ协议的原因。
9. 比较和对比回退*N*帧ARQ协议和选择性重复ARQ协议。
10. 比较和对比HDLC和PPP，哪一个是面向字节的，哪一个面向位的？
11. 定义捎带技术及其用途。
12. 本章中利用流水线操作的协议是哪一个？

练习题

13. 对图11.42中的数据进行字节填充。

ESC			Flag			ESC	ESC	ESC		Flag	

图11.42 练习题13

14. 对图11.43中的数据进行位填充。

00011111110011111010001111111111000011111

图11.43 练习题14

15. 为字节填充设计两个简单的算法。发送方使用前者添加字节，接收方使用后者去除字节。
16. 为位填充设计两个简单的算法。发送方使用前者添加位，接收方使用后者去除位。
17. 一个发送方发送一系列分组到同一个目的地，使用一个5位长度的序列号。如果序列号以0开始，发送一百个分组后序号是多少？
18. 使用一个5位序号，在下列协议中，发送和接收窗口的最大尺寸分别是多少？
    a. 停止等待ARQ　　b. 回退*N*帧ARQ
    c. 选择性重复ARQ
19. 为使用捎带的最简协议设计一个双向算法。注意双方都使用同一个算法。
20. 为使用捎带的停止等待协议设计一个双向算法。注意双方都使用同一个算法。
21. 为使用捎带的停止等待ARQ设计一个双向算法。注意双方都使用同一个算法。
22. 为使用捎带的回退*N*帧ARQ设计一个双向算法。注意双方都使用同一个算法。
23. 为使用捎带选择性等待ARQ设计一个双向算法。注意双方都使用同一个算法。
24. 图11.44显示了一个状态示意图来模拟停止等待ARQ中发送方站点的行为。该状态中S_n有一个值（0或1）。箭头表示传输。请解释标记为A和B的两个传输事件。
25. 图11.45显示了一个状态示意图来模拟停止等待ARQ中接收方站点的行为。该状态中R_n有一个值（0或1）。箭头表示传输。请解释标记为A和B的两个传输事件。

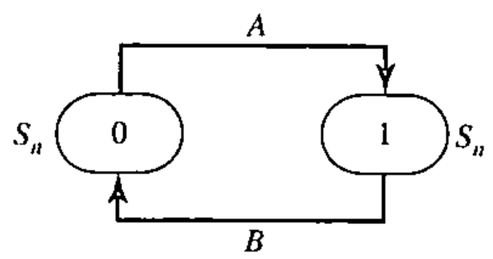


图11.44 练习题24

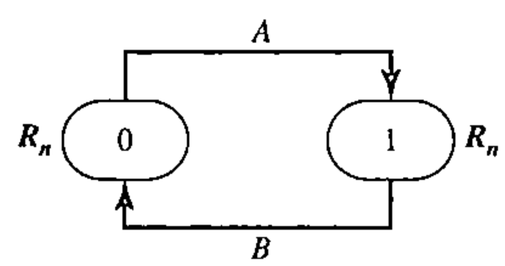


图11.45 练习题25

26. 在停止等待ARQ中，我们可以将练习题24和练习题25中发送方和接收方的算法综合起来。一个状态来界定R_n和S_n的值。这意味着我们有四种状态，由（x，y）决定，x指的是S_n的值，而y指的是R_n的值。换言之，图11.46中会有四种状态。请解释标记为A、B、C和D四个传

输事件。

27. 使用停止等待ARQ协议的系统的一个定时器有6微秒超时。为4个帧画一张类似图11.11的流量图，假定往返延迟是4微秒，且没有数据帧或者控制帧被丢失或损坏。
28. 如果练习题27中超时时间为4微秒，而往返延迟为6微秒，请画图。
29. 如果练习题27中第一个帧（帧0）被丢失了，请画图。
30. 一个系统使用停止等待ARQ协议。如果每个分组承载1 000位数据，发送方和接收方之间的距离是5 000公里，传输速度是2×10^8米，那么传输1百万位需要多长时间？忽略传输、等待和处理延迟。且我们假定没有数据帧和控制帧丢失或损坏。
31. 练习题30中，如果使用回退N帧ARQ协议，且窗口大小是7，需要多长时间？忽略用于头部和尾部的开销。
32. 练习题30如果使用选择性重复ARQ协议，且窗口大小是4，需要多长时间？忽略用于头部和尾部的开销。

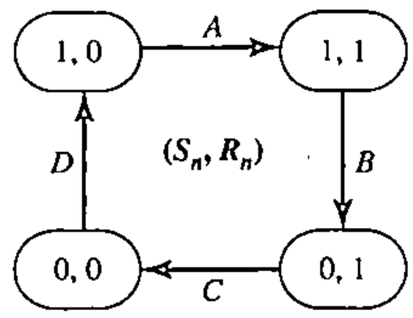


图11.46 练习题26

第12章 多路访问

在第11章我们讨论了数据链路控制，它是为链路提供可靠通信的一种机制。在我们描述的协议中，我们假定在发送方和接收方之间存在可用的专用链路（或通道）。这个假定可能正确也可能不正确。事实上，如果我们有一条专用链路，就像我们连接因特网时使用PPP作为数据链路控制协议时，那么这个假定是正确的，我们就无需其他任何东西了。

另一方面，如果我们使用移动电话去与其他移动电话连接，通道（分配给售方公司的波段）并不是专用的。几步以外在用移动电话和他的朋友聊天的一个人就可能和我们使用同一个通道。

我们认为数据链路层有两个子层。上子层负责数据链路控制，下子层负责解决对共享介质的访问。如果通道是专用的，我们并不需要下子层。图12.1显示了在数据链路层中的两个子层。

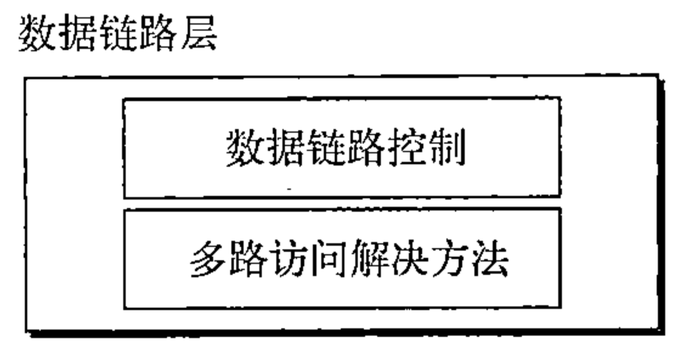


图12.1 数据链路层被分为两个功能定位的子层

在第13章中，我们将会看到IEEE确实为LAN做了如此的分割。上子层负责流量控制和差错控制，被称为逻辑链路控制层（LLC）；下子层主要负责多路访问的解决方法，被称为介质访问控制层（MAC）。

当节点或站点被连接并使用一个称为多点链路或广播链路的公共链路时，我们就需要一个多路访问协议去协调链路的访问。介质访问的控制问题与在集会上发言的规则相似。有程序来为发言授权并保证没有两个人会在同一时间发言，发言者不会打扰到彼此，不会独占发言权等。

多点网络的情况与此相似。许多正式的协议被设计用来处理共享链路的访问。我们将这些协议分为三类，如图12.2所示。

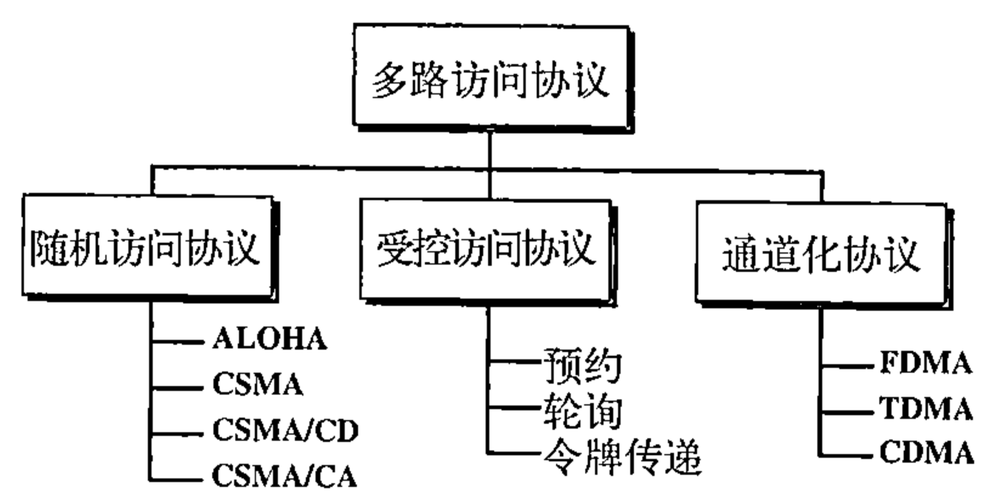


图12.2 本章讨论的多路访问协议的分类

12.1 随机访问

在**随机访问**（random access）或**竞争**（contention）访问方法中，没有任何站点是优于其他站点的，也没有任何站点能控制其他站点。一个站点没有权利允许或者不允许另一个站点的发送。每一次，当一个站点要发送数据时，都要用协议定义的程序来决定是否要发送。这个决定取决于介质的状态（空闲或忙碌）。换言之，只要遵循预定义的程序，包括介质状态的检测，满足条件的每一个站点都能传输数据。

该方法之所以如此命名因为两个特性。第一，每一个站点的传输没有特定的时间表。站点的传输是随机的。这就是该方法称为随机访问的原因。第二，没有规则来规定下一个将要发送的站点是哪一个。为了访问介质，站点展开竞争。这就是该方法称为竞争访问的原因。

在随机访问中，每个站点都有权访问介质且不受控于其他站点。然而，如果当一个以上的站点想要发送时，便会产生**冲突**（collision），那么便会有帧被损坏或修改。为了避免访问

冲突或在访问冲突发生时解决这个问题，每个站点遵循一个程序，该程序回答了以下问题：

- 站点何时能访问介质？
- 如果介质忙碌，站点能做什么？
- 站点如何确定传输的成功或者失败？
- 如果发生访问冲突，站点能做什么？

本章中我们学习的随机访问方法是由一个十分有趣的称之为ALOHA的协议发展而来的，ALOHA使用一个十分简单的程序称为**多路访问**（multiple access， MA）。该方法是外加一种程序（就是在站点传送之前要对介质进行侦听）改进而来。它称为载波侦听多路访问（CSMA）。该方法后来又演化成两个并行的方法：**带冲突检测的载波侦听多路访问**（carrier sense multiple access with collision detection，CSMA/CD）和**带避免冲突的载波侦听多路访问**（carrier sense multiple access with collision avoidance，CSMA/CA）。CSMA/CD告诉站点一旦检测到冲突，下一步该做什么，而CSMA/CA尝试如何避免冲突。

12.1.1 ALOHA

ALOHA是最早的随机访问方法，它由夏威夷大学在20世纪70年代初期开发出来。它被设计用于无线电（无线）LAN，但也可以使用在任何共享介质上。

很明显，在这种安排中有一些潜在的冲突。介质为各个站点所共享，当一个站点发送数据时，其他站点也可能在同时尝试发送，两个站点的数据相互冲突并且相互混淆。

纯ALOHA

最早的ALOHA协议称为**纯ALOHA**（pure ALOHA）。这是一个简单的但优秀的协议。它的概念是只要每个站点有帧要发送，它就发送帧。然而，既然只有一条共享通道，不同站点之间的帧就有可能发生冲突。图12.3描述了一个在纯ALOHA中帧冲突的例子。

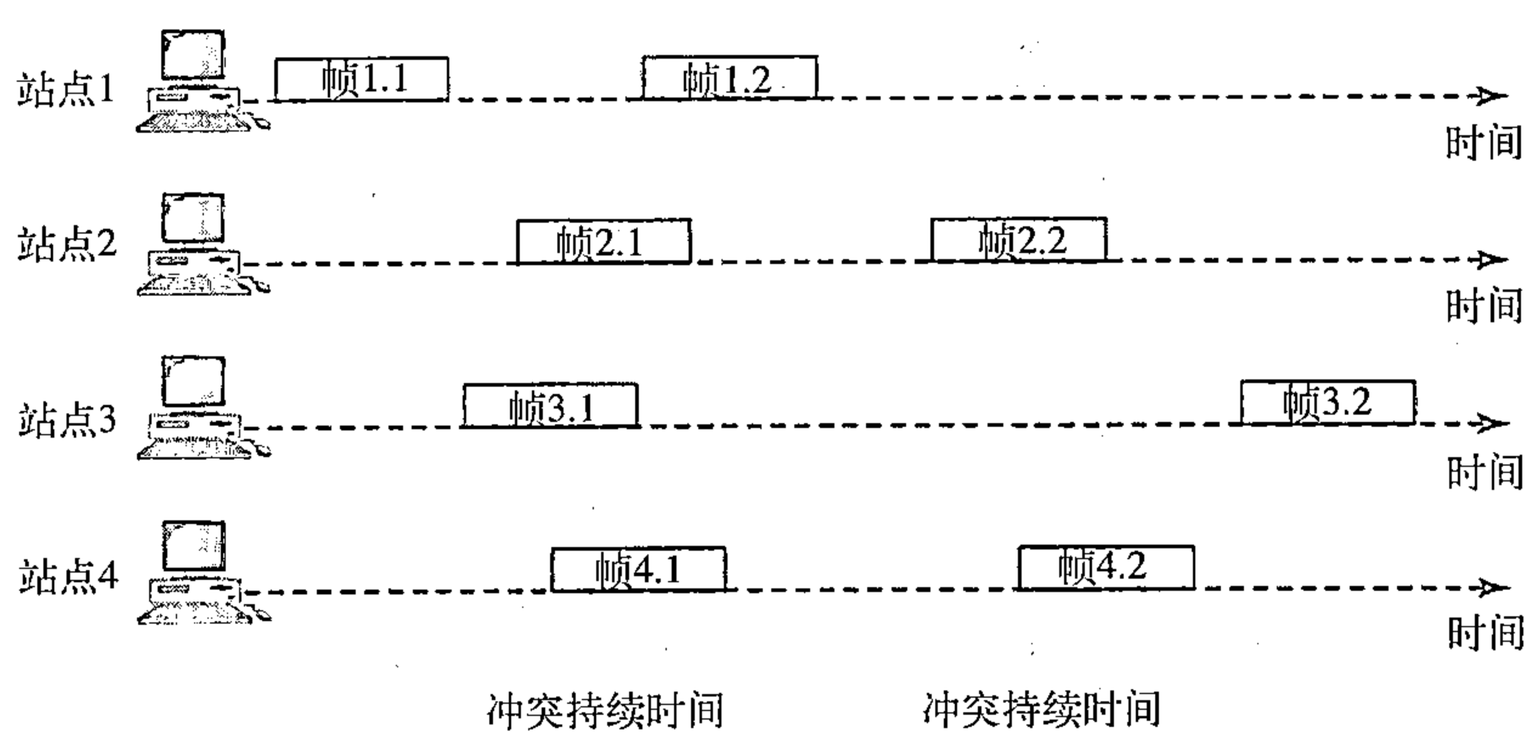


图12.3　纯ALOHA网络中的帧

图中有4个站点（非现实的假设）为访问共享通道彼此竞争。每个站点发送两个帧，共享介质中共有8个帧。在为共享通道展开的多帧竞争中，某些帧冲突了。图12.3中只有两个帧是成功的：来自站点1的帧1.1和来自站点3的帧3.2。必须要提醒的是，只要一个帧的一个位与另一个帧的一个位同时存在于通道中，就会产生冲突且两个帧都会被损坏。

很明显，我们需要重发在传输过程中损坏的帧。纯ALOHA协议依赖于接收方的确认。当一个站点发送一个帧时，它就等待接收方发送一个确认。如果超时之后确认还未达到，站点就认为帧（或者确认）被损坏了，于是重发帧。

一个冲突涉及两个或更多的站点。如果这些站点在超时后都尝试重发帧，那么这些帧可能再次发生冲突。纯ALOHA指定超时后，每个站点在重发帧之前都会随机等待一段时间。这个随机事件有助于避免更多的冲突。我们称这个时间为补偿时间T_B。

纯ALOHA还需要第二个方法来阻止重新传输的帧造成通道拥堵。一个站点在经过最大次数（K_{max}）的重发尝试后，它必须放弃并在以后再试。图12.4描述了基于上述策略的纯ALOHA进程。

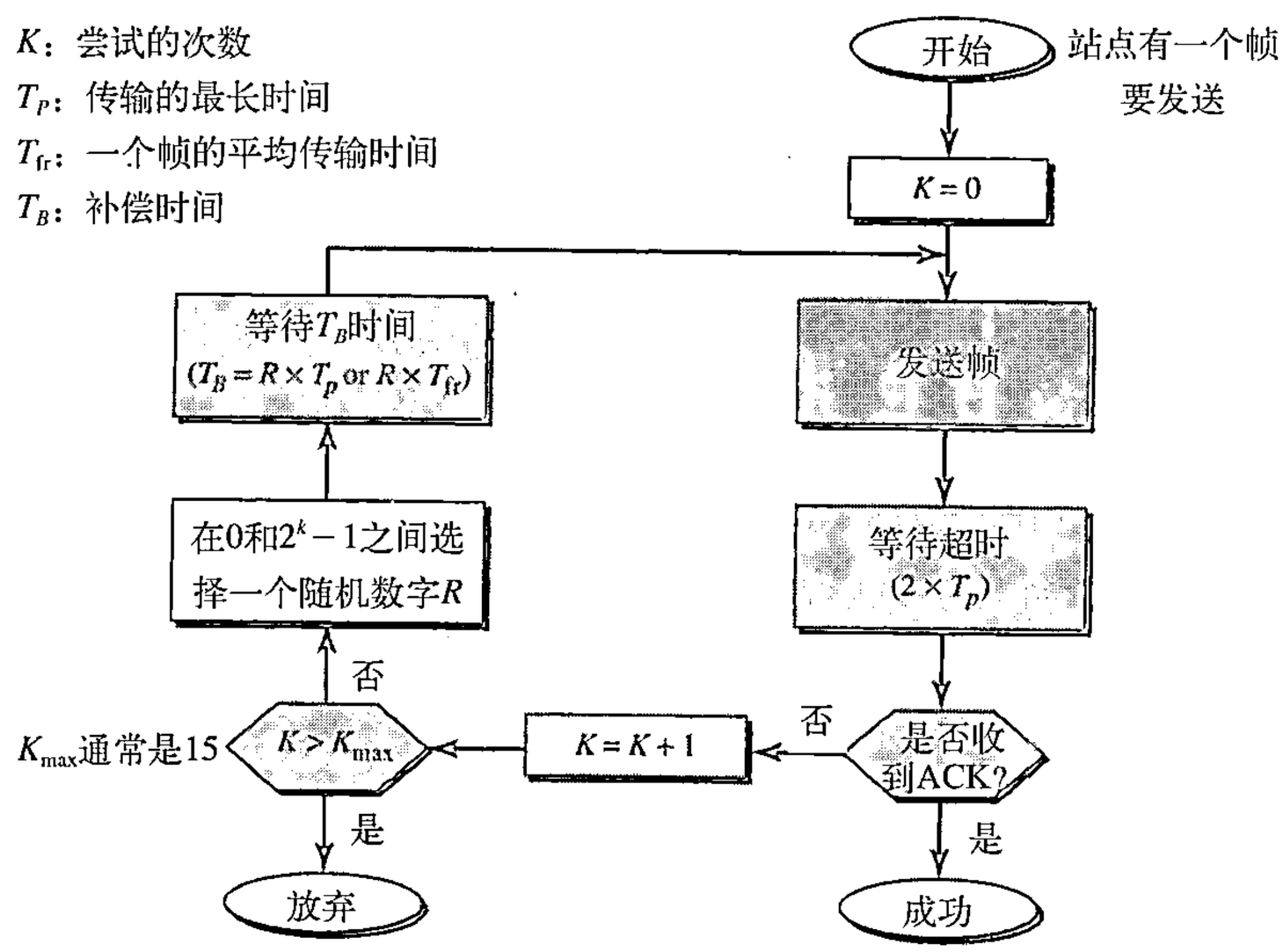


图12.4 纯粹ALOHA协议的进程

超时周期等于往返传输延迟的最大可能值，也就是两个最远距离站点之间发送一个帧所需时间的两倍（$2\times T_P$）。补偿时间T_B是一个随机值，主要取决于K（未成功的传输尝试次数）的值。T_B的公式取决于实现。一个通用公式是**二进制指数回退**（binary exponential back-off）。这种方法中，每一次传输随机选择0到2^k-1范围内的一个数，该数乘以T_P（传输的最长时间）或乘以T_{fr}（一个帧的平均传输时间）以得到T_B的值。注意在这个进程中，每一次冲突后随机数的范围便扩大了。K_{max}的常用值是15。

例12.1

一个无线ALOHA网络中的站点之间的最远距离是600公里。如果我们假设信号的传播速度是3×10^8米/秒，那么$T_P=600\times10^5/(3\times10^8)=2$毫秒。于是，对于不同的$K$值，我们可以得到不同的$T_B$值。

a. 如果$K=1$，取值范围是{0，1}。站点会产生一个随机变量，变量值为0或1。这就意味着T_B是0（0×2）毫秒或2（1×2）毫秒，取决于产生的随机变量。

b. 如果$K=2$，取值范围是{0，1，2，3}。这就意味着T_B是0，2，4或6毫秒，取决于产生的随机变量。

c. 如果$K=3$，取值范围是{0，1，2，3，4，5，6，7}。这就意味着T_B是0，2，4，……14毫秒，取决于产生的随机变量。

d. 要提醒的一点是，如果$K>10$，它通常被设定为10。

脆弱时间 我们来计算可能发生冲突时间的长短，即**脆弱时间**（vulnerable time）的长短。假设站点发送固定长度的帧，且每个帧的发送时间是T_{fr}秒。图12.5显示了站点A的脆弱时间。

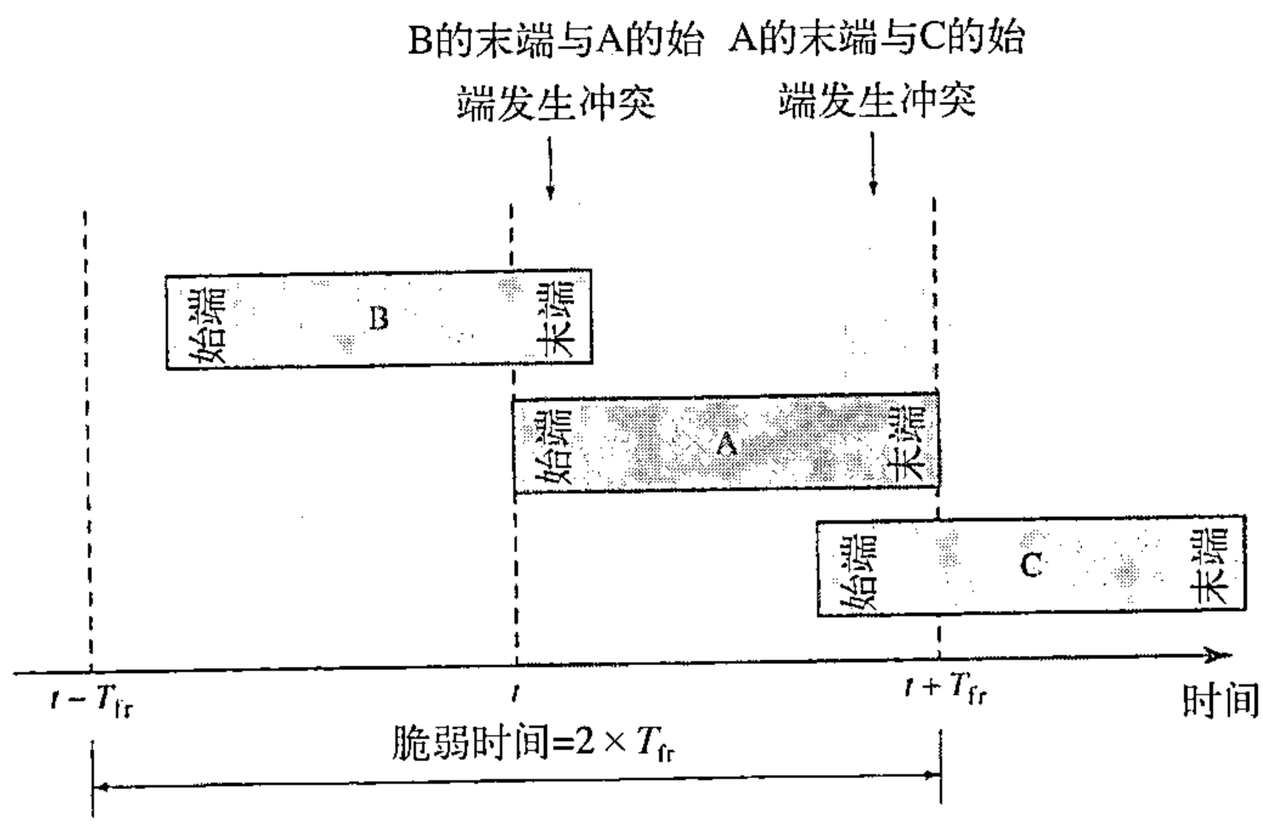


图12.5 纯ALOHA协议的脆弱时间

站点A在时刻t发送一个帧。设想站点B在时刻$t-T_{fr}$和t之间已经发送了一个帧。B帧的末端与A帧的始端发生了冲突。另一方面，假定站点C在时刻t和$t+T_{fr}$之间发送了一个帧。此时，站点A和站点C的帧之间发生了冲突。C帧的始端与A帧的末端发生了冲突。

图12.5中，我们可以看到脆弱时间，即纯粹ALOHA中一个冲突可能发生的时间是帧传输时间的2倍。

纯ALOHA脆弱时间 $=2\times T_{fr}$

例12.2

一个纯ALOHA网络在一个传输速度为200kbps的共享通道上传输200位的帧。使这个帧在传输过程中避免冲突要达到什么样的条件?

解：

帧的平均传输时间T_{fr}是200比特/200kbps，即1毫秒。脆弱时间是2×1毫秒＝2毫秒。这就意味着在该站点开始传输前1毫秒，没有站点会开始传输，并且当该站点正在传输的1毫秒期间也没有站点会开始传输。

吞吐量 我们称G为一个帧传输时间内系统产生的帧的平均数量。能证明纯ALOHA成功传输帧的平均数量是$S=G\times e^{-2G}$。当$G=1/2$，得到最大吞吐量S_{max}是0.184。换言之，在一个帧的传输时间内产生二分之一个帧（即两个帧的传输时间产生一个帧），这些帧的18.4%能成功到达目的地。因为脆弱时间是帧传输时间的2倍，故这是一个可预期的结果。因此，如果一个站点在脆弱时间内只产生一个帧（且在这段时间内没有其他站点产生帧），这个帧便能成功地到达它的目的地。

纯ALOHA的吞吐量是$S=G\times e^{-2G}$，当$G=(1/2)$时，最大吞吐量 $S_{max}=0.184$。

例12.3

一个纯ALOHA网络在一个传输速度为200kbps的共享通道上传输200位的帧。如果系统（包括所有站点）产生以下数量的帧，吞吐量将是多少?

a. 每秒1 000个帧

b. 每秒500个帧

c. 每秒250个帧

解：

帧的传输时间是200比特/200kbps，即1毫秒。

a. 如果系统每秒产生1 000个帧，也就是每毫秒产生一个帧。负载为1。在这种情况下，即$S=0.135$ (13.5%)。也就是说，吞吐量是$1\ 000\times0.135=135$帧。1 000个帧中只有135个可能成功。

b. 如果系统每秒产生500个帧，也就是每毫秒产生1/2个帧。负载为1/2。在这种情况下，即$S=0.184$ (18.4%)。也就是说，吞吐量是$500\times0.184=92$帧。500个帧中只有92个可能成功。注意这就是最大吞吐量百分比的例子。

c. 如果系统每秒产生250个帧，也就是每毫秒产生1/4个帧。负载为1/4。在这种情况下，即$S=0.152$ (18.4%)。也就是说，吞吐量是$250\times0.152=38$帧。250个帧中只有38个可能成功。

时隙ALOHA

纯ALOHA的脆弱时间是$2\times T_{fr}$。这是因为没有规则来定义站点何时能发送。一个站点可能在另一个站点已经开始发送之后马上发送，也可能在另一个站点完成发送之前马上发送。时隙ALOHA的发明改进了纯ALOHA的效率。

在**时隙ALOHA**（slotted ALOHA）中，我们把时间分割成T_{fr}秒的时隙，并强制站点只有在时隙开始之时才能发送。图12.6描述了一个在时隙ALOHA中帧冲突的例子。

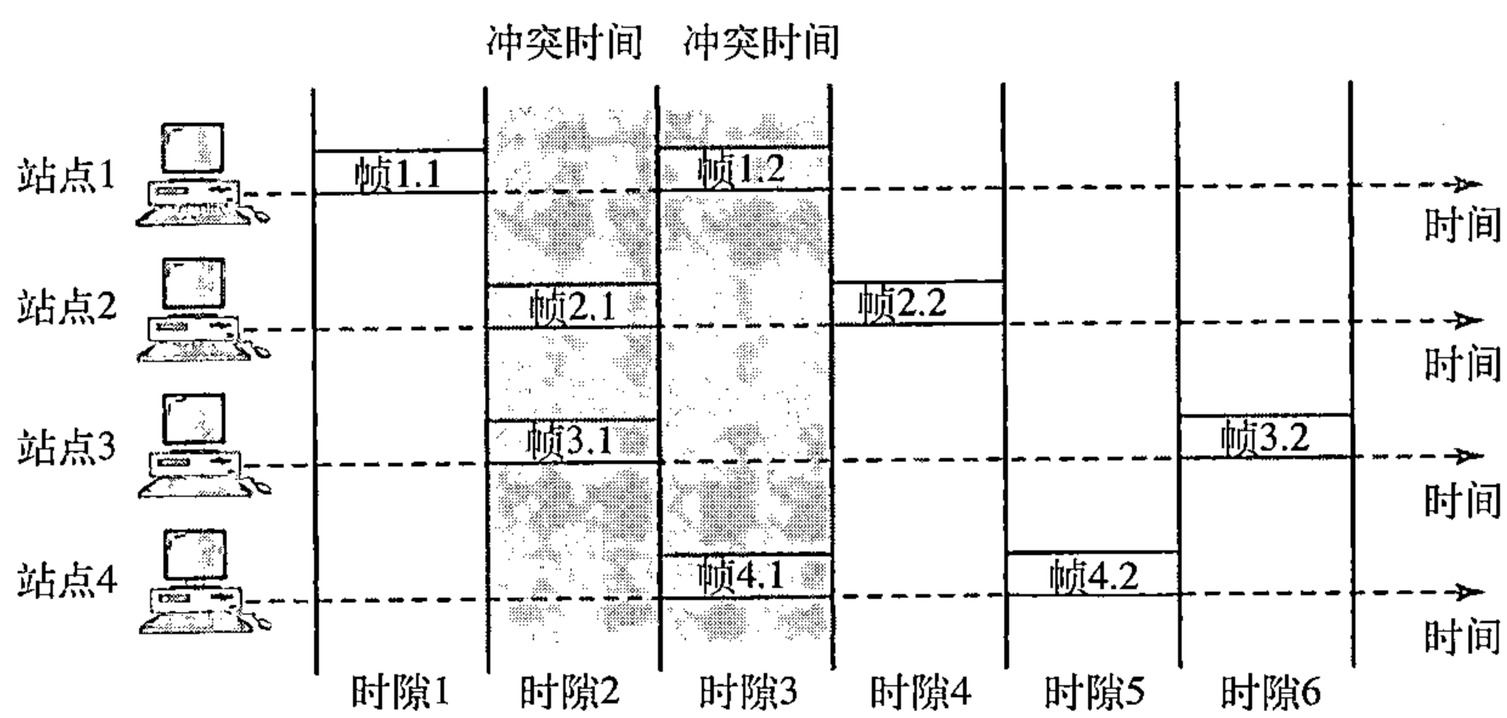


图12.6 时隙ALOHA中的帧

因为只允许一个站点在同步时隙的开始时发送帧，如果一个站点错过了这个时刻，就要等到下一个时隙的开始之时。这就意味着在时隙开始之时，发送帧的站点在该时隙就已经完成了发送。当然，如果两个站点试图在同一个时隙的开始之时发送帧，那么仍然存在冲突的可能。无论如何，现在脆弱时间减少了一半，如图12.7所示，它等于T_{fr}。

图12.7说明了时隙ALOHA的脆弱时间只是纯ALOHA的一半。

时隙ALOHA的脆弱时间 $=T_{fr}$

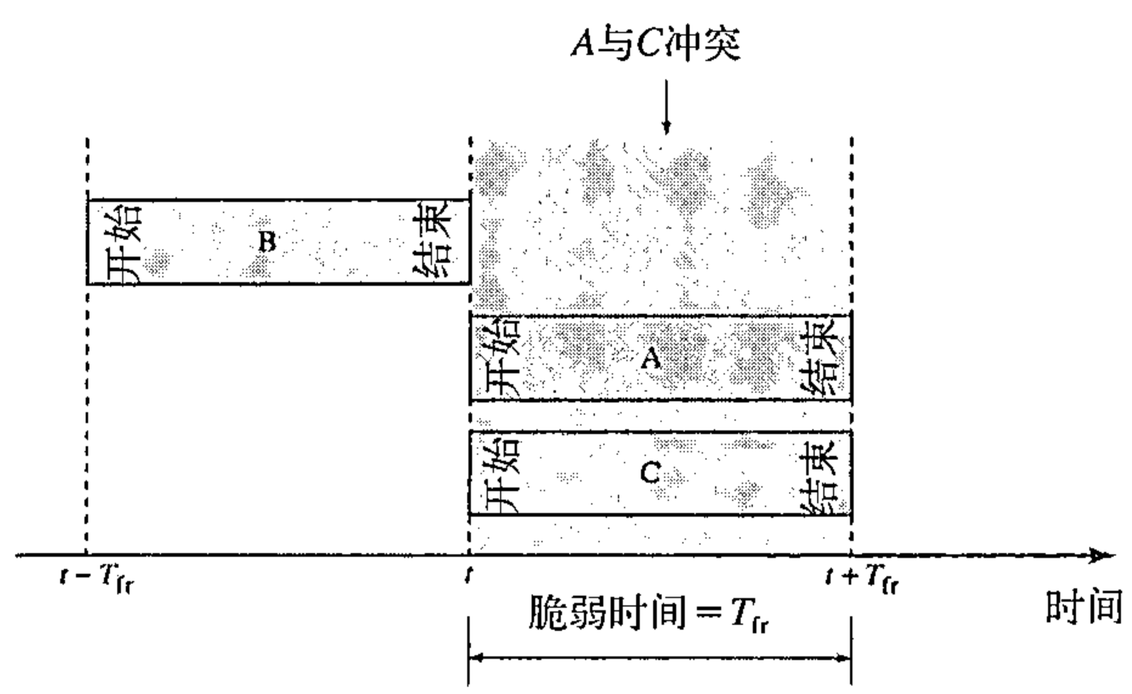


图12.7 时隙ALOHA协议中的易受攻击时间

吞吐量 能证明时隙ALOHA成功传输帧的平均数量是$S=G\times e^{-G}$。当$G=1$时，得到最大吞吐量S_{max}是0.368。换言之，当一个帧传输时间内有一个帧产生的话，这些帧的36.8%会成功达到它们的目的地。这个结果是能被

预知的，因为脆弱时间等于帧传输时间。因此，如果一个站点在脆弱时间内只产生一个帧（且在这个时段内没有其他站点产生帧），这个帧便能成功地到达它的目的地。

时隙ALOHA的吞吐量是$S = G \times e^{-G}$。当$G=1$时，最大吞吐量$S_{max}=0.368$

例12.4

一个时隙ALOHA网络使用一个带宽为200kbps的共享通道发送200比特的帧。如果系统（包括所有站点）产生以下数量的帧，吞吐量将是多少？

a. 每秒1 000个帧

b. 每秒500个帧

c. 每秒250个帧

解：

此情况与前一个练习相似，除了网络使用时隙ALOHA来代替纯ALOHA之外。帧传输时间是200/200kbps，即1微秒。

a. 此例中G为1，因此$S=G \times e^{-G}$，即$S=0.368$（36.8%）。这意味着吞吐量是1 000 × 0.368 = 368帧。1 000个帧中只有368个能成功。注意这就是最大吞吐量百分比的例子。

b. 此例中G为1/2，因此$S=G \times e^{-G}$，即$S=0.303$（30.3%）。这意味着吞吐量是500 × 0.303 = 151帧。500个帧中只有151个能成功。

c. 此例中G为1/4，因此$S=G \times e^{-G}$，即$S=0.195$（19.5%）。这意味着吞吐量是250 × 0.195 = 49帧。250个帧中只有49个能成功。

12.1.2 载波侦听多路访问（CSMA）

为了减少冲突发生的概率从而提高性能，我们采用CSMA方法。如果站点在发送前先侦听一下介质，其冲突的概率就会减少。**载波侦听多路访问**（Carrier sense multiple access，CSMA）要求每一个站点在发送之前首先要监听介质（或者检查介质的状态）。换句话说，CSMA基于的原理就是“传输前先侦听”或“说话前先听”。

虽然CSMA能够减少冲突的概率，但是它不能消除冲突。原因如图12.8，一个CSMA网络的时空模型中所示。站点与一个共享通道连接（通常是一个专用介质）。

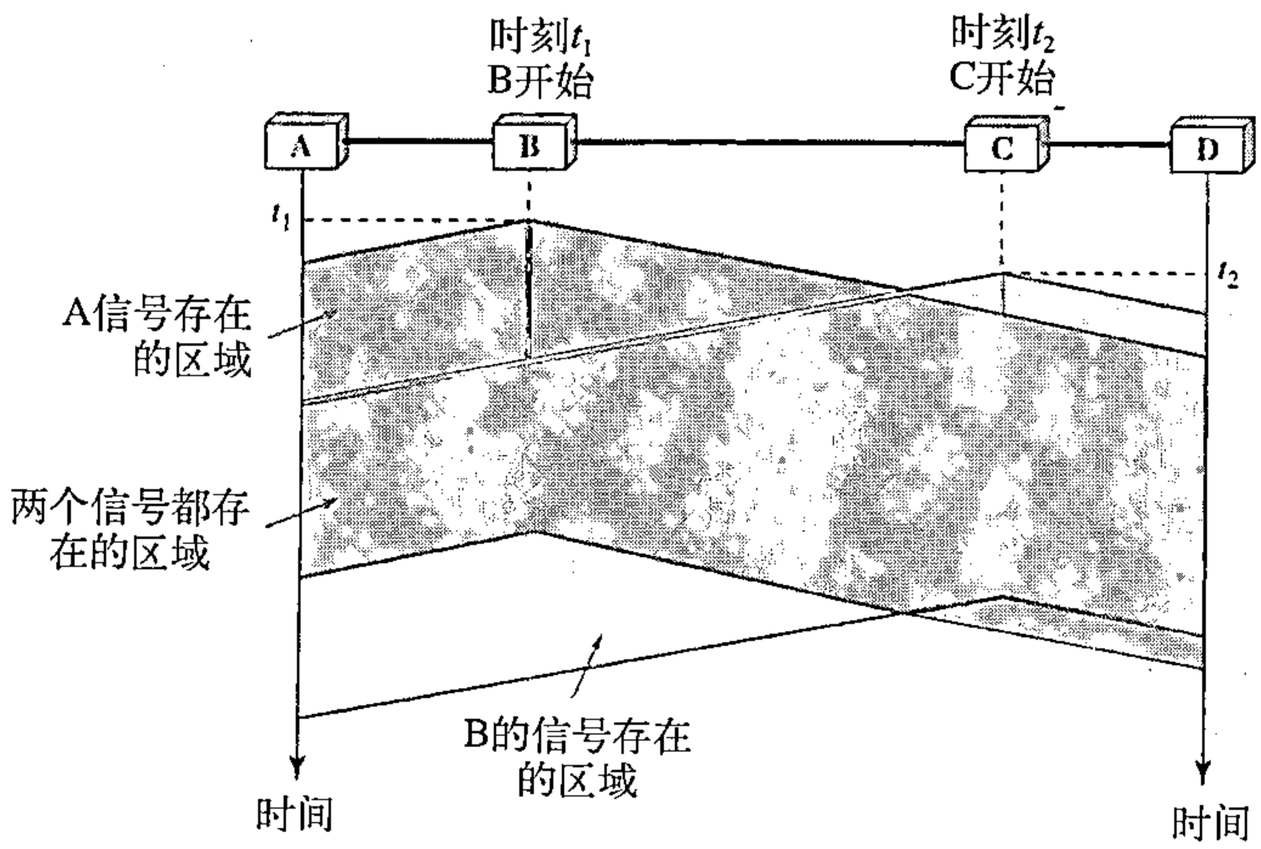


图12.8　CSMA中冲突的时空模型

冲突的概率依然存在的原因是因为传播的延迟。当站点发送一帧时，它的第一个位到达每一个站点和每一个站点检查到它的信号都需要一定的时间（尽管这一时间非常之短）。换句

话说，一个站点之所以能够侦听到介质并发现它处于空闲状态，仅仅是因为其他站点的传播信号还没有到达该站点。

在时刻t_1，站点B侦听介质并发现介质处于空闲状态，所以它发送一个帧。在时刻t_2 ($t_2>t_1$)，站点C侦听介质并发现它处于空闲状态，因为这时从站点B发来的第一个位还没有到达站点C。站点C也发送一个帧。于是这两个信号冲突且两个帧同时被损坏了。

脆弱时间

CMSA的脆弱时间是**传播时间**T_p。这是一个信号从介质的末端到另一端的传播时间。当一个站点发送一个帧，且其他站点也试图在这个时间段内发送帧时，就会发生冲突。但是，如果帧的第一个位已经到达了介质的末端，其他每个站点都能“听到”这个位，并停止发送。图12.9显示了最差的状况。最左边的站点A在时刻t_1发送一个帧，它在时刻t_1+T_p到达最右边的站点D。灰色区域显示了时间上和空间上的脆弱区域。

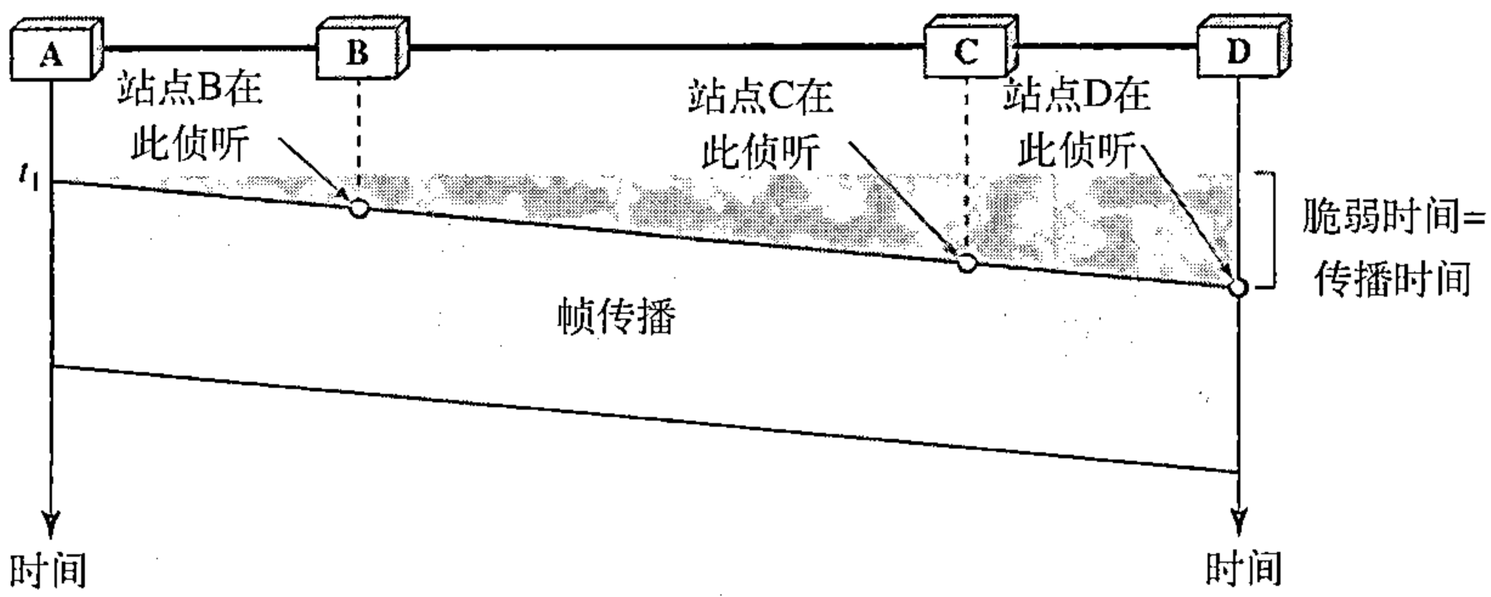


图12.9 CMSA的脆弱时间

持续方法

如果通道繁忙，站点将如何做？如果通道空闲，站点又将做什么？有三种方法被设计用来回答这些问题：1-持续方法，非持续方法和p-持续方法。图12.10描述了当一个站点发现通道繁忙时三种持续方法的表现。

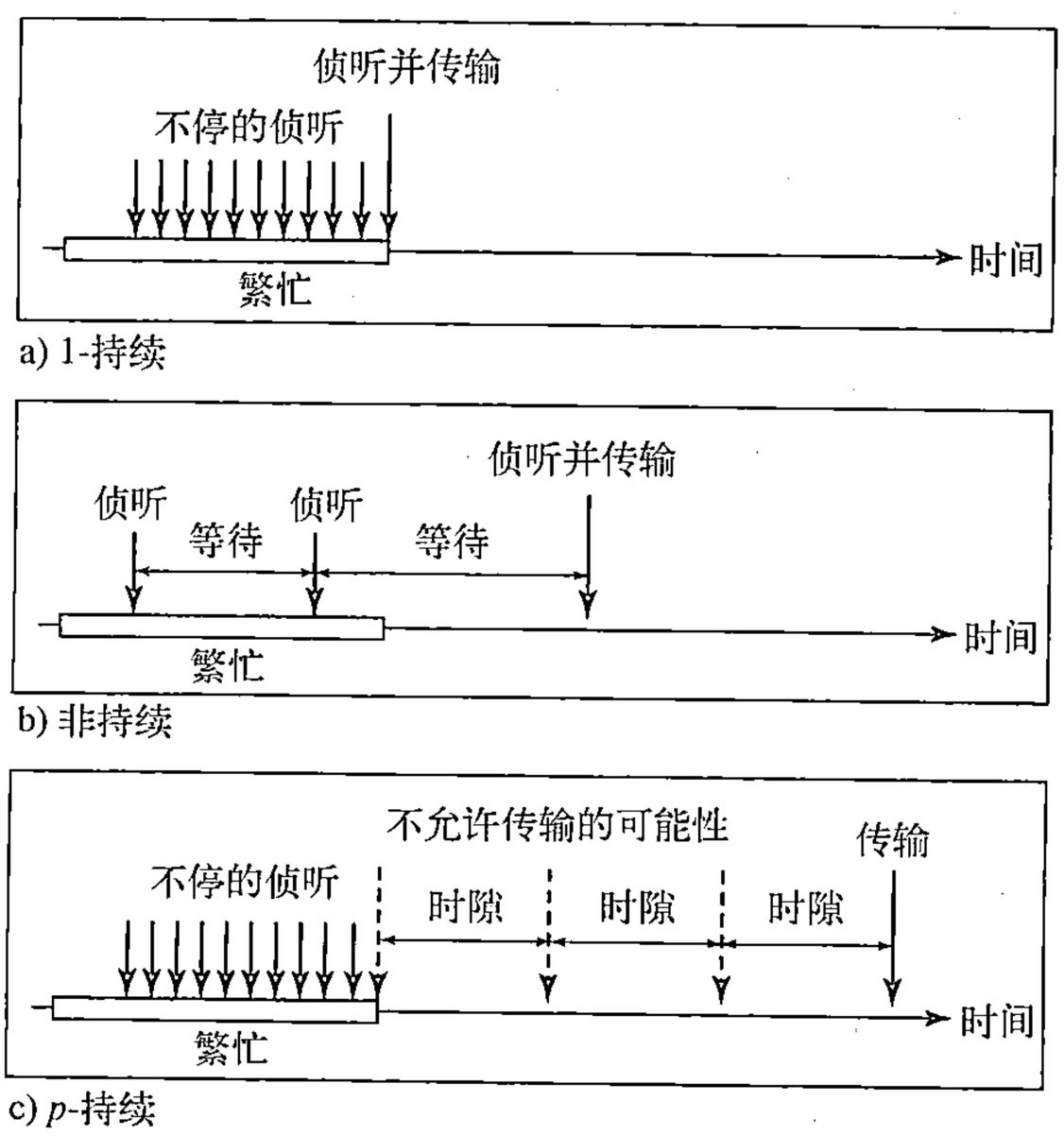


图12.10 三种持续方法的表现

图12.11描述了这些方法的流量图。

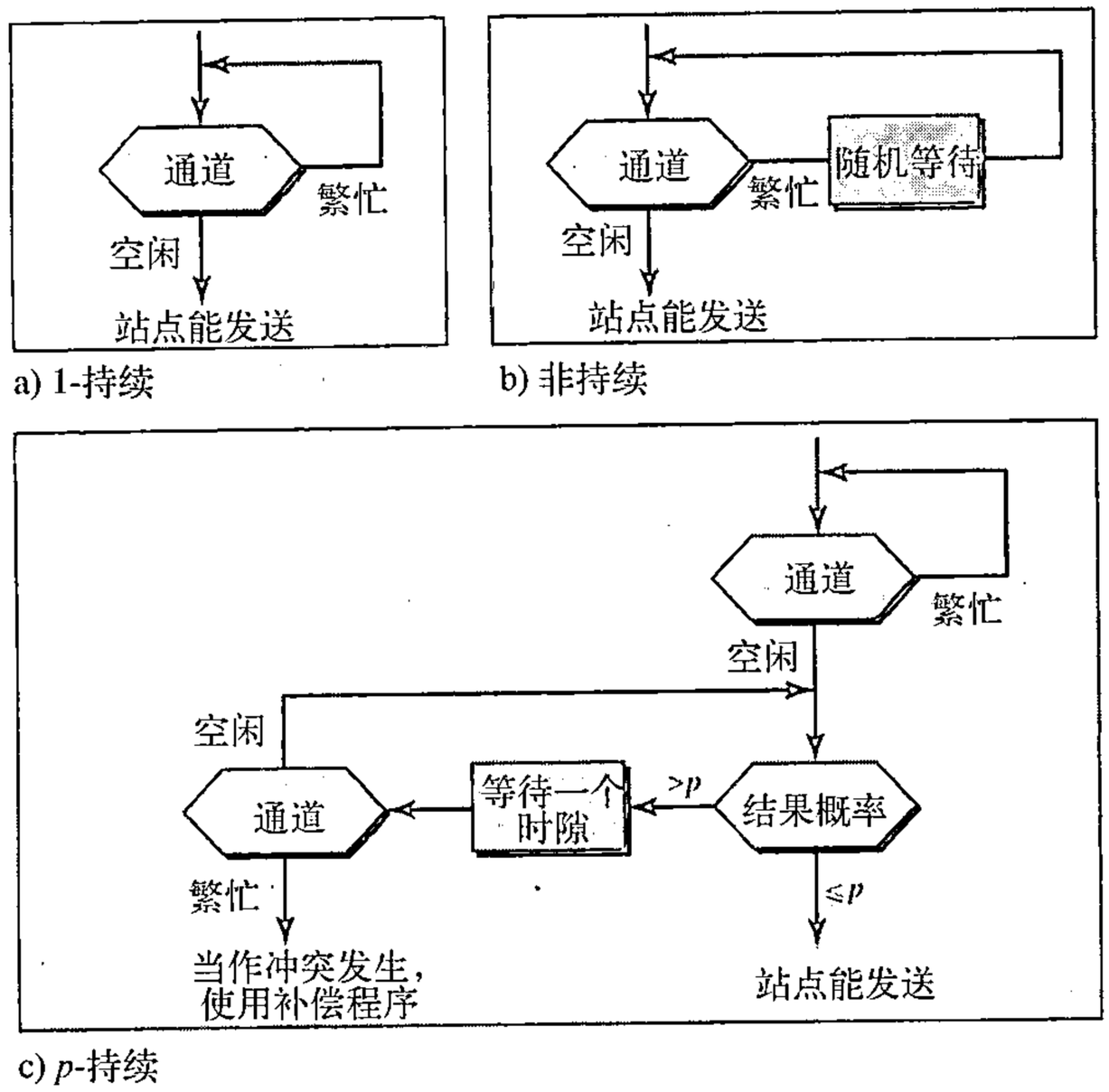


图12.11　三种持续方法的流量图

1-持续　**1-持续方法**（1-persistent method）是最简单直接的。该方法是，如果站点发现线路空闲，那么它就立即发送帧（概率是1）。这种方法发生冲突的机率最高，因为在发现了线路空闲后，两个或更多的站点都可以发送它们的帧。在第13章中，我们将看到以太网使用该方法。

非持续　在**非持续方法**（nonpersistent method）中，要发送帧的站点侦听线路，如果线路空闲，它就立即发送。如果线路不空闲，站点就等待一个随机时间，然后再次侦听线路。非持续策略可以减少冲突的概率，因为不太可能存在两个或更多个站点等待相同的时间并且再次立即重新尝试。因为当有站点要发送帧而介质却空闲，所以这种方法降低了网络的效率。

p-持续　如果通道有时隙且时隙周期等于或大于最大传播时间时，使用**p-持续方法**（*p*-persistent method）。*p*-持续方法综合了其他两个策略的优点。它既减少了冲突的机会，又提高了网络效率。在此方法中，站点发现线路空闲后它会采取以下步骤：

1. 如果概率是p，站点发送帧。
2. 如果概率是$q=1-p$，站点会等待下一个时隙的开始并再次检查线路。
   a. 如果线路空闲，它将进入步骤1。
   b. 如果线路繁忙，它会当做一个冲突已经发生了并使用补偿程序。

12.1.3　带冲突检测的载波侦听多路访问（CSMA/CD）

CSMA方法没有规定冲突之后的处理程序。带冲突检测的载波侦听多路访问（CSMA/CD）提出了处理冲突的算法。

在这一方法中，任意站点都可以发送帧，然后就监控介质看传送是否成功。如果成功，站点完成发送；如果不成功，就说明存在冲突，需要重新发送此帧。

为了更好地理解CMSA/CD，我们来看看冲突涉及的两个站点发送的第一个位的情况。每个站点都继续发送帧中的其他位直到它检测到冲突，图12.12描述了第一个位发生冲突的情况，冲突的双方是站点A和站点C。

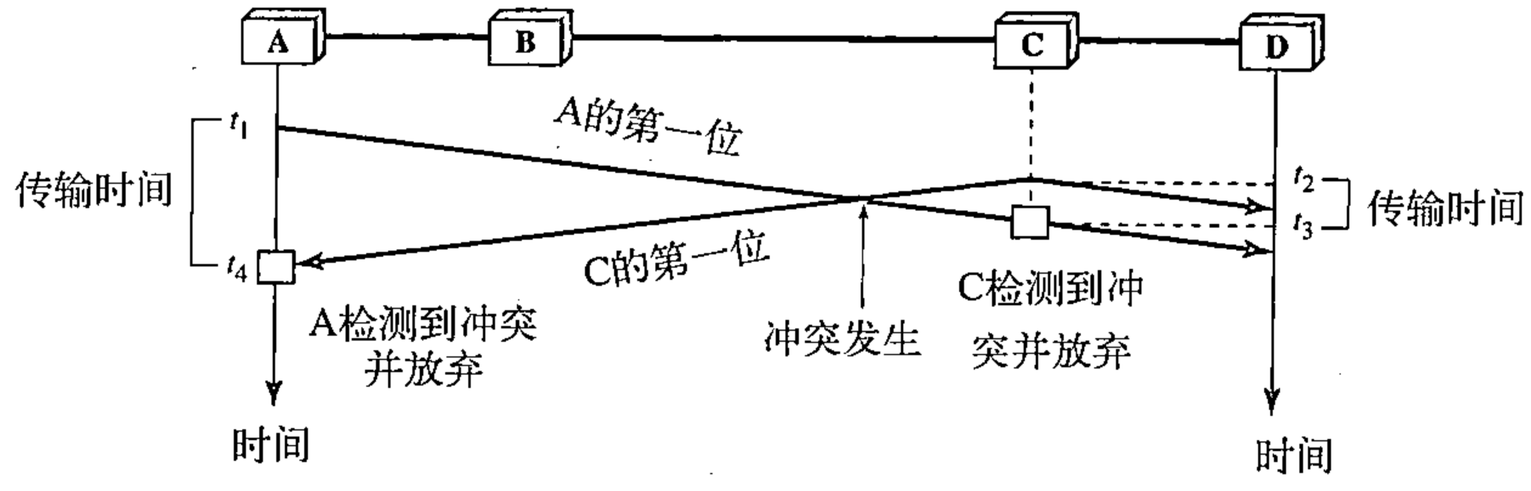


图12.12 在CMSA/CD中第一位冲突的情况

在时刻t_1，站点A执行它的持续程序并开始发送帧。在时刻t_2，站点C还未侦听到A发送的第一个位。站点C执行它的持续程序并开始发送帧，都是从左向右传输的。在时刻t_2后的某一时刻，冲突发生了。在时刻t_3，当站点C收到A的第一个位时，它检测到冲突。站点C马上（或者一小段时间后，但我们假设为马上）放弃了传输。在时刻t_4，当站点A收到C的第一个位时，它检测到冲突，它也同样马上放弃了传输。在图12.12中，我们可以看到站点A进行传输的持续时间是t_4-t_1；站点C进行传输的持续时间是t_3-t_2。随后我们可以看到，因为这个协议，在该协议中任何被比特率所分割的帧的长度都必须大于这些持续的时间段。在时刻t_4，虽然A帧的传输还未完成，但却被放弃了；在时刻t_3，虽然C帧的传输还未完成，但却也被放弃了。

现在我们明白了这两个传输的持续时间，图12.13显示了一张更为完整的图。

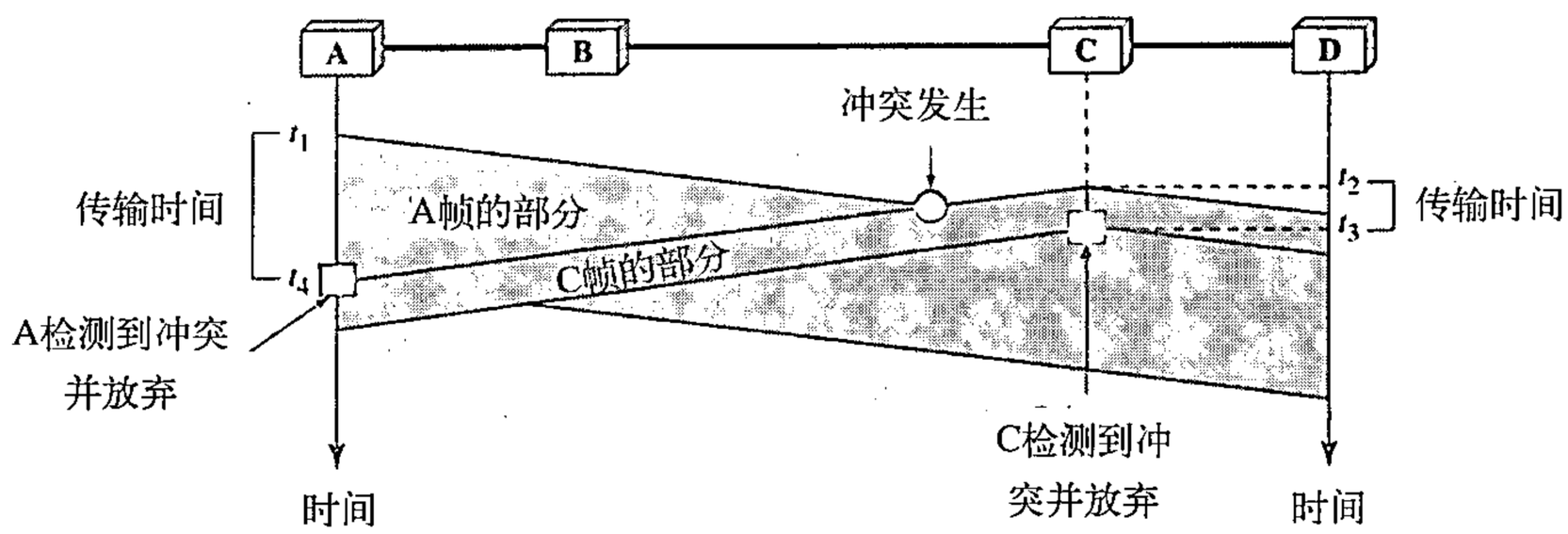


图12.13 在CSMA/CD 中的冲突和放弃传输

帧的最小长度

为了实现CSMA/CD，需要限定帧的长度。在发送帧的最后一位前，发送站点必须检测冲突，如有任何冲突要放弃传输。这是因为一旦整个帧被发送了，站点就无法保留帧的副本并无法掌控线路的冲突检测。因此，帧传输时间T_{fr}必须至少是最大传播时间T_p的两倍。为了理解这个原由，让我们来考虑最差的情况。如果两个站点卷入一个冲突中，且这两个站点距离最远，信号从第一个站点到达第二个站点要花费时间T_p，冲突的作用使得到达第一个站点还需要花费时间T_p。因此，便要求第一个站点在$2T_p$时间后仍在进行传输。

例12.5

使用CMSA/CD的一个网络，带宽是10Mbps。如果最大传播时间（包括设备的延迟并忽略发送一个干扰信号所需的时间）是25.6μs，那么帧的最小长度是多少？

解：

帧的传输时间是$T_{fr}=2\times T_p=51.2\mu s$。这就是说，最差的情况下，一个站点需要传输51.2μs后才能检测到冲突。帧的最小长度是10Mbps × 51.2μs=512比特或64字节。这是标准以太网中帧的最小尺寸，在第13章中会有介绍。

程序

现在让我们看一下图12.14中的CMSA/CD流量图。它与ALOHA协议的流量图相似，但存在着不同。

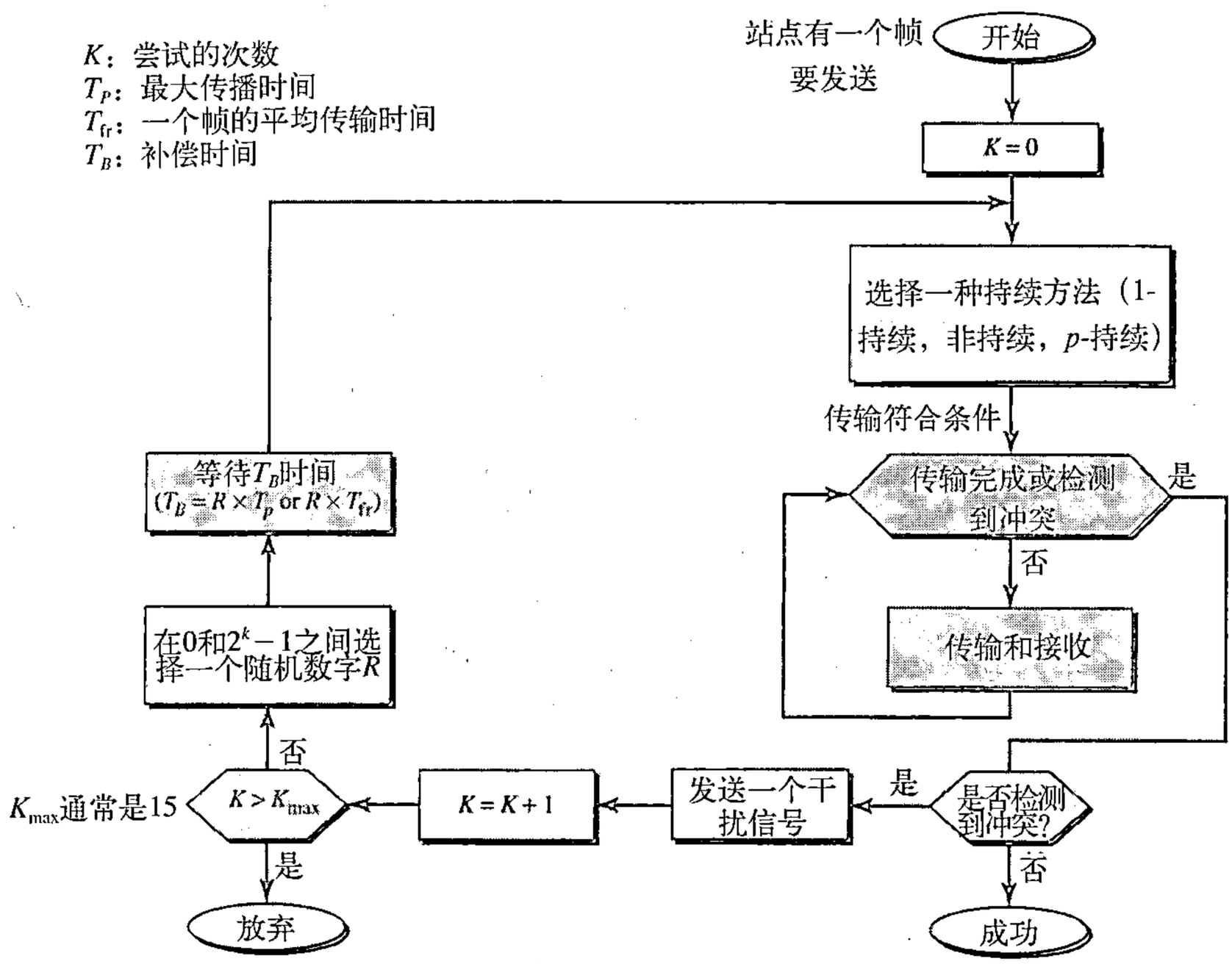


图12.14 CSMA/CD的流量图

第一个不同是增加了持续程序。在开始发送帧之前，我们要使用之前讨论过的持续程序中的一种（1-持续，非持续或者*p*-持续）来侦听通道。图12.11中相关的图形将被持续程序中的一种所代替。

第二个不同是帧的传输。在ALOHA中，我们先传输一个完整的帧，然后等待确认。在CMSA/CD中，传输和冲突检测是一个连续的过程。我们并不是先发送一个完整的帧，然后才检测冲突。站点的传输和接收是连续的和同步的（使用两个不同的端口）。我们用一个回路来显示传输的一个连续的过程，我们不断地进行监控以检测出以下两种情况中的一种：传输完成了或者检测到一个冲突。这两个事件都能使传输停止。如果从回路出来，还是没有检测到冲突，这就意味着传输已经完成了，整个帧被传输了。否则，便有冲突发生了。

第三个不同是万一其他站点仍未检查到冲突，便发送一个短小的**干扰信号**（jamming signal）使冲突加强。

能量级别

通道中的能量级别有三个值：0，正常和不正常。在级别0，通道是空闲的。在正常级别，一个站点成功地占用了通道并正在发送一个帧。在非正常级别，发生了冲突且能量的级别是正常级别的两倍。将要发送或者正在发送帧的站点必须监控能力级别来测定通道是处于空闲，繁忙还是冲突状态。图12.15描述了这个情形。

吞吐量

CSMA/CD的吞吐量大于纯ALOHA或时隙ALOHA。基于不同的持续方法及在*p*-持续方法中不同的*p*值，不同的*G*值将有不同的最大吞吐量。1-持续方法中，当$G=1$时，得到最大吞吐

量约为50%。非持续方法中，当G值在3到8之间时，能得到高达90%的最大吞吐量。

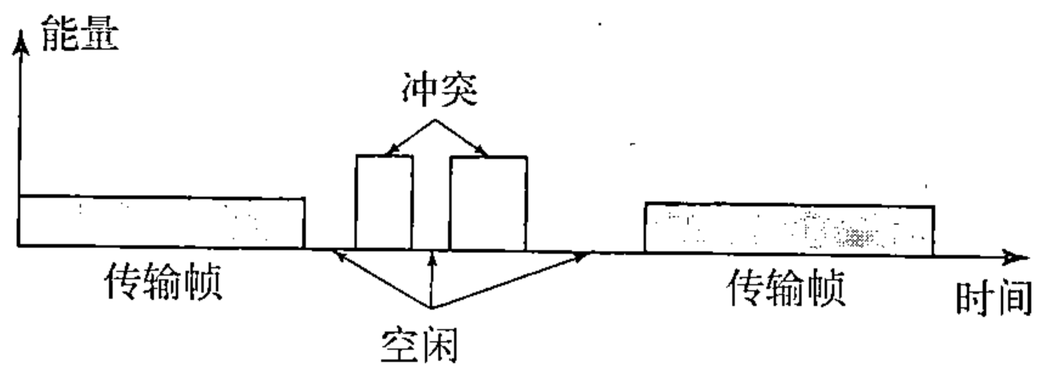


图12.15 传输，空闲或冲突时的能力级别

12.1.4 带冲突避免的载波侦听多路访问（CSMA/CA）

CMSA/CD的基本理念是一个站点在传输过程中检测到冲突时，要有能力去接收信号。当没有冲突时，站点接收到一种信号：它自身的信号。当冲突发生时，站点接收到两种信号：它自身的信号和第二个站点传输的信号。为了区分以上两种情况，不同情况中接收到的信号应该明显不同。换言之，来自第二个站点的信号需要为第一个站点产生的信号增加大量的能量。

在一个有线网络中，接收信号的能量与发送信号的能量基本一样，或者因为线路长度较短，或者因为存在中继器放大了发送方和接收方之间的能量。这就是说在冲突中，被检测到的能量几乎翻倍。

然而，在一个无线网络中，大量发送信号的能量在传输中丢失。接收信号的能量也很小。因此，一个冲突可能只增加5%～10%的额外能量。这对有效的冲突检测起不了什么作用。

因为无线网络的冲突无法检测到，故而我们要避免冲突。带冲突避免的载波侦听多路访问（CSMA/CA）就是为这种网络发明的。通过使用CSMA/CA三种方法来避免冲突：帧间间隔，竞争窗口和确认，如图12.16所示。

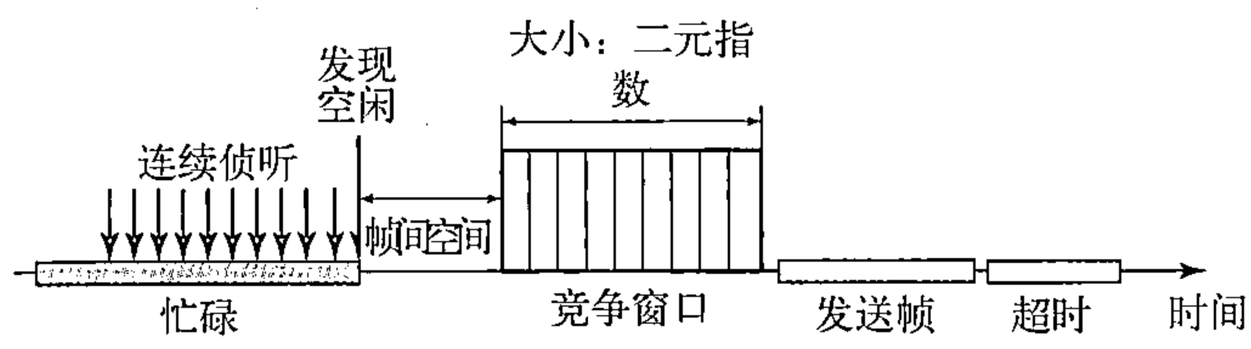


图12.16 CSMA/CA中的时间安排

帧间间隔（IFS）

首先，通过推迟传输来避免冲突，即使通道是空闲的。当发现通道是空闲的时，站点并不马上发送。它会等待一段时间，这段时间称之为**帧间间隔**（interframe space，IFS）。虽然当通道被侦听时，它看上去是空闲的，但是一个远端站点可能早已开始传输了。只是远端站点的信号还未到达这个站点而已。IFS时间使得远端站点传输的信号能到达该站点。如果在IFS时间后，通道仍然是空闲的，那么站点就能发送了，但是它仍然要等待一段时间，这段时间等于竞争时间（以后解释）。IFS变量也能用来区分站点或帧的优先次序。例如，一个站点被赋值的IFS越短，它的优先权越高。

在CSMA/CA中，IFS也能用来定义一个站点或一个帧的优先权。

竞争窗口

竞争窗口将时间分割成时隙。预备发送的站点选择一个随机时隙作为等待的时间。窗口中时隙的数量是根据补偿策略中的二元指数而变化的。这就是说第一次设定一个时隙，以后每次站点在IFS时间后无法检测到空闲通道时，时隙的数量都要翻番。这与p-持续方法很相似，

除了在后者中等待站点根据随机结果来决定时隙的数量以外。竞争窗口中有趣的一点是，每次时隙后站点都要侦听通道。然而，如果站点发现通道繁忙，它并不重启程序，而是停止定时器并当侦听到通道空闲时再重启定时器。这给了站点最长等待时间的优先权。

> 在CSMA/CA中，如果站点发现通道繁忙，它并不重启竞争窗口的时钟，而是停止定时器并当通道空闲时再重启定时器。

确认

虽然用了所有这些防范措施，但是仍然有可能产生冲突而导致帧的损坏。另外，数据也可能在传输中被损坏。肯定的确认和定时器的超时将有助于确保接收方收到帧。

程序

图12.17说明了程序。注意通道在IFS之前和之后都要被侦听。在竞争时间内，通道也要被侦听。在竞争窗口中的每一个时隙内，通道都要被侦听。如果发现通道空闲，定时器继续；如果通道繁忙，定时器停止并在通道空闲后再继续。

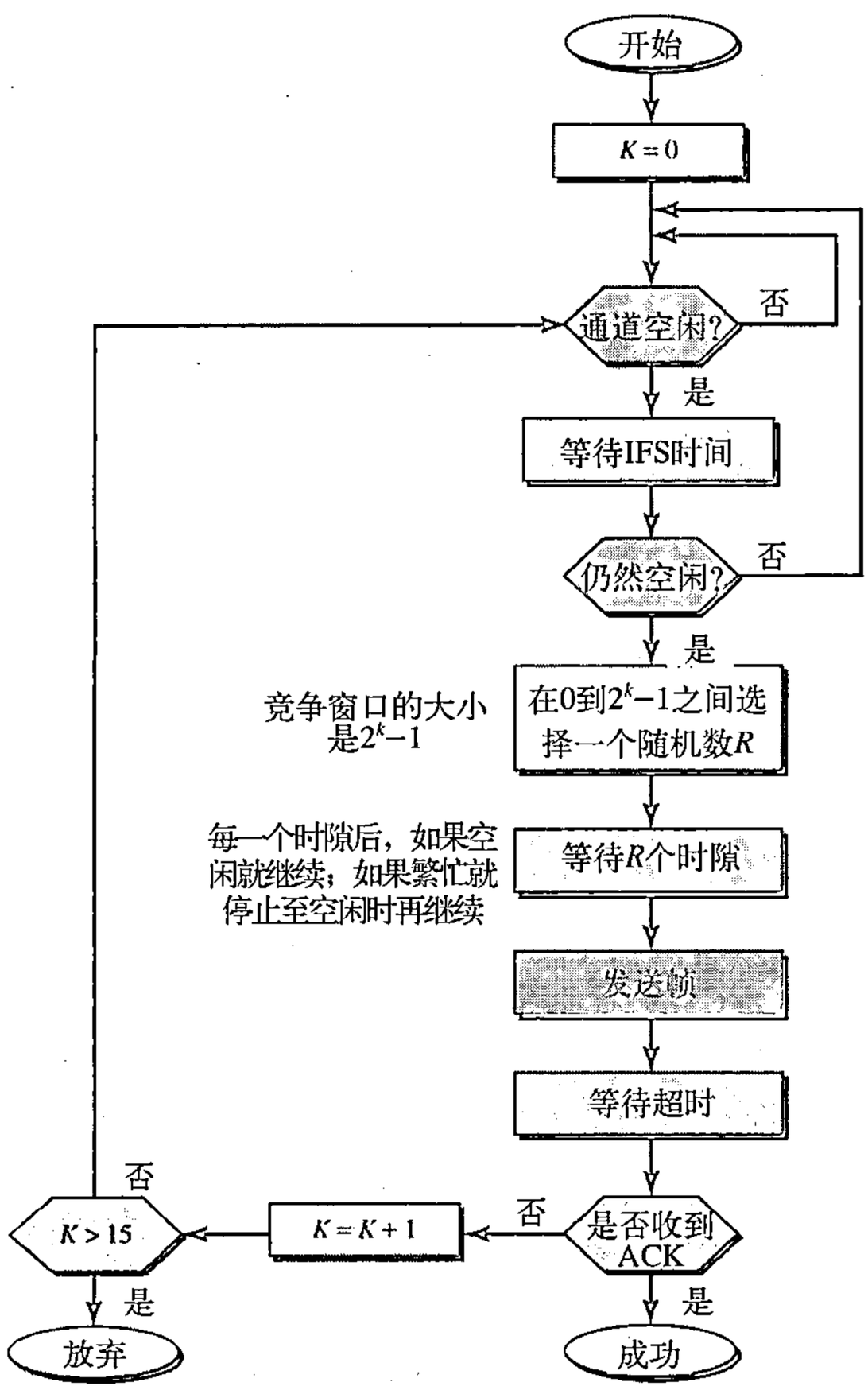


图12.17 CSMA/CA的流量图

CSMA/CA和无线网络

CSMA/CA被广泛地使用在无线网络中。然而，上述的程序还不能处理一些与无线网络相关的特殊问题，比如隐藏的终端或暴露的终端。我们将看到如何通过为上述协议添加握手特

性来解决这些问题。CSMA/CA在无线网络中的应用会在第14章中讨论。

12.2 受控访问

在**受控访问**（controlled access）中，站点之间相互协商以确定哪一个站点有权发送。某一个站点没有得到其他站点的认可就无权发送。下面我们讨论三种流行的受控访问方法。

12.2.1 预约

在**预约**（reservation）方法中，站点在发送数据之前需要预约。将时间划分为时隙，在每一个时隙内，在数据帧之前先发送一个预约帧。

如果系统中有N个站点，在预约帧中就恰好会有N个预约子时隙。每个子时隙均属于一个站点。当一个站点想要发送一个数据帧时，它在自己的子时隙中做一个预约。已经预约完的站点可以在预约帧之后发送数据帧。

图12.18描述了一个有五个站点和一个有五个子时隙的预约帧的情形。在第一个间隙，只有站点1，3和4做了预约。在第二个间隙，只有站点1做了预约。

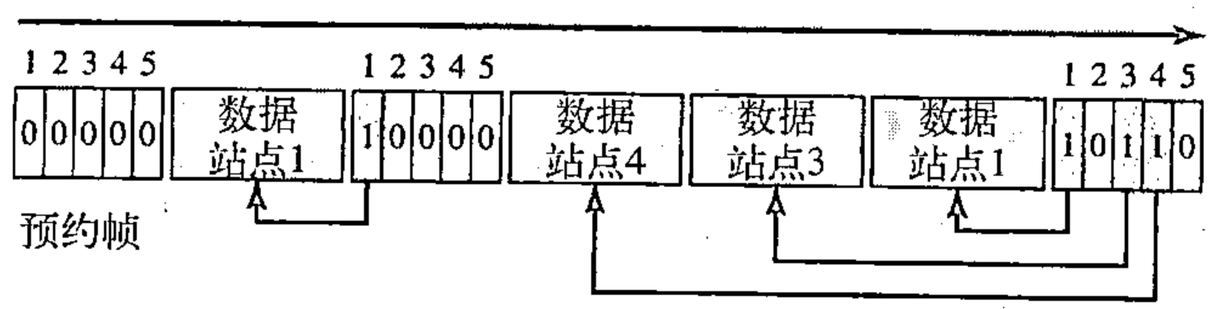


图12.18 预约访问方法

12.2.2 轮询

轮询（polling）方法的拓扑结构是一个设备作为**主站**（primary station），而另一些设备作为**从站**（secondary station）。所有的数据交换都要通过主站点进行，即使最终的目的站点是从站点时也是这样。主设备控制链路，而从设备只是跟随着它的指令工作。主设备决定在给定的时间内哪个设备被容许使用通道。所以，主设备始终是一个会话的发起者。（见图12.19）

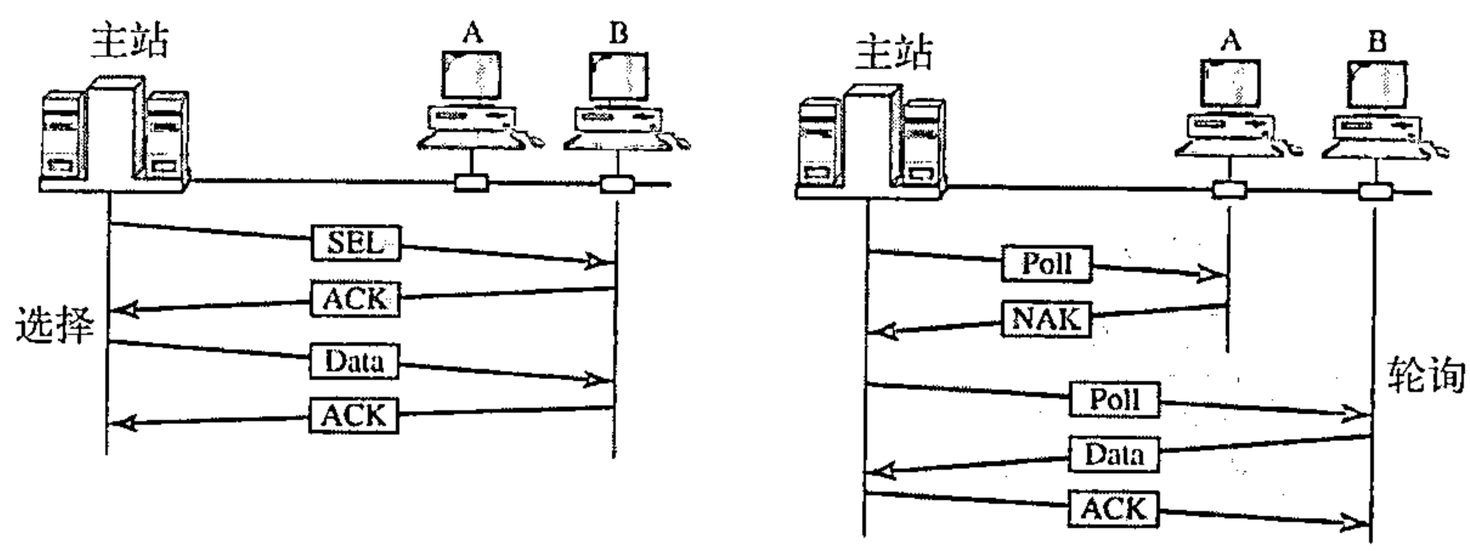


图12.19 在轮询访问方法中的选择和轮询功能

如果主设备希望接收数据，它就询问从设备是否有数据要发送，这一功能称做轮询。如果主设备希望发送数据，它就通知从设备准备接收，这一功能称做选择。

选择

在主设备要发送数据时就使用选择功能。请记住，主设备控制链路。如果主设备既不发送数据也不接收数据，那么它就知道链路是可用的。这时，如果它有数据要发送，主设备就可以发送，但是它不清楚的是，目标设备是否准备接收。所以，主设备必须通知从设备这即将到来的传输，并等待从设备发送它已经处于准备好状态的确认。在发送数据之前，主设备要创建并发送一个选择（SEL）帧，其中的一个字段包含目标从设备的地址。

轮询

主设备使用轮询功能向从设备请求传输。当主设备准备接收数据时，它必须逐个询问每一个设备是否要发送数据。当询问到第一个设备时，如果它没有数据要发送，就用一个NAK帧来响应；如果它要发送数据，就用数据（用数据帧格式）来响应。如果响应的是否认帧(NAK)，主设备就用同样的方法轮询下一个从站点，直到它找到一个要发送数据的站点为止。如果响应是数据帧，主设备就读取该帧并返回一个确认（ACK帧），表示已经接收。

12.2.3 令牌传递

在**令牌传递**（token-passing）方法中，网络中的站点被组织在一个逻辑环中。换言之，对每一个站点来说，都有它的前驱和后继。前驱是指逻辑环中处于该站点之前的站点；而后继是指在环中处于该站点之后的站点。该站点是目前正在进行访问的站点。访问的权限由该站点的前驱站点传递给它。当该站点没有数据再要发送时，便将权限传递给它的后继站点。

但是访问通道的权限如何一个站点一个站点地进行传递呢？在此方法中，一个特殊的信息分组被称为**令牌**（token）在环中循环。令牌的持有赋予站点访问通道并发送数据的权限。当一个站点有一些数据要发送时，它要等待从它的前驱站点传来的令牌。然后，它持有令牌并发送数据。当这个站点没有数据再要发送时，它释放令牌并传递给环中的下一个逻辑站点。站点只有在下一个循环中再次收到令牌时才能再发送数据。在这个过程中，如果一个站点收到了令牌，但是它没有数据要发送，它便将令牌传递给下一个站点。

在这种访问方法中，令牌管理是必需的。站点占有令牌的时间必须得到限制。令牌必须得到监控以确定它没有丢失或损坏。例如，如果一个站点持有令牌失败了，令牌就会在网络中消失。令牌管理的另一个功能是给站点和被传输的数据类型分配优先权。最后，令牌管理能够让低优先权站点将令牌释放给高优先权的站点。

逻辑环

在令牌传递网络中，站点不必要真的连接在一个环中，这个环只是逻辑的。图12.20显示了能创造逻辑环的四种不同的物理拓扑结构。

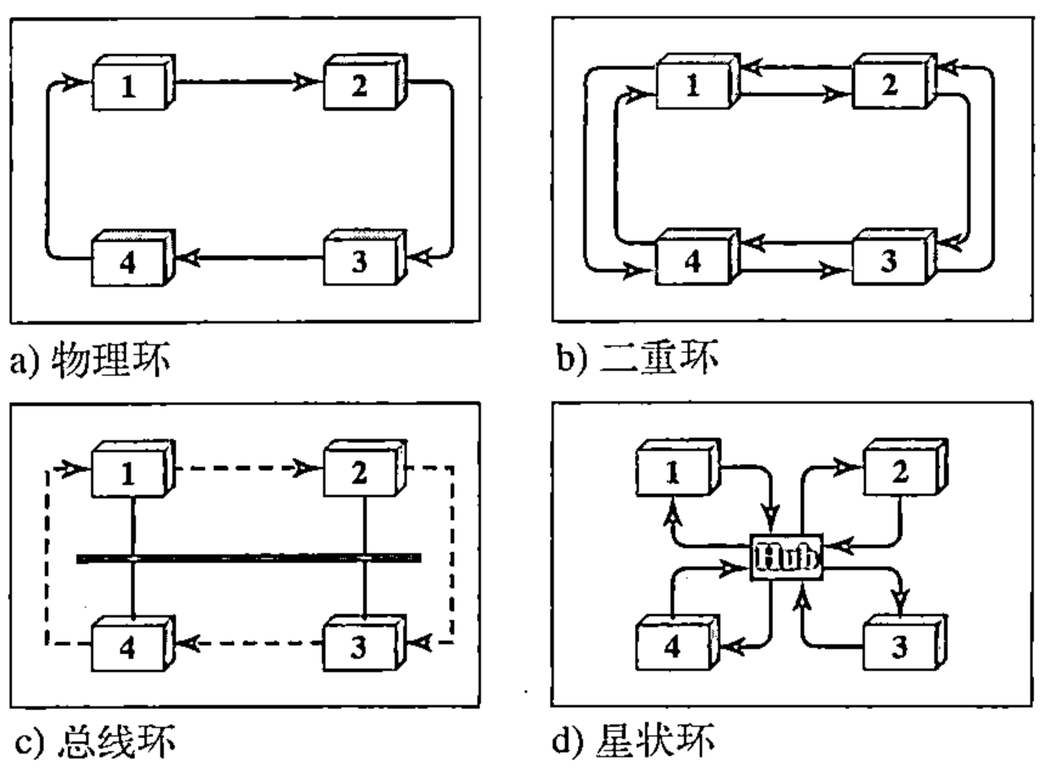


图12.20 在令牌传递访问方法中的逻辑环和物理拓扑结构

在物理环拓扑结构中，当一个站点把令牌传给它的后继站点时，其他站点无法看到令牌；后继站点是线路中的下一个。这意味着令牌不必知道下一个后继站点的地址。这种结构的问题是如果链路中的一个链路（在两个邻近站点之间的介质）出现了故障，整个系统就失败了。

二重环拓扑结构使用了一个二级（辅助的）环，二级环的运行方向与主环的正好相反。二级环只在紧急情况才使用（就像汽车的备用胎一样）。如果在主环链路中的一个链路发生了故障，系统会自动地将两个环合并为一个临时的环。当发生故障的链路修复好后，辅助环又进入空闲状态。注意要运行这种拓扑结构，每个站点都要有两个传输端口和两个接收端口。被称为光纤分布式数据接口（Fiber Distributed Data Interface, FDDI）和铜线分布式数据接口（Copper Distributed Data Interface, CDDI）的高速令牌环网络就使用这个拓扑结构。

总线环拓扑结构，也称为令牌总线。在此拓扑结构中，站点与称为总线的单根光纤连接。然而，它们也组成了一个逻辑环，因为每个站点都知道它后继站点的地址（为了令牌管理的目的，它们同样也知道前驱站点的地址）。当一个站点完成数据发送后，它释放令牌并将后继站点的地址插入令牌中。只有地址与令牌的目的地址相匹配的站点才能访问共享介质。IEEE标准化的令牌总线局域网，就使用这种拓扑结构。

在星状环拓扑结构中，物理拓扑结构是星状的。其中有一个网络集线器充当连接器。网络集线器中的电线形成了环；站点通过两个电线与环相连。这种拓扑结构使得网络不容易发生故障，因为如果一个链路失败了，网络集线器可以忽略它，而其他的站点还是能够运行的。而且从环中增加和减少站点比较方便。IBM设计的令牌环局域网仍在使用这种拓扑结构。

12.3 通道化

通道化（channelization）也是一种多路访问方法。在这一方法中，不同的站点间在时间上、频率上或通过编码来共享链路的可用带宽。在本节我们讨论三种信道化协议：FDMA、TDMA和CDMA。

> 在第16章中，当我们讨论移动电话系统时，将看到所有这些方法的应用。

12.3.1 频分多路访问（FDMA）

在**频分多路访问**（frequency-division multiple access， FDMA）中，可用的带宽被划分成频带，每一个站点都使用分配给它的频带发送数据。换言之，每一个频带都预留给特定的站点使用，该频带永远属于一个站点。每个站点也使用带通滤波器来限定传输器的频率。为了防止站点间的冲突，通过小的防护频带将指定的波段彼此分开。图12.21说明了FDMA的概念。

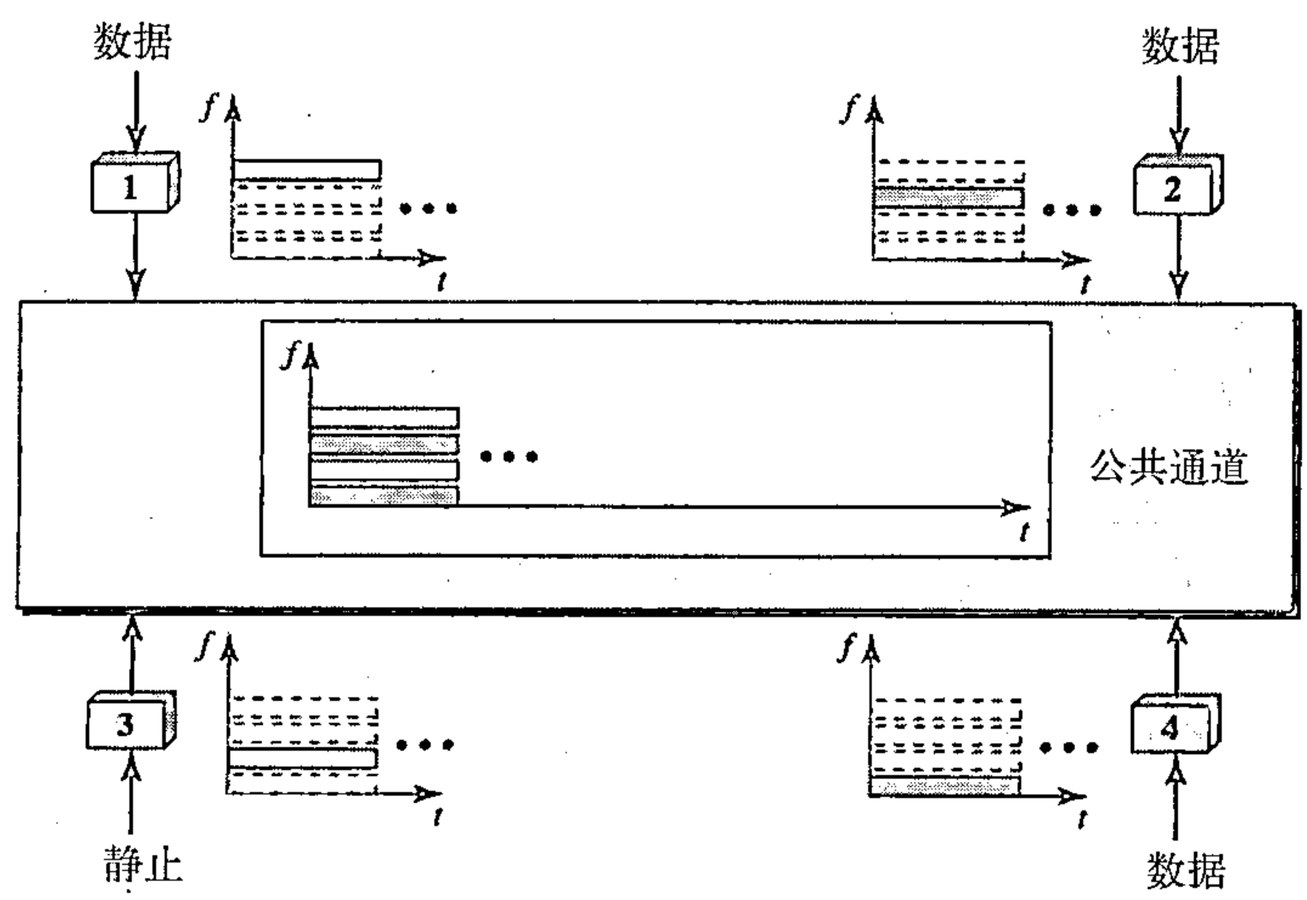


图12.21 频分多路访问（FDMA）

在FDMA中，公共通道的可用带宽被分割成频带并由防护频带将其分离。

FDMA为整个通信期间指定一个预定频率的频带。这就意味着FDMA能方便地使用流数据（无法分组的连续的数据流）。在第16章中，我们将看到移动电话系统是如何利用这一特性的。

我们必须强调的是，虽然FMDA与FDM概念上看似相似，但是两者之间存在区别。第6章介绍的FMD是一种物理层的技术，能组合低带宽通道的负载并使用一个高带宽通道来传输它们。这些被组合的通道是低通。多路复用器调制信号，将它们组合并产生一个带通信号。每个通道的带宽都被多路复用器转换了。

另一方面，FDMA是数据链路层的一种访问方法。每个站点的数据链路层都告知它的物理层把传递给它的数据转换成带通信号。信号必须在被指定的频带中产生。在物理层并没有实际的多路复用器存在。每个站点产生的信号是自动带通过滤的。当它们被发送到公共通道时，它们是混合在一起的。

12.3.2 时分多路访问（TDMA）

在**时分多路访问**（time-division multiple access， TDMA）中， 所有的站点在时间上共享通道的带宽。每一个站点被分配一个时隙，只有在这一时隙内它才能够发送数据。每个站点在指定的时隙中传输数据。图12.22说明了TDMA的概念。

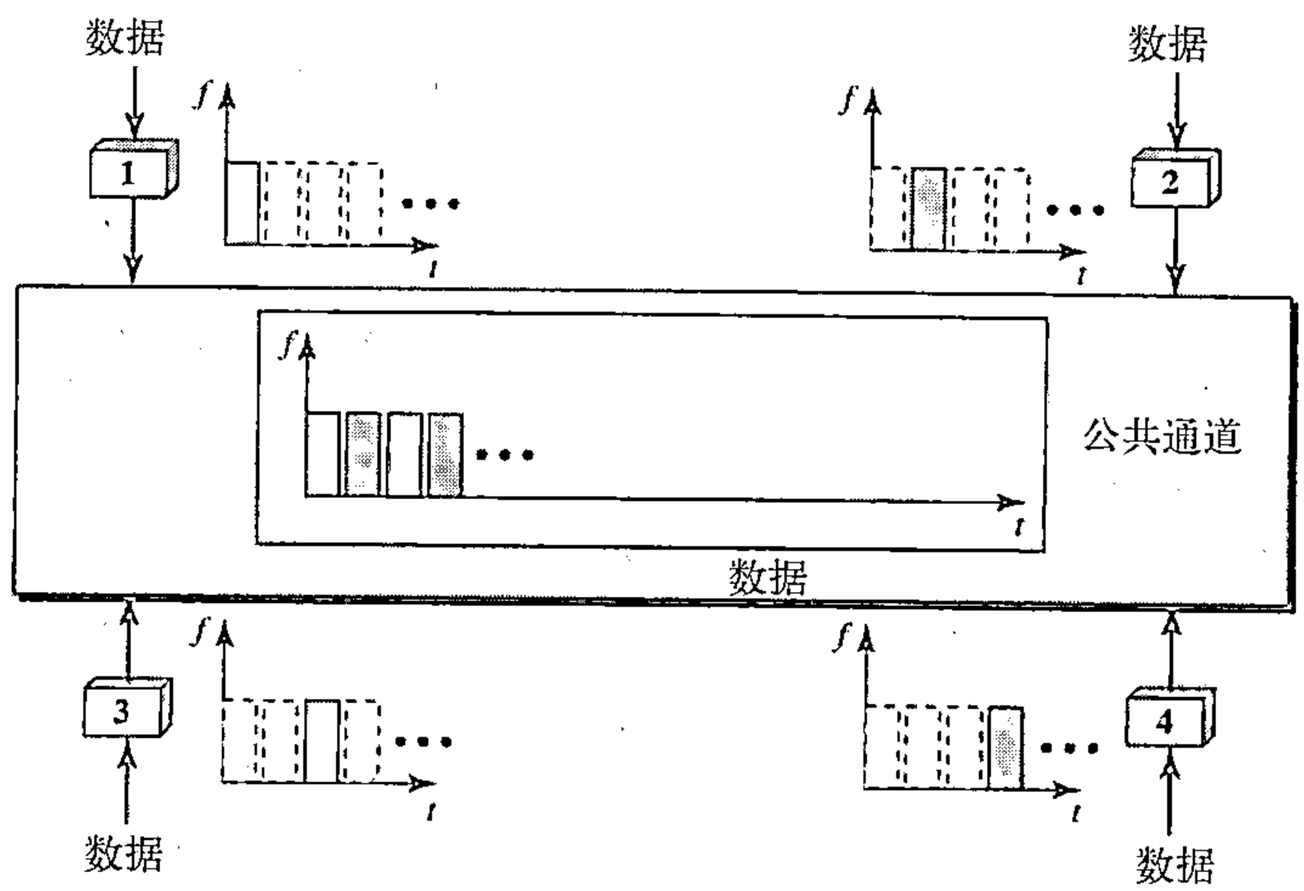


图12.22 时分多路访问（TDMA）

TDMA的主要问题在于如何使不同的站点达到同步。每个站点都要知晓自身时隙的开始点和所处的位置。如果站点分布在一个广泛的区域内，那么系统产生的传播延迟会给前述问题带来困难。为了使延迟得到补偿，我们插入了保护时间。通过在时隙开始点插入同步位（通常称为前导位）来完成同步化。

在TDMA中，带宽被视为是时间上共享的一个通道。

我们同样要强调，虽然TDMA与TDM在概念上看似相似，但它们也存在不同。第6章中介绍的TDM是一种物理层的技术，它将来自低速通道的数据组合起来并使用一个高速通道来传输它们。这个过程使用一个物理多路复用器从每个通道交叉存取数据单元。

另一方面，TDMA是数据链路层的一种访问方法。每个站点的数据链路层都告知它的物理层使用指定的时隙。在物理层并没有实际的多路复用器存在。

12.3.3 码分多路访问（CDMA）

码分多路访问（code-division multiple access，CDMA）的概念在几十年前就已经提出来了。电子技术的新发展使得它的实现成为可能。与FDMA不同，CDMA仅有一个通道占据链路整个的带宽；而与TDMA不同的是，所有的站点都可以同时发送数据，没有时间的共享问题。

> 在CDMA中，一个通道同时承载所有的传输。

类比

我们先来给出一个类比。CDMA简指使用不同的编码来通信。例如，在一个大房间里有许多人，有两个人能用英语通话而其他的人不懂英语。另外两个人能用中文通话，也只有他们两人懂中文，依此类推。换言之，公共通道即该例中的房屋空间，几对人很容易进行沟通，却要使用不同的语言（编码）。

思想

我们假设有四个站点1，2，3和4连接在同一通道上。来自站点1的数据是d_1，站点2的是d_2，依此类推。分配给第一个站点的编码是c_1，第二个的是c_2，依此类推。我们假定被分配的编码有两个特性。

1. 两个编码相乘得0。
2. 编码自身相乘得4（站点的数量）。

通过这两个假想的特性，我们来看看以上四个站点如何使用同一公共通道发送数据，如图12.23所示。

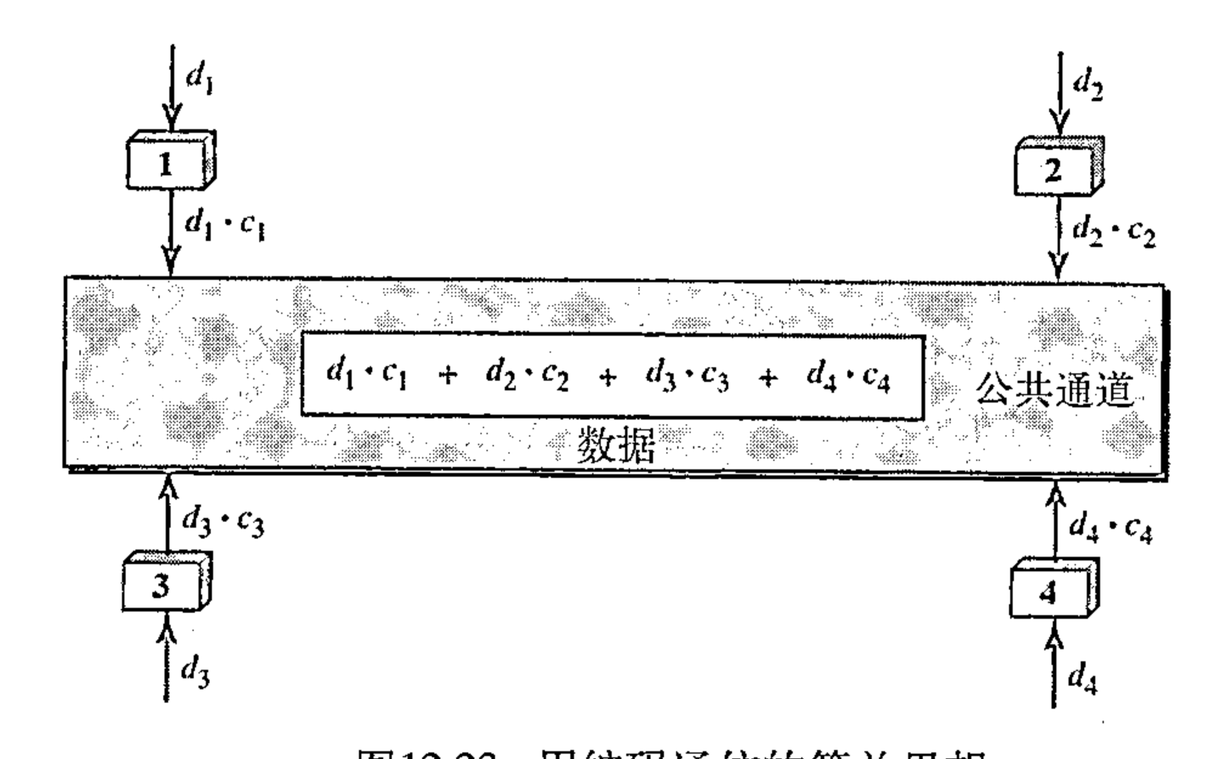


图12.23 用编码通信的简单思想

站点1将它的数据与编码相乘（正如我们将看到的那样，这是一种特殊的乘法）得到$d_1 \times c_1$，站点2将它的数据与编码相乘得到$d_2 \times c_2$，依此类推。通道上的数据是所有这些结果的总和，如图中方块部分所示。任何站点想要接收的数据是发送方的编码和数据的乘积。例如，假定站点1和站点2正在通话。站点2想知道站点1在说什么。它在通道中将数据c_1和站点1的编码d_1相乘。

因为（$c_1 \times c_1$）=4，但（$c_2 \times c_1$）、（$c_3 \times c_1$）和（$c_4 \times c_1$）都为0，站点2为了获取从站点1传来的数据，将结果除以了4。

> 数据$=(d_1 \cdot c_1+d_2 \cdot c_2+d_3 \cdot c_3+d_4 \cdot c_4) \cdot c_1=d_1 \cdot c_1 \cdot c_1+d_2 \cdot c_2 \cdot c_1+d_3 \cdot c_3 \cdot c_1+d_4 \cdot c_4 \cdot c_1=4 \times d_1$

芯片

CDMA基于编码理论。每个站点被指定一个编码，编码是一个被称之为芯片数字序列，如图12.24所示。图中的编码是上一个例子中的。

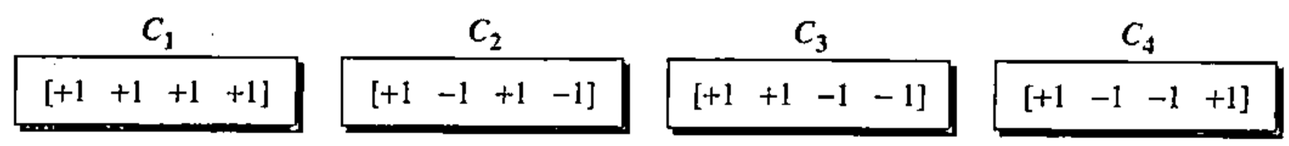


图12.24　芯片序列

本章后面的内容还会介绍如何选择序列。现在，我们需要知道的是：序列的选择并非随机的，它们是被谨慎选择的。它们被称为**正交序列**（orthogonal sequences），有以下的特性：

1. 每个序列有N个元素，而N正是站点的数量。

2. 如果将序列乘以一个数，序列中的每个元素都乘以这个数。这称为序列和标量的乘积。例如，

$$2 \times [+1+1-1-1]=[+2+2-2-2]$$

3. 如果将两个相同序列的元素逐个相乘，并将结果相加，结果为N，而N是每个序列中元素的数量。这就是两个相同序列的内积。例如，

$$[+1+1-1-1] \times [+1+1-1-1]=1+1+1+1=4$$

4. 如果将两个不同序列的元素逐个相乘，并将结果相加，结果为0。这就是两个不同序列的内积。例如，

$$[+1+1-1-1] \times [+1+1+1+1]=1+1-1-1=0$$

5. 将两个序列相应的元素相加，结果是另一个序列。例如，

$$[+1+1-1-1] + [+1+1+1+1]=[+2+2\ \ 0\ \ 0]$$

数据的表示

我们遵循的编码规则：如果一个站点发送0位，编码为－1；如果发送1位，编码为+1。当站点空闲时，它不发送信号，编码为0。如图12.25所示。

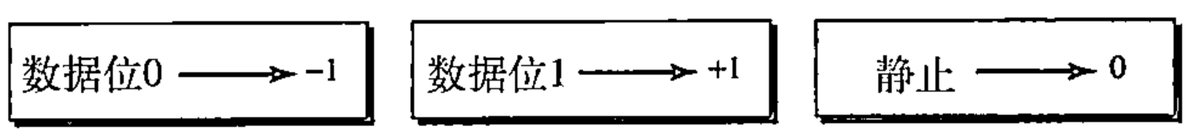


图12.25　CDMA中数据的表示法

编码和解码

作为一个简单的例子，我们来看看四个站点如何共享一个时间间隔为1位的链路。程序容易为附加的时间间隔重复。我们假定站点1和站点2正在发送0位，站点4正发送1位，而站点3是静止的。发送方的数据被依次编码为－1，－1，0和+1。每个站点将相应的数与它的芯片序列（正交序列）相乘，这对每个站点来说都是唯一的。结果是产生一个新的序列并被发送到通道上。为了简化，我们假定所有的站点在同一时点发送结果序列。通道上的序列是所有四个序列的总和。图12.26说明了该种情况。

现在假定静止的站点3正在“听”站点2。站点3将通道上的数据乘以站点2的编码[+1 −1 +1−1]，得到：

$$[-1\ -1\ -3\ +1] \times [+1\ -1\ +1\ -1] = -4/4=-1 \Rightarrow \text{位}1$$

信号水平

如果我们说明了每个站点产生的数字信号及目的地所恢复的数据（见图12.27），便能更

好的理解这个过程。图中显示了每个站点相应的信号（为简化使用了NRZ－L）及在公共通道中的信号。

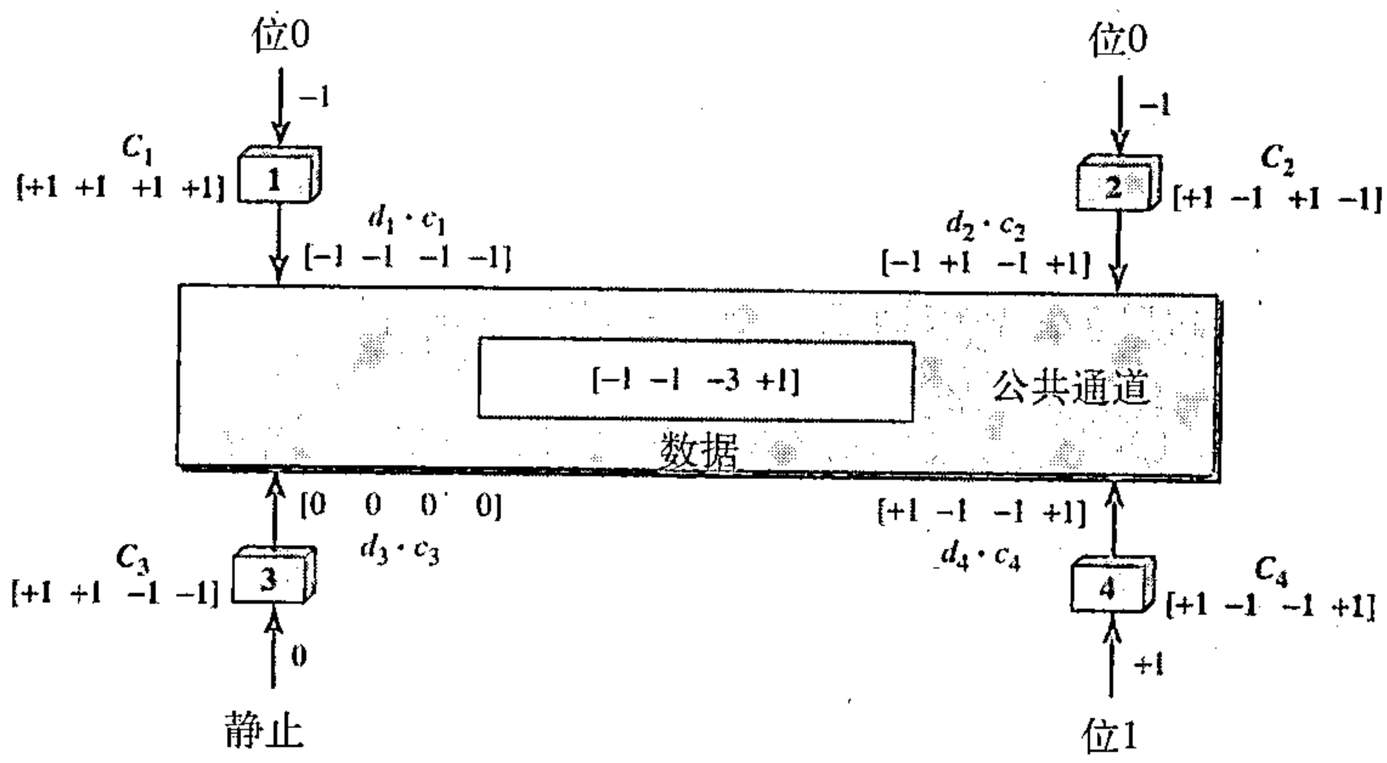


图12.26 CDMA的共享通道

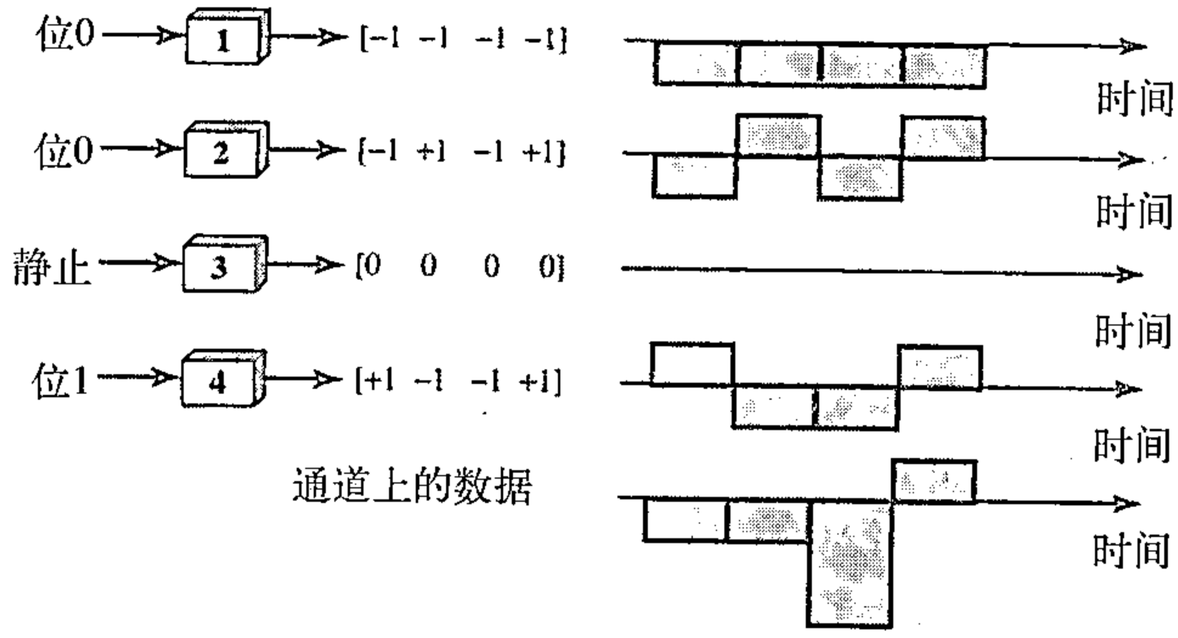


图12.27 在CDMA中四个站点产生的数字信号

图12.28显示了通过使用对站点2的编码，站点3可以检测到来自站点2的数据。通道中的总数据与代表站点2的芯片编码的信号相乘（内积运算）产生一个新的信号。然后，站点将信号下的领域整合并相加，得到的值是－4，－4除以4的结果－1代表位0。

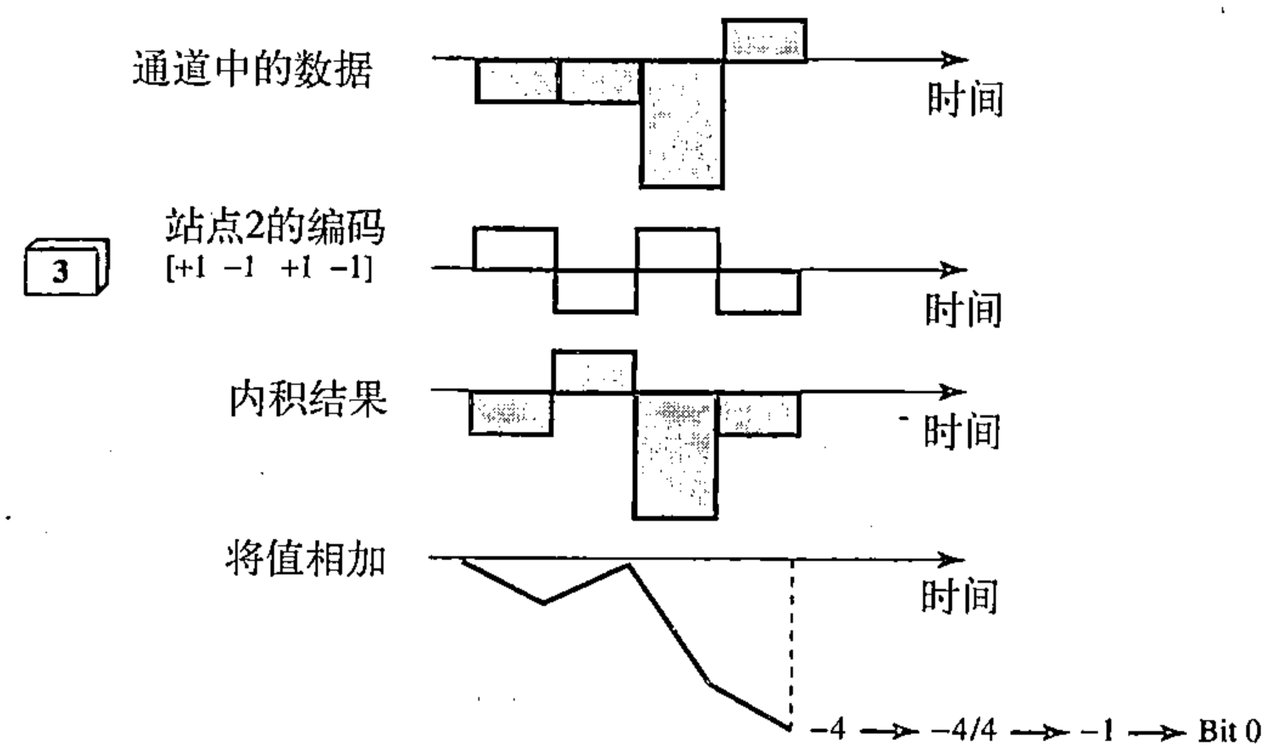


图12.28 CDMA中合成信号的解码

序列的产生

为产生芯片序列，我们使用一张行列数相同的Walsh表，这是一张二维表见图12.29。

在Walsh表中，每一行是一个芯片序列。对于一芯片序列的W_1有一行和一列。我们可以为W_1选+1或−1，这里选+1。根据Walsh，如果我们知道N序列W_N的表，我们就能产生$2N$序列W_{2N}的表，如图12.29所示。带上标线的$\overline{W}_N$代表W_N的反码，这里+1的反码是−1，而−1的反码是+1。图12.29也说明了我们如何从W_1产生W_2和W_4。在我们选择了W_1之后，W_2就能从四个W_1中得出，最后一个是W_1的反码。在W_2产生之后，W_4就能从四个W_2得出，最后一个是W_2的反码。当然，W_8由四个W_4组成，依此类推。注意在W_N产生后，会分配给每一个站点与某一行对应的芯片。

$$W_1 = \begin{bmatrix} +1 \end{bmatrix} \qquad W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & \overline{W_N} \end{bmatrix}$$

a) 两个基本规则

$$W_1 = \begin{bmatrix} +1 \end{bmatrix} \qquad W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} \qquad W_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

b) W_1、W_2和W_4的产生

图12.29 产生Walsh表的通用规则和示例

我们需要强调的是，序列个数N需要是2的幂。换言之，$N=2^m$。

Walsh表中的序列数需要是$N=2^m$。

例12.6

找出以下网络的芯片：

a. 两个站点

b. 四个站点

解：

我们可以使用图12.29中的W_2和W_4。

a. 对于两个站点的网络，我们有[+1 +1]和[+1 −1]。

b. 对于四个站点的网络，我们有[+1 +1 +1 +1]，[+1 −1 +1 −1]，[+1 +1 −1 −1]和[+1 −1 −1 +1]

例12.7

如果在网络中有90个站点，序列个数是多少？

解：

序列个数需要是2^m。我们可以选择$m=7$，$N=2^7$或128。那么我们可以使用90个序列作为芯片。

例12.8

如果用通道中总的数据乘以发送方的芯片编码再除以站点个数，那么可以证明接收站点能够接收特定发送方发送的数据。

解：

让我们使用前面的四个站点的例子证明第一个站点。我们可以说通道上的数据$D=d_1 \cdot c_1+d_2 \cdot c_2+d_3 \cdot c_3+d_4 \cdot c_4$。需要得到站点1发送的数据的接收方用$c_1$乘以这个数据。

$$\begin{aligned} D \cdot c_1 &= (d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4) \cdot c_1 \\ &= d_1 \cdot c_1 \cdot c_1 + d_2 \cdot c_2 \cdot c_1 + d_3 \cdot c_3 \cdot c_1 + d_4 \cdot c_4 \cdot c_1 \\ &= d_1 \times N + d_2 \times 0 + d_3 \times 0 + d_4 \times 0 \\ &= d_1 \times N \end{aligned}$$

当我们用N除以结果，我们就得到d_1。